Pruning and Sparsity in Large Language Models

Weight Sparsity, Structured and Depth Pruning, Expert Pruning, and Activation Sparsity: Theory, Methods, Systems, and Practice

Paper 2 of the series *LLM Optimization and Compression*

A monograph-style survey
literature coverage through October 2026

29 October 2026

Reading note. Scores, forecasts, and rankings in this document are the author's subjective judgments informed by cited evidence. Figures marked with a dagger (†) are quoted from the cited papers as recalled by the author and were not re-checked against the source in the preparation session; figures marked with a double dagger (‡) were checked against paper abstracts or pages during preparation (October 2026); figures marked "illustrative" are synthetic.

Abstract

Pruning removes parameters, structures, or computation from a trained neural network while trying to preserve its function. For large language models (LLMs) the question has changed in character between 2022 and 2026: early work asked whether billion-parameter transformers can be sparsified at all without retraining; current work asks which form of sparsity yields real end-to-end savings on real hardware, at what recovery cost, and at what hidden cost to reasoning, long-context behavior, calibration, safety, and rare knowledge. This monograph surveys the field end to end. It develops the mathematical foundations (sparse approximation, the compressed-sensing viewpoint, the lottery-ticket hypothesis and its status at LLM scale, intrinsic dimension, pruning as combinatorial optimization, ℓ_0 and ℓ_1 relaxations, hard-concrete gates); derives the classical criteria (magnitude, Taylor, Fisher, Optimal Brain Damage/Surgeon) and the closed-form layerwise reconstruction updates behind SparseGPT; analyzes activation-aware scoring (Wanda, RIA); covers N:M semi-structured sparsity and sparse tensor cores, structured width and depth pruning, pruning-plus-distillation pipelines (Minitron, Sheared-LLaMA, Nemotron Elastic), Mixture-of-Experts pruning and merging, attention-head pruning, contextual activation sparsity (Deja Vu, CATS, TEAL, ReLU-fication), sparse training and scaling laws, joint sparsity and quantization, systems issues, and evaluation methodology. It then synthesizes capabilities, pros and cons, risks, a practical decision guide, and an explicit rubric-based priority assessment. The central conclusions are: (i) unstructured weight sparsity is a solved one-shot problem at 50% but largely a systems dead end on GPUs; (ii) structured pruning combined with distillation is the dominant production route to smaller dense models; (iii) activation sparsity and MoE expert pruning are the fastest-moving areas with the best hardware-agnostic prospects for decoding; (iv) depth pruning is cheap but is the most damaging to generative reasoning; and (v) perplexity is an inadequate acceptance test for any pruned model.

Contents

Abstract ii

1 Executive Summary of Findings 1

1.1 Seven findings that organize the field . . . 1
1.2 Bottom-line recommendations . . . 2
1.3 Priority summary . . . 3
1.4 How to read this document . . . 3

2 Introduction and Motivation 5

2.1 Why prune a language model? . . . 5
2.2 A short history, 1989–2026 . . . 6
2.3 Scope, conventions, and notation . . . 7
2.4 A cost model for sparsity . . . 7
2.5 The compressibility landscape in one picture . . . 7
2.6 Organization of the monograph . . . 8

3 Mathematical Foundations 10

3.1 Sparse approximation . . . 10
3.1.1 Best k-term approximation . . . 10
3.1.2 Sparse regression and NP-hardness . . . 11
3.2 The compressed-sensing viewpoint . . . 11
3.2.1 Iterative hard thresholding . . . 12
3.2.2 Convex relaxation . . . 12
3.3 Pruning as combinatorial optimization . . . 12
3.3.1 Loss-based formulation . . . 12
3.3.2 Taylor expansion and saliency . . . 13
3.4 Relaxations: ℓ_0 regularization and hard-concrete gates . . . 14
3.4.1 Straight-through and Gumbel-softmax mask distributions . . . 14
3.5 Over-parameterization, redundancy, and intrinsic dimension . . . 14
3.5.1 Why networks are prunable at all . . . 14
3.5.2 The strong lottery ticket hypothesis . . . 15
3.6 The lottery ticket hypothesis and its status at LLM scale . . . 15
3.7 Error propagation through layers . . . 16
3.8 Sparsity allocation across layers . . . 17

3.9 Summary and a map of the theory to later chapters . . . 17

4 A Taxonomy of Pruning and Sparsity Methods 19

4.1 Five axes . . . 19
4.2 A tree of methods . . . 19
4.3 Comparison table . . . 19
4.4 Terminology disambiguation . . . 20
4.5 Which comparisons are fair . . . 21

5 Classic Criteria and One-Shot Second-Order Pruning 23

5.1 Magnitude pruning . . . 23
5.2 Gradient, Taylor and Fisher importance . . . 23
5.3 Optimal Brain Damage and Optimal Brain Surgeon . . . 24
5.3.1 Optimal Brain Damage . . . 24
5.3.2 Optimal Brain Surgeon . . . 24
5.4 Layerwise reconstruction: OBC and the exact Hessian . . . 25
5.5 SparseGPT . . . 26
5.5.1 Idea 1: a fixed column order and the Cholesky shortcut . . . 26
5.5.2 Idea 2: adaptive mask selection in blocks . . . 26
5.5.3 Idea 3: lazy batch updates . . . 27
5.5.4 What SparseGPT assumes and where it fails . . . 28
5.6 Optimal updates for a fixed mask and ADMM refinement . . . 28
5.7 Practical details that matter . . . 29
5.8 Summary of second-order pruning . . . 29

6 Activation-Aware Scoring, Layerwise Allocation and Mask Refinement 30

6.1 Wanda: weights and activations . . . 30
6.1.1 Why the Wanda score is a diagonal reconstruction criterion . . . 30
6.1.2 Results and where Wanda is weaker . . . 31
6.2 RIA and relatives: relative importance and channel permutation . . . 32
6.3 Gradient-informed and learned scores . . . 32
6.4 Non-uniform layerwise sparsity . . . 33
6.5 DSnoT and training-free mask refinement . . . 34
6.6 Relationship to activation-aware quantization . . . 34
6.7 Summary . . . 35

7 Semi-Structured N:M Sparsity and Sparse Tensor Cores 36

7.1 The 2:4 format and its hardware mechanics . . . 36

7.1.1 Compressed representation . . . 36
7.1.2 Sparse matrix-multiply mechanics . . . 36
7.2 Accuracy: why 2:4 is harder than unstructured 50% . . . 38
7.2.1 Counting masks . . . 38
7.2.2 Reasoning and generative tasks . . . 38
7.3 Post-training 2:4 methods . . . 38
7.4 Learning the mask: SR-STE, transposable masks, MaskLLM . . . 39
7.4.1 Sparse-refined straight-through estimator (SR-STE) . . . 39
7.4.2 Transposable masks for training . . . 39
7.4.3 MaskLLM . . . 39
7.5 Beyond 2:4: milder patterns, hybrids, and activation sparsity . . . 40
7.5.1 (2N-2):2N patterns and sliding windows . . . 40
7.5.2 Hybrid sparse–dense formats . . . 41
7.5.3 2:4 on activations . . . 41
7.6 Software stack . . . 41
7.7 A realism check on speedups . . . 41

8 Structured Width Pruning: Channels, Heads, Hidden Dimensions 43
8.1 Structures, couplings and parameter accounting . . . 43
8.1.1 Coupled structures . . . 43
8.1.2 Parameter accounting . . . 43
8.2 LLM-Pruner . . . 44
8.3 SliceGPT: pruning after a change of basis . . . 45
8.3.1 Computational invariance . . . 45
8.3.2 Choosing Q by PCA and slicing . . . 45
8.4 FLAP: fluctuation-based adaptive structured pruning . . . 46
8.5 Gradient-free and learned structured pruning . . . 47
8.6 Sheared-LLaMA: targeted structured pruning plus continued pre-training . . . 47
8.6.1 Targeted structured pruning . . . 47
8.6.2 Dynamic batch loading . . . 48
8.7 Recent structured-pruning methods (2025–2026) . . . 48
8.8 Comparison of width-pruning methods . . . 49
8.9 Design rules extracted from the literature . . . 49

9 Depth Pruning, Layer Dropping and Layer Skipping 51
9.1 The residual-stream view and why layers can be redundant . . . 51
9.1.1 Block influence . . . 51

9.1.2 Angular distance and the Gromov et al. finding . . . 52
9.2 Methods for static layer removal . . . 53
9.3 What depth pruning does to generative reasoning . . . 54
9.4 Healing and recovery . . . 54
9.5 Dynamic depth: early exit and layer skipping . . . 54
9.6 Latency arithmetic and when depth pruning pays . . . 55

10 Pruning plus Distillation: Compressing Large Models into Families 57
10.1 Knowledge distillation for LM recovery . . . 57
10.1.1 Objective . . . 57
10.2 The Minitron recipe . . . 58
10.2.1 Importance estimation from activations . . . 58
10.2.2 Pipeline . . . 58
10.2.3 Reported findings and lessons . . . 58
10.3 From Minitron to many-in-one: Nano 2, Elastic, Puzzle . . . 59
10.4 Small models released through pruning . . . 60
10.5 Cost arithmetic: when is pruning plus distillation cheaper? . . . 61
10.6 Limits and failure modes . . . 61

11 Mixture-of-Experts Pruning and Merging; Attention-Head Pruning 63
11.1 MoE basics and why experts are a compression target . . . 63
11.2 Expert importance and pruning . . . 63
11.2.1 Frequency, gate and activation statistics . . . 63
11.2.2 Derivation of a router-weighted expert score . . . 64
11.2.3 Exhaustive reconstruction search and skipping . . . 64
11.2.4 Task-specific pruning . . . 65
11.3 Merging experts . . . 65
11.3.1 Why merging may be intrinsically lossy . . . 65
11.3.2 Reported scale of results . . . 66
11.3.3 Systems consequences . . . 67
11.4 Attention-head pruning and redundancy . . . 67
11.4.1 Classical findings . . . 67
11.4.2 Heads in LLMs: GQA, retrieval heads, streaming heads . . . 67

12 Contextual and Activation Sparsity 69
12.1 Definitions and sources of sparsity . . . 69
12.2 Thresholding theory: which activations may be skipped? . . . 69

12.2.1 Optimal skipping under an error bound . . . 69
12.2.2 Thresholds from the activation distribution . . . 70
12.3 Deja Vu: predicting contextual sparsity . . . 70
12.4 Training-free thresholding: CATS, TEAL, LaRoSA . . . 71
12.5 Inducing sparsity: ReLU-fication and related training . . . 72
12.6 Systems: kernels, batching and offloading . . . 73
12.6.1 Sparse GEMV . . . 73
12.6.2 The batching problem . . . 73
12.7 Positioning relative to other compression families . . . 75

13 Sparse Training, Scaling Laws, and Sparsity with Quantization 76
13.1 Dynamic sparse training . . . 76
13.1.1 The SET–RigL lineage . . . 76
13.1.2 Sparse training at LLM scale . . . 77
13.2 Scaling laws with sparsity . . . 78
13.2.1 Dense baseline . . . 78
13.2.2 Sparse extensions . . . 78
13.3 Prune a large model or train a small one? . . . 78
13.4 Sparse fine-tuning and recovery . . . 79
13.5 Joint sparsity and quantization . . . 80
13.5.1 Shared machinery . . . 80
13.5.2 Storage arithmetic: effective bits per original weight . . . 80
13.5.3 Rate–distortion view: is sparsity ever the right way to spend bits? . . . 80

14 Systems: Kernels, Formats, and the No-Speedup Trap 82
14.1 The roofline view of LLM inference . . . 82
14.2 Index overhead and break-even density . . . 82
14.2.1 Formats . . . 82
14.3 Why unstructured sparsity rarely speeds up GPUs . . . 83
14.3.1 Load-as-sparse, compute-as-dense . . . 84
14.4 Block sparsity, N:M, and the flexibility–efficiency spectrum . . . 84
14.5 CPUs, edge devices and accelerators . . . 84
14.6 The KV cache: what weight pruning does not touch . . . 85
14.7 Serving-engine integration . . . 86
14.8 Measurement checklist . . . 86

15 Evaluation Methodology and What Pruning Removes 88

15.1 Perplexity: what it measures and what it hides . . . 88
15.2 Downstream benchmarks and their blind spots . . . 88
15.3 Calibration-data dependence . . . 89
15.4 What does pruning remove? . . . 90
15.4.1 Knowledge capacity . . . 90
15.4.2 Evidence on tails . . . 91
15.4.3 Knowledge versus skill . . . 91
15.4.4 Side effects: safety, alignment, fairness, calibration . . . 91
15.5 A recommended evaluation protocol . . . 92

16 Recent Advances Timeline (2022–October 2026) and Current Capabilities 94
16.1 Timeline . . . 94
16.1.1 Trends visible in the timeline . . . 94
16.2 What is achievable today . . . 96
16.2.1 Weight sparsity: levels and quality . . . 96
16.2.2 Structured and distilled models . . . 96
16.2.3 MoE expert compression . . . 96
16.2.4 Real speedups . . . 96
16.2.5 Rules of thumb . . . 97

17 Pros, Cons, and Risks 99
17.1 Advantages and disadvantages by family . . . 99
17.2 Risk register . . . 99
17.2.1 R1: Accuracy cliffs . . . 99
17.2.2 R2: Compounding of small per-token errors in generation . . . 99
17.2.3 R3: Brittleness . . . 99
17.2.4 R4: Calibration-set bias . . . 100
17.2.5 R5: Safety, alignment, fairness side effects . . . 100
17.2.6 R6: Loss of rare knowledge and tail behavior . . . 101
17.2.7 R7: Recovery cost . . . 101
17.2.8 R8: Hardware and software dependence . . . 101
17.2.9 R9: The no-speedup trap . . . 101
17.2.10 R10: Reproducibility . . . 102
17.2.11 R11: Security and provenance . . . 102

18 Practical Decision Guide 104
18.1 Flowchart . . . 104

18.2 Decision tables . . . 104
18.2.1 By primary constraint . . . 104
18.2.2 By workload . . . 104
18.2.3 By hardware . . . 104
18.3 Recipe cards . . . 104
18.4 Pre-deployment checklist . . . 107

19 Estimated Priority in the LLM Engineering Community 109
19.1 Rubric . . . 109
19.2 Scoring rationale . . . 110
19.3 Scores and final ranking . . . 111
19.3.1 Final ranking with confidence . . . 111
19.4 Interpreting and stress-testing the ranking . . . 113

20 Open Problems, Research Directions, and Conclusions 114
20.1 Open problems . . . 114
20.1.1 P1. Reasoning-preserving compression . . . 114
20.1.2 P2. Hardware–pattern co-design . . . 114
20.1.3 P3. Theory and practice of calibration . . . 114
20.1.4 P4. Predicting the accuracy cliff . . . 114
20.1.5 P5. Lottery tickets at LLM scale . . . 115
20.1.6 P6. Controlled comparison of routes to a small model . . . 115
20.1.7 P7. MoE compression . . . 115
20.1.8 P8. Activation sparsity beyond batch size one . . . 115
20.1.9 P9. Unified compression of weights and KV cache . . . 115
20.1.10 P10. Safety, alignment and security of pruned models . . . 115
20.1.11 P11. Non-transformer and multimodal models . . . 116
20.1.12 P12. Standards for evaluation and reporting . . . 116
20.1.13 P13. Sparse and elastic training at scale . . . 116
20.1.14 P14. Better theory for second-order and global pruning . . . 116
20.2 Conclusions . . . 116

A Notation 118

B Additional Derivations 119
B.1 Lazy block updates equal sequential updates . . . 119
B.2 Closed-form expected ℓ_0 of a hard-concrete gate . . . 119
B.3 Fisher information equals the Hessian of the negative log-likelihood . . . 119

B.4 Truncated second moments used in Table 3.1 120
B.5 Expected distinct experts and batch union 120
B.6 Average parameter count under cubic pruning 120
B.7 Amdahl-style end-to-end speedup 120
B.8 Why the $(2N-2)$:$2N$ decomposition has speedup bound $N/(N-1)$ 120
B.9 Merging two experts 121

C Glossary **122**

Bibliography **124**

Chapter 1

Executive Summary of Findings

This monograph is a survey of pruning and sparsity techniques for large language models (LLMs), written with a deliberately practical orientation: for each family of methods we ask what the theory says, what the algorithm does, what has been measured, and what an engineer who must ship a model can reasonably expect. The body of the text runs to twenty chapters; this chapter compresses the main findings so that a reader with limited time can decide which chapters to read.

1.1 Seven findings that organize the field

Finding 1: One-shot weight sparsity at 50% is solved; beyond that it is not

Post-training, calibration-based methods that use second-order layerwise reconstruction (SparseGPT) or a cheap activation-aware score (Wanda, RIA) can remove about half of the weights of models from 7B to 175B parameters with a modest perplexity increase and no gradient-based retraining [42, 44, 45]. Above roughly 60–70% unstructured sparsity, quality degrades sharply for all one-shot methods, and layerwise sparsity allocation (OWL, AlphaPruning) and mask refinement (DSnoT, ADMM- and ALPS-style solvers) matter more than the base criterion [46, 47, 50, 51, 54]. Perplexity remains misleadingly benign relative to generative reasoning at the same sparsity (Chapters 5, 6, 15).

Finding 2: Unstructured sparsity rarely gives wall-clock speedups on GPUs

Memory-bound decoding could in principle profit from sparse weights, but index overhead, irregular access, loss of tensor-core utilization, and the efficiency of modern low-bit dense kernels combine to make unstructured sparsity uncompetitive with 4-bit quantization at the same memory footprint on mainstream GPUs. The one hardware-supported pattern, 2:4 semi-structured sparsity on NVIDIA sparse tensor cores, gives a 2× peak math throughput but at 50% sparsity costs accuracy, particularly for generative reasoning; reported end-to-end speedups are modest (for example, up to 1.2× decoding on a B200 for FP8 with a hybrid 2:4/dense format‡ [65]) (Chapters 7, 14).

Finding 3: Structured pruning plus distillation is the dominant production route

Removing whole channels, heads, neurons, or layers yields smaller dense models that run on every stack. Pure post-training structured pruning degrades quickly beyond 20–30% parameter removal, but combined with a distillation phase of tens to a few hundred billion tokens it produces competitive small models at a small fraction of from-scratch cost: Minitron (Llama 3.1 8B→4B, Mistral NeMo 12B→8B), Sheared-LLaMA, the Nemotron Nano 2 9B derived from a 12B parent, and Nemotron Elastic, which reports extracting 9B and 6B nested models from a 12B parent with 110B tokens‡ and a claimed 360× cost reduction over training each size from scratch [75, 111–113, 115]. Meta's Llama 3.2 1B and

3B were reported by Meta to use pruning plus distillation from larger Llama 3.1 models [117] (Chapters 8, 10).

Finding 4: Depth pruning is cheap but costs generative ability

Many consecutive deep layers of pretrained transformers can be removed with small changes in multiple-choice accuracy [94, 95], which makes layer pruning attractive. However, generative reasoning, arithmetic, and long-form coherence degrade much more and recover poorly: a 2026 study reports that even about 100B tokens of post-pruning training leaves deficits on simple arithmetic‡ [101] (Chapter 9).

Finding 5: Mixture-of-Experts pruning and activation sparsity are the fastest-moving areas

For MoE models, whose total parameter count dominates memory, one-shot expert pruning and merging guided by router statistics (REAP, REAM, and earlier work) can reduce expert counts by 25–50% with small losses on code and chat benchmarks: REAP reports near-lossless code generation for Qwen3-Coder-480B and Kimi-K2 at 50% expert pruning‡ [131, 132]. For dense models, training-free activation sparsity (TEAL, LaRoSA) offers 40–50% activation sparsity at small quality cost with decoding speedups of roughly 1.3–1.8× in the kernels of the respective papers‡ [160, 161], and composes with weight quantization (Chapters 11, 12).

Finding 6: Scaling laws favor sparsity only in specific regimes

Pruning a large model is usually superior to training a small dense model from scratch at the same token budget, but if one can afford a full training run, a dense model of the target size is a strong baseline. Sparse pre-training can match dense pre-training at equal compute and yields smaller models when pruning begins around 25% and ends around 75% of the compute budget‡ [171]; under data scarcity, moderate sparsity can improve loss-optimality‡ [174] (Chapter 13).

Finding 7: Perplexity is not an acceptable acceptance test

Calibration-data choice, task type (knowledge retrieval versus multi-step generation), long-context behavior, calibration of probabilities, safety refusals, and fairness all respond differently to pruning than perplexity does. A 2026 study of Wanda found only a 3.5% perplexity increase at 50% sparsity for Mistral-7B yet an 83.7% increase in a stereotype-reliance score‡ [187]; earlier benchmarks (LLM-KICK) showed knowledge-intensive tasks falling off well before perplexity [180]; and reasoning models pruned with calibration on input text alone may produce longer chains and worse accuracy, motivating on-policy reconstruction data [121] (Chapters 15, 17).

1.2 Bottom-line recommendations

1. If you need a *smaller dense model for any hardware*, use structured width pruning (with attention to GQA group structure) plus logit distillation from the unpruned teacher; budget at least several billion and ideally tens of billions of tokens. This is the most reliable route.

2. If you serve a *very large MoE model* and are memory-bound, evaluate router-aware expert pruning or merging, calibrated on a mix of the domains you care about (including generative samples), before considering anything more exotic.

3. If you run *decode-heavy dense inference* and can adopt custom kernels, test training-free activation sparsity (TEAL-style) on top of your quantized model; it avoids retraining and preserves the weights.

4. Treat *unstructured weight pruning* as a research tool or as a storage-compression step for hardware that can exploit it (CPUs with sparse kernels, some accelerators); do not expect GPU speedups.

5. Treat 2:4 *sparsity* as attractive only when (a) you can afford sparse fine-tuning or learned masks, (b) your stack uses cuSPARSELt-class kernels, and (c) you have validated generative tasks, not only perplexity.

6. Use *depth pruning* for fast, low-cost shrinkage of classification-style or retrieval-style workloads and as a throughput lever for latency-bound deployments, but validate multi-step reasoning, code, and arithmetic explicitly.

7. In all cases: re-run the full evaluation suite, including safety, calibration, long context, and domain tails, after pruning.

1.3 Priority summary

Chapter 19 scores twelve sub-topics on a stated five-criterion rubric (production adoption, research momentum, hardware tailwind, marginal benefit left, risk-adjusted reliability). With the explicit caveat that the scores are the author's subjective judgments informed by cited evidence and not measured data, the ranking is: (1) structured width pruning with distillation, robustly first under every weighting tested (confidence: high); (2) MoE expert pruning, merging and skipping (medium); (3) training-free contextual activation sparsity (medium); (4) depth pruning (medium); (5–7) head-level methods, joint sparsity–quantization and $N{:}M/2{:}4$ sparsity, a statistical tie in the middle; (8–10) post-training structured pruning without distillation, dynamic depth and induced activation sparsity; and (11–12) sparse training and one-shot unstructured pruning as *deployment* priorities (although the latter remains a highly influential research substrate). Evaluation and trustworthiness methodology is treated as a cross-cutting enabler that would rank second if scored as a sub-topic.

1.4 How to read this document

A reader mainly interested in decisions can read Chapters 1, 16, 17, 18 and 19. A reader implementing methods should read Chapters 3–12 and Appendix B. A researcher looking for gaps should start with Chapter 20. Throughout, figures marked with † were quoted from papers as recalled by the author without re-verification in the preparation session, figures marked with ‡ were verified against abstracts or paper pages in October 2026, and any plot described as illustrative is synthetic and not experimental data.

Scope and epistemic status

The field moved quickly during 2025–2026, and several works cited here are preprints that may have been revised after the versions consulted. Where the text gives a number without a mark, it is either a definitional or arithmetic quantity computed in the text, or a typical order of magnitude flagged as such. No benchmark number in this document was produced by the author's own experiments.

Chapter 2

Introduction and Motivation

2.1 Why prune a language model?

A transformer language model of P parameters stored at b bits per parameter occupies $Pb/8$ bytes. A 70-billion-parameter model in 16-bit precision therefore needs 140 GB for weights alone, before the key–value cache and activations are counted. Two quantities govern the cost of serving such a model: the memory footprint, which decides how many accelerators are required, and the memory bandwidth consumed per generated token, which decides decoding latency. During autoregressive decoding at small batch size, every generated token requires reading essentially all active weights once from device memory, performing roughly $2P$ floating-point operations per token, so the arithmetic intensity is about one floating-point operation per byte for 16-bit weights, far below the ridge point of any modern accelerator. Decoding is thus *memory-bandwidth-bound*, and a rough upper bound on single-stream decoding speed is

$$\text{tokens/s} \lesssim \frac{B_{\text{mem}}}{\text{bytes of weights read per token}}, \tag{2.1}$$

with B_{mem} the achieved memory bandwidth.

Example 2.1 (Back-of-envelope decoding bound; illustrative arithmetic)**.** Take a dense 70B model in bf16 (140 GB) and a GPU with roughly 3.3 TB/s of memory bandwidth (the order of magnitude of current high-end accelerators). Equation (2.1) gives about $3.3 \times 10^{12} / 1.4 \times 10^{11} \approx 24$ tokens/s if the whole model could be held on one device (it cannot; 140 GB exceeds the capacity of a single 80 GB device, so in practice the weights are sharded across devices, and the aggregate bandwidth is then multiplied accordingly). Halving the number of bytes read—by 4-bit quantization of all weights, or by removing half of the weights in a format with negligible metadata—ideally doubles this bound. A *structured* 50% reduction of width in the feed-forward blocks gives close to the same ideal speedup with no metadata at all, but costs far more accuracy than unstructured 50% sparsity does. This tension between how much accuracy is preserved and how much of the theoretical saving can be realized is the central theme of this document.

Pruning attacks the same costs from a different direction than quantization (treated in the companion paper of this series): quantization reduces the *bits per parameter*, pruning reduces the *number of parameters or the amount of computation*, and the two compose. The questions that structure this monograph are:

1. **Where is the redundancy?** Are there weights, neurons, heads, layers, or experts that can be removed with little effect, and can they be found cheaply?
2. **How do we find it without retraining?** Can the pruned model be obtained with a few hundred calibration samples, in hours on one GPU, even for models with tens or hundreds of billions of parameters?
3. **Can the saving be realized?** Does the sparsity pattern map onto hardware that actually executes faster or fits in less memory?

4. **What is lost?** Are the removed components truly redundant, or do they carry rare knowledge, tail behavior, safety properties, or reasoning capacity that standard metrics fail to detect?

2.2 A short history, 1989–2026

Classical era (1989–2015). The idea that trained networks contain removable parameters is as old as multi-layer perceptrons. LeCun, Denker and Solla's Optimal Brain Damage (OBD) [1] used a diagonal approximation of the loss Hessian to estimate the loss increase when a weight is set to zero, and Hassibi and Stork's Optimal Brain Surgeon (OBS) [2] used the full inverse Hessian, additionally producing a closed-form compensating update of the remaining weights. These two papers established the saliency-plus-compensation template that reappears, three decades later, in SparseGPT. Han et al. [3, 4] revived magnitude pruning with iterative retraining for convolutional networks and, with quantization and entropy coding, popularized the compression pipeline.

Lottery tickets and the sparsity debates (2017–2021). Zhu and Gupta's gradual magnitude pruning schedule [26], the Lottery Ticket Hypothesis (LTH) of Frankle and Carbin [5], the critical assessments of pruning [27, 29, 30], and the broad survey of Hoefler et al. [28] framed the questions of when sparse subnetworks exist, whether they can be found at initialization [7, 34], and whether sparse training can match dense training [165, 169]. For BERT-scale transformers, the community developed movement pruning [38], structured pruning with distillation (CoFi [76]), block pruning [78], and second-order approaches (oBERT [37]).

One-shot LLM era (2022–2024). The scale of GPT-3-class models rendered retraining-based pruning impractical, and two works shifted the field: Optimal Brain Compression [36] reformulated OBS as an exact layerwise problem with efficient updates, and SparseGPT [42] scaled this to 100B+ parameters, reporting 50% unstructured sparsity for OPT-175B and BLOOM-176B in a few GPU-hours with small perplexity loss[†]. Wanda [44] showed that the product of weight magnitude and input-activation norm is competitive with SparseGPT without any weight update. Structured methods (LLM-Pruner [73], SliceGPT [74], FLAP [80], Sheared-LLaMA [75]) and depth-pruning (ShortGPT [94], [95]) emerged in parallel, along with contextual sparsity (Deja Vu [153]).

Production era (2024–2026). NVIDIA's Minitron work [111, 112] operationalized pruning plus distillation for model families, later extended to hybrid Mamba–Transformer reasoning models and to nested "elastic" models [113, 115]. Learned 2:4 masks [62], training-free activation sparsity [160], MoE expert pruning and merging [131, 132], and the first critical examinations of what pruning does to reasoning, safety, and bias [101, 121, 187] define the present frontier. Chapter 16 gives a dated timeline.

2.3 Scope, conventions, and notation

We consider decoder-only transformers [196] with L blocks, each containing multi-head (or grouped-query [141]) self-attention with projections W_Q, W_K, W_V, W_O and a feed-forward network (FFN), either dense with two projections or gated (SwiGLU) with $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$, or a mixture of experts. For a linear layer, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration inputs are collected in $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$ (columns are token activations). A *mask* is $\mathbf{M} \in \{0,1\}^{d_{\text{out}} \times d_{\text{in}}}$ and the pruned weights are $\mathbf{M} \odot \mathbf{W}$ possibly followed by an update. The *sparsity* of a mask is the fraction of zeros. Appendix A collects all symbols.

Definition 2.1 (Sparsity kinds)**.** *Unstructured* sparsity allows arbitrary zero patterns. *Semi-structured* (N:M) sparsity requires exactly (or at most) N non-zeros in every aligned group of M consecutive weights. *Structured* sparsity removes entire rows, columns, heads, channels, layers, or experts, producing a smaller dense model. *Activation* (or *contextual*) sparsity skips computation associated with activations (or neurons) that are small or predicted to be inactive for the current input, leaving weights untouched.

Two further conventions matter. First, throughout we distinguish *theoretical* compression (parameter counts, FLOPs) from *realized* savings (storage with metadata, wall-clock latency, throughput under batching). Second, we distinguish *post-training* (one-shot, calibration only) from *training-time* approaches (pruning with fine-tuning, distillation, or sparse training), since their cost differs by orders of magnitude.

2.4 A cost model for sparsity

To reason about when sparsity helps, we use a simple two-term model for the time of one matrix–vector product $y = \mathbf{W}x$ (decoding step):

$$T \;\approx\; \max\Big(\underbrace{\frac{\text{bytes}(\mathbf{W}) + \text{bytes}(\text{index})}{B_{\text{mem}}\,\eta_{\text{mem}}}}_{T_{\text{mem}}},\; \underbrace{\frac{\text{FLOPs}}{F_{\text{peak}}\,\eta_{\text{cmp}}}}_{T_{\text{cmp}}} \Big) + T_{\text{overhead}}\,, \tag{2.2}$$

where $\eta_{\text{mem}}, \eta_{\text{cmp}} \in (0,1]$ are achieved fractions of peak. Dense low-batch decoding sits at T_{mem} with η_{mem} close to 0.8 or more in well-tuned kernels. A sparsity format can reduce bytes of $\mathbf{W}$ by a factor close to the density, but adds index bytes and typically reduces η_{mem} because of irregular access and reduced vectorization. For unstructured sparsity at density ρ with 16-bit values and 16-bit column indices (CSR-like), the stored bytes are $2\rho\, d_{\text{out}} d_{\text{in}} \cdot 2$, i.e., 4 bytes per non-zero versus 2 bytes per dense weight. Break-even density is therefore $\rho = 0.5$ *before* any efficiency loss: at 50% unstructured sparsity, a naive CSR layout stores as many bytes as the dense matrix. Real formats use bitmaps or compressed indices (Chapter 14) but the order of magnitude of the overhead explains why moderate unstructured sparsity yields no speedup.

2.5 The compressibility landscape in one picture

Figure 2.2 shows the qualitative trade-off curves that recur in the literature. The curves are schematic, drawn to convey ordering and shape (flat regime, knee, cliff), not to report any specific benchmark.

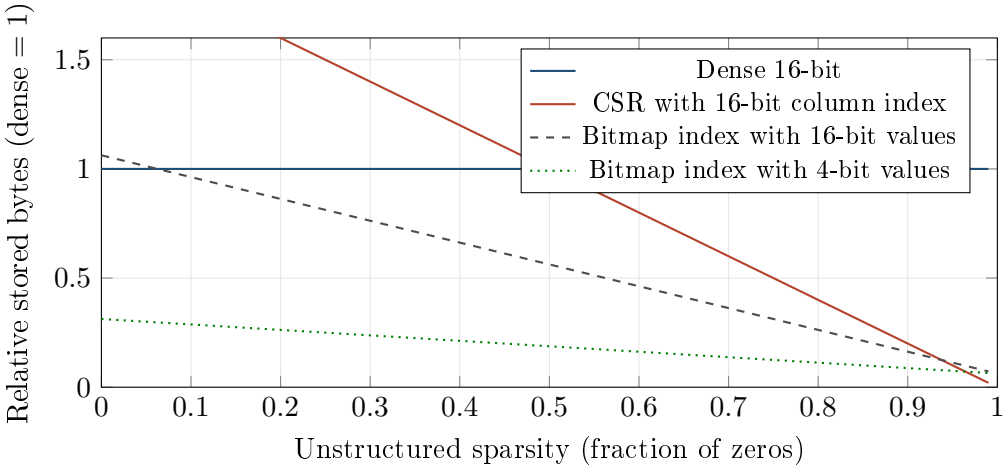


Figure 2.1: Illustrative storage models (computed from the formulas, not measured), relative to a dense 16-bit matrix. A 16-bit CSR layout (value plus 16-bit column index per non-zero) costs $2(1-s)$ and crosses the dense size at 50% sparsity. A bitmap layout with 16-bit values costs $(1-s)+1/16$; with 4-bit values it costs $(1-s)/4+1/16$ (the same 1-bit-per-weight bitmap is now a large fraction of the total).

2.6 Organization of the monograph

Chapter 3 develops mathematical foundations; Chapter 4 gives a taxonomy. Chapters 5–7 cover weight-level pruning: classic criteria and second-order one-shot methods, activation-aware scoring, and N:M sparsity with hardware. Chapters 8–11 cover structured families: width, depth, distillation pipelines, and MoE/attention pruning. Chapter 12 covers contextual activation sparsity, and Chapter 13 covers sparse training, scaling laws, and joint sparsity–quantization. Chapter 14 covers systems; Chapter 15 covers evaluation and what pruning removes. Chapters 16–20 synthesize: timeline and capabilities, pros/cons and risks, decision guide, priority assessment, open problems, and conclusions.

Takeaway

Pruning is an *approximation* problem with a *systems* constraint. The mathematics asks which parameters can be removed; the systems question asks whether the remaining structure is executable faster. A method is useful in practice only if both answers are favorable.

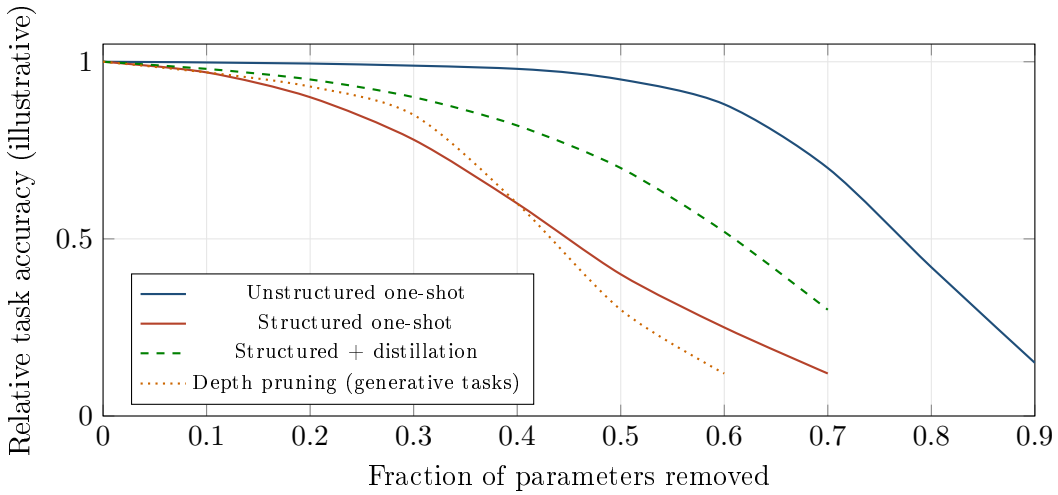


Figure 2.2: Schematic (synthetic, illustrative) accuracy-versus-removal curves for four families. Real curves depend on model, task, calibration data, and recovery budget; the figure encodes only the commonly reported ordering: unstructured one-shot degrades latest; structured one-shot degrades earlier; recovery training shifts the structured knee right; generative tasks are more sensitive than classification-style tasks.

Chapter 3

Mathematical Foundations

This chapter assembles the mathematical tools used throughout: sparse approximation, the compressed-sensing viewpoint, pruning as combinatorial optimization, relaxations (ℓ_1, ℓ_0, hard-concrete gates), second-order loss approximations, the lottery-ticket hypothesis and its status at LLM scale, over-parameterization and intrinsic dimension, and error propagation through depth. Proofs are given where short; otherwise we give sketches and cite the source.

3.1 Sparse approximation

3.1.1 Best k-term approximation

Let $w \in \mathbb{R}^N$ and let $\Sigma_k = \{v \in \mathbb{R}^N : \|v\|_0 \le k\}$ be the set of k-sparse vectors. The best k-term approximation error is $\sigma_k(w)_2 = \min_{v\in\Sigma_k} \|w - v\|_2$.

Proposition 3.1 (Magnitude thresholding is optimal in the Euclidean norm)**.** *A minimizer of $\|w - v\|_2$ over $v \in \Sigma_k$ is obtained by keeping the k entries of w of largest absolute value and setting the rest to zero. The error is $\sigma_k(w)_2^2 = \sum_{i\notin S^\star} w_i^2$ with $S^\star$ the index set of the k largest $|w_i|$.*

Proof. For a fixed support S with $|S| \le k$, the best v with $\operatorname{supp}(v) \subseteq S$ is $v_S = w_S$, with error $\sum_{i\notin S} w_i^2 = \|w\|_2^2 - \sum_{i\in S} w_i^2$. Minimizing over S is therefore equivalent to maximizing $\sum_{i\in S} w_i^2$, which is achieved by choosing the k largest squared entries. □

Proposition 3.1 is the only setting in which magnitude pruning is *exactly* optimal: it requires (a) a squared-error objective on the weights themselves (equivalently, an isotropic Hessian) and (b) orthonormal coordinates. Both assumptions fail for neural networks, which motivates every refinement in this monograph: curvature-aware criteria (Chapter 5) replace (a), and activation-aware scoring (Chapter 6) replaces the identity metric on weights by the input covariance.

Example 3.1 (How much energy does magnitude pruning discard? Computed from distributions)**.** If entries of w are i.i.d. $\mathcal{N}(0,1)$ and we keep the largest-magnitude fraction ρ, the retained energy fraction is $2\big(t\varphi(t) + Q(t)\big)$ with $t = \Phi^{-1}(1-\rho/2)$, φ the standard normal density and $Q = 1 - \Phi$. For i.i.d. Laplace entries (density $\frac{1}{2}e^{-|x|}$) the retained fraction is $e^{-t}(t^2+2t+2)/2$ with $t = -\ln\rho$. Table 3.1 lists values computed from these formulas (exact analytic values, not measurements). The heavier-tailed Laplace distribution is more compressible: at 50% sparsity, 96.7% of the energy is kept versus 92.9% for the Gaussian.

Two lessons follow. First, a purely weight-space criterion on near-Gaussian weight matrices loses a non-trivial share of energy even at moderate sparsity, so any method that preserves function at 50% must exploit structure *beyond* weight magnitude, such as correlated inputs (a few directions carry most of the signal) and compensation by the remaining weights. Second, real LLM weight matrices are not i.i.d.: they contain a few large outlier input channels (activation outliers) that

Table 3.1: Fraction of squared-norm energy retained when keeping the largest-magnitude fraction ρ of i.i.d. entries (analytic values from Example 3.1).

Kept fraction ρ	0.9	0.7	0.5	0.3	0.2	0.1	0.05
Sparsity $1-\rho$	10%	30%	50%	70%	80%	90%	95%
Gaussian	0.9995	0.9854	0.9287	0.7833	0.6498	0.4393	0.2791
Laplace	0.9998	0.9942	0.9667	0.8786	0.7809	0.5954	0.4241

dominate outputs [147, 149], which is precisely why activation-aware scores such as Wanda work.

3.1.2 Sparse regression and NP-hardness

Layerwise pruning of a linear map reduces to sparse least squares. For a layer with weight row $w \in \mathbb{R}^{d_{\text{in}}}$ and calibration inputs $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, find $\hat{w}$ with at most k non-zeros minimizing $\|w^\top \mathbf{X} - \hat{w}^\top \mathbf{X}\|_2^2$. Setting $A = \mathbf{X}^\top$ and $b = \mathbf{X}^\top w$, this is the problem

$$\min_{\hat{w}} \ \|A\hat{w} - b\|_2^2 \quad \text{s.t.} \quad \|\hat{w}\|_0 \le k, \tag{3.1}$$

where the target is exactly representable ($b = Aw$) but we want a sparse surrogate.

Theorem 3.1 (Hardness of sparse approximation; Natarajan 1995 [24])**.** *Given $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^n$ and $\varepsilon > 0$, deciding whether there is $\hat{w}$ with $\|A\hat{w} - b\|_2 \le \varepsilon$ and $\|\hat{w}\|_0 \le k$ is NP-hard.*

Consequently exact layerwise pruning is intractable in the worst case, and all practical algorithms are heuristics: greedy (OBS/SparseGPT), thresholding (magnitude, Wanda), convex relaxations (ℓ_1), iterative hard thresholding, or ADMM-based solvers [50, 51]. Hardness is a worst-case statement; Section 3.2 explains why average-case instances with well-conditioned A are easier.

3.2 The compressed-sensing viewpoint

Compressed sensing [22, 23] studies recovery of a sparse vector $w^\star$ from linear measurements $b = Aw^\star$ with $n \ll d$. In pruning, the roles are different: w is *not* sparse, and we seek a sparse $\hat{w}$ whose *outputs on the data* match. Nonetheless the same conditions govern whether simple algorithms succeed.

Definition 3.1 (Restricted isometry property)**.** A satisfies the RIP of order s with constant $\delta_s \in (0, 1)$ if for all s-sparse v: $(1 - \delta_s)\|v\|_2^2 \le \|Av\|_2^2 \le (1 + \delta_s)\|v\|_2^2$.

For layerwise pruning, $A^\top A = \mathbf{X}\mathbf{X}^\top = \frac{1}{2}\mathbf{H}$ is the (scaled) input second-moment matrix. RIP requires that every set of s input channels has well-conditioned sub-covariance. This is violated in LLMs: inputs to transformer sublayers (after normalization) have a few extremely high-variance channels and strongly correlated groups of channels [147–149], so the condition number of $\mathbf{H}$ is large, which is why one-shot weight pruning requires damping $\mathbf{H} + \lambda I$ (Chapter 5).

Algorithm 1 Iterative Hard Thresholding (IHT) for layerwise pruning [25]

Require: $A \in \mathbb{R}^{n \times d}$, $b \in \mathbb{R}^n$, sparsity level k, step size $\mu \le 1/\|A\|_2^2$, iterations T

1: $\hat{w}^{(0)} \leftarrow \mathrm{Top}_k(w)$ ▷ warm-start from magnitude pruning
2: **for** $t = 0, \dots, T-1$ **do**
3: $\quad g \leftarrow A^\top(A\hat{w}^{(t)} - b)$
4: $\quad \hat{w}^{(t+1)} \leftarrow \mathrm{Top}_k\big(\hat{w}^{(t)} - \mu g\big)$ ▷ gradient step then keep k largest
5: **end for**
6: **return** $\hat{w}^{(T)}$

3.2.1 Iterative hard thresholding

Theorem 3.2 (IHT linear convergence under RIP; Blumensath–Davies [25], informal)**.** *If A satisfies RIP of order $3k$ with $\delta_{3k} < 1/\sqrt{32}$ and $\mu = 1$ (with $\|A\|_2 \le 1$), then IHT converges linearly to within a constant multiple of the best k-term approximation error plus noise.*

Sketch. Each iteration decomposes the error into a component supported on the union of three supports (previous iterate, current iterate, and best k-term set) of size at most $3k$. On such sets the gradient step is a contraction by a factor $O(\delta_{3k})$, and the thresholding step increases the error by at most a constant factor. For small enough δ_{3k} the product is below one. □

In practice the conditions are unverified and IHT on LLM layers is run for a handful of iterations from a magnitude warm start. ALPS-style solvers add an operator-splitting (ADMM) formulation with a penalty continuation schedule [50, 51], which is the same ℓ_0 projection step as in line 4 of Algorithm 1 but with a variable-splitting consensus term.

3.2.2 Convex relaxation

Replacing $\|\hat{w}\|_0$ in (3.1) by $\|\hat{w}\|_1$ gives the lasso [21]:

$$\min_{\hat{w}} \ \tfrac{1}{2}\|A\hat{w} - b\|_2^2 + \lambda\|\hat{w}\|_1, \tag{3.2}$$

convex and solvable by proximal gradient (ISTA), whose proximal step is soft thresholding $\mathcal{S}_{\lambda\mu}(z) = \mathrm{sign}(z)\max(|z| - \lambda\mu, 0)$. Under RIP-type conditions, ℓ_1 recovers sparse signals exactly [23]. The practical drawback in LLM pruning is twofold: λ controls sparsity only indirectly (an outer search is needed to hit a target density), and the shrinkage biases surviving weights, which then require a debiasing least-squares refit on the support—exactly the OBS update of Chapter 5.

3.3 Pruning as combinatorial optimization

3.3.1 Loss-based formulation

Let $\mathcal{L}(\theta)$ be the training or calibration loss over parameters $\theta \in \mathbb{R}^P$. Global pruning to density ρ is

$$\min_{\mathbf{M} \in \{0,1\}^P,\ \|\mathbf{M}\|_0 \le \rho P} \ \min_{\theta'} \ \mathcal{L}(\mathbf{M} \odot \theta')\,. \tag{3.3}$$

The inner minimization represents retraining on the support; the one-shot setting freezes $\theta' = \theta$ or allows a closed-form update. The outer problem is a $\binom{P}{\rho P}$-sized combinatorial search. LLM-scale practice therefore decomposes it in three ways:

1. **Local (layerwise) decomposition.** Replace $\mathcal{L}$ by the sum of per-layer reconstruction errors on the *layer's own inputs*, solved layer by layer.

2. **Row-wise decomposition.** For a linear layer, the reconstruction error separates over output rows.

3. **Sparsity allocation.** The per-layer densities ρ_ℓ are chosen separately (uniform, outlier-weighted, learned).

Proposition 3.2 (Separability across output rows)**.** *Let $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$ and a mask $\mathbf{M}$. The reconstruction error $\|\mathbf{WX} - (\mathbf{M} \odot \hat{\mathbf{W}})\mathbf{X}\|_F^2$ equals $\sum_{i=1}^{d_{\text{out}}} \|(w_i - \boldsymbol{m}_i \odot \hat{w}_i)^\top \mathbf{X}\|_2^2$, with each row subproblem depending only on its own mask row and weights, provided the sparsity constraint is imposed per row or the global budget is handled by a separable allocation.*

Proof. The Frobenius norm squared is the sum of squared Euclidean norms of rows, and row i of $(\mathbf{M} \odot \hat{\mathbf{W}})\mathbf{X}$ is $(\boldsymbol{m}_i \odot \hat{w}_i)^\top \mathbf{X}$. □

Proposition 3.2 is why SparseGPT parallelizes over rows and why the Hessian $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$ is *shared* by all rows of a layer: only one $d_{\text{in}} \times d_{\text{in}}$ matrix needs to be factorized per layer. For per-row sparsity constraints (Wanda's default comparison group) or N:M constraints, rows are fully independent.

3.3.2 Taylor expansion and saliency

Expanding around a trained point θ with gradient $g = \nabla\mathcal{L}(\theta)$ and Hessian $\mathbf{H}_\theta$ gives, for a perturbation δ,

$$\mathcal{L}(\theta + \delta) - \mathcal{L}(\theta) \approx g^\top \delta + \tfrac{1}{2}\delta^\top \mathbf{H}_\theta \delta. \tag{3.4}$$

Setting a single weight θ_q to zero means $\delta = -\theta_q e_q$, giving first-order saliency $|g_q \theta_q|$ (the Molchanov et al. criterion [31, 32]) and second-order saliency $\frac{1}{2}\theta_q^2 [\mathbf{H}_\theta]_{qq}$ (OBD [1]) when $g \approx 0$ at convergence. Details, together with OBS compensation, are in Chapter 5. The Fisher information $F = \mathbb{E}[\nabla \log p \, \nabla \log p^\top]$ coincides with the Hessian of the negative log-likelihood at the data distribution when the model is well-specified, and the empirical Fisher is the usual surrogate [33, 35]; for the squared reconstruction loss in a linear layer, the Hessian is exact and equals $2\mathbf{X}\mathbf{X}^\top$ with no approximation (Proposition 3.3).

Proposition 3.3 (Exact layerwise Hessian)**.** *For $\mathcal{E}(w) = \|(w - \hat{w})^\top \mathbf{X}\|_2^2$ viewed as a function of $\hat{w} \in \mathbb{R}^{d_{\text{in}}}$, the gradient is $-2\mathbf{X}\mathbf{X}^\top(w - \hat{w})$ and the Hessian is the constant matrix $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$.*

Proof. $\mathcal{E}(\hat{w}) = (w - \hat{w})^\top \mathbf{X}\mathbf{X}^\top (w - \hat{w})$ is a quadratic form; differentiate twice. □

Because the quadratic is exact, the Taylor expansion (3.4) has no remainder, and the OBS update of Chapter 5 is exactly optimal for a single weight removal. This is the key reason second-order pruning is so much better behaved in the layerwise setting than in the global one, where $\mathbf{H}_\theta$ is only a local model.

3.4 Relaxations: ℓ_0 regularization and hard-concrete gates

Learning masks by gradient descent requires a differentiable surrogate of $\|\mathbf{M}\|_0$. Louizos et al. [18] attach to each weight (or group) a gate z with a stretched, rectified Concrete distribution [19, 20]. Let $u \sim \mathcal{U}(0,1)$, $\alpha > 0$ a learnable location parameter, $\beta > 0$ a temperature, and stretch limits $\gamma < 0 < 1 < \zeta$. Then

$$s = \sigma\Big(\tfrac{1}{\beta}\big(\log u - \log(1-u) + \log\alpha\big)\Big), \tag{3.5}$$

$$\bar{s} = s(\zeta - \gamma) + \gamma, \qquad z = \min\big(1, \max(0, \bar{s})\big). \tag{3.6}$$

Because of the clip, z takes the exact values 0 and 1 with positive probability, while remaining a differentiable function of $\log\alpha$ (a.s.) via the reparameterization trick. The probability that a gate is non-zero is available in closed form:

$$\Pr(z \neq 0) = \sigma\big(\log\alpha - \beta\log(-\gamma/\zeta)\big), \tag{3.7}$$

so the expected ℓ_0 norm $\sum_j \Pr(z_j \neq 0)$ is a smooth penalty. Typical settings in the original paper are $\gamma = -0.1, \zeta = 1.1, \beta = 2/3$.

Why hard-concrete gates matter for LLM pruning

Hard-concrete gates are the standard way to learn *structured* masks (heads, FFN channels, hidden dimensions, whole layers) jointly with the weights: CoFi [76], structured pruning of Wang et al. [77], Sheared-LLaMA [75] and Compresso [83] all use ℓ_0-style gates with Lagrangian constraints on the target architecture (Chapter 8). The price is that learning masks requires gradient computation through the full model, with memory and compute proportional to model size, which restricts the approach to moderate scales or short mask-learning phases.

The Lagrangian formulation used for targeted pruning adds a penalty $\lambda_1(\hat{s} - s_{\text{target}}) + \lambda_2(\hat{s} - s_{\text{target}})^2$ with $\hat{s}$ the expected sparsity and λ_1, λ_2 trained by gradient *ascent*, which drives the expected sparsity to the target while the weights and gate parameters descend on the task loss.

3.4.1 Straight-through and Gumbel-softmax mask distributions

For discrete patterns, such as choosing N of M weights, one can parameterize a categorical distribution over the $\binom{M}{N}$ candidate masks and sample with the Gumbel-softmax relaxation [19, 20]. For 2:4 there are six candidate masks per group of four. MaskLLM [62] learns the logits of this categorical distribution end to end with frozen weights (Chapter 7). The Gumbel-softmax with temperature τ and a straight-through estimator provides gradient signals whose bias vanishes as $\tau \to 0$ and whose variance grows as $\tau \to 0$—the familiar bias–variance trade-off for discrete relaxations.

3.5 Over-parameterization, redundancy, and intrinsic dimension

3.5.1 Why networks are prunable at all

Overparameterized networks interpolate training data with many equivalent solutions, so after training the parameter vector sits in a large flat valley rather than at an isolated minimum.

Three complementary viewpoints explain prunability:

Compression-based generalization. Arora et al. [17] show that a network whose function can be approximated by a much smaller compressed network (in a technical, noise-stability sense) has generalization bounds governed by the compressed size. Empirically, stable layers (those that attenuate injected noise) are more compressible, which links prunability to noise stability.

Intrinsic dimension. Li et al. [15] define the intrinsic dimension of an objective landscape as the minimal dimension d of a random subspace $\theta = \theta_0 + P\phi$, $\phi \in \mathbb{R}^d$, in which one can reach (say 90% of) the full-model performance. Aghajanyan et al. [16] found that the intrinsic dimension of fine-tuning pre-trained language models is remarkably small and decreases with model size, which explains the success of parameter-efficient adaptation (LoRA [85]) and suggests the weight updates are low-complexity relative to P.

Rethinking generalization. Zhang et al. [197] showed deep nets can fit random labels, so capacity alone does not explain generalization; training dynamics (implicit bias) select simple solutions that tend to be compressible.

Remark 3.1. Intrinsic dimension concerns the *update* $\theta - \theta_0$ in a random subspace, not the pretrained weights themselves. Low intrinsic dimension of fine-tuning does not imply that pretraining weights are redundant; indeed knowledge-intensive tasks (Chapter 15) reveal sensitivity to pruning that the intrinsic-dimension picture alone would not predict. We therefore treat these results as evidence for the *existence* of low-complexity descriptions, not as a license for aggressive pruning.

3.5.2 The strong lottery ticket hypothesis

Theorem 3.3 (Strong LTH, informal; Malach et al. [10])**.** *For any target network f of depth ℓ, width d and bounded weights, a randomly initialized network g that is a factor* $\mathrm{poly}(d, \ell, 1/\varepsilon, \log(1/\delta))$ *wider and one layer deeper contains, with probability at least $1 - \delta$, a subnetwork (obtained by masking only, without changing weights) that approximates f within ε.*

Sketch. Each target weight is approximated by a sparse sum of random weights using subset-sum arguments: with enough random candidates, some subset sums approximate any value in the interval to within ε', and the construction extends to neurons and layers by composition. □
The result is existential: it says nothing about how to find the mask efficiently, and the required overparameterization factor is polynomial, not constant. Ramanujan et al. [9] gave empirical evidence with edge-popup, and Zhou et al. [8] showed supermasks over random weights beat chance.

3.6 The lottery ticket hypothesis and its status at LLM scale

Definition 3.2 (Lottery ticket hypothesis (LTH) [5])**.** A randomly initialized dense network contains a subnetwork ("winning ticket") that, when trained in isolation from its original initialization θ_0, matches the test accuracy of the full network in at most the same number of iterations.

The procedure to find tickets is iterative magnitude pruning (IMP).

Algorithm 2 Iterative magnitude pruning with weight rewinding [5, 6]

Require: initial weights θ_0, rewind iteration k, rounds R, per-round prune fraction p

1: Train θ_0 for k steps to obtain θ_k (rewind point); set $\mathbf{M} \leftarrow \mathbf{1}$
2: **for** $r = 1, \ldots, R$ **do**
3: Train $\mathbf{M} \odot \theta_k$ to convergence to obtain $\theta^{(r)}$
4: Remove the fraction p of the smallest-magnitude surviving weights in $\theta^{(r)}$ (global or layerwise); update $\mathbf{M}$
5: **end for**
6: **return** mask $\mathbf{M}$ and rewind point θ_k

Evidence and refinements. Plain LTH (rewinding to θ_0) fails for deeper networks at scale; rewinding to an early iterate θ_k (k a small fraction of training) restores the effect, and Frankle et al. [6] connect this to *linear mode connectivity*: tickets exist when SGD noise early in training stops changing the basin, i.e., two runs from θ_k with different data order are linearly connected without a loss barrier. Pruning at initialization (SNIP [34] and successors) fails to find tickets of comparable quality, and Frankle et al. [7] showed the quality of the found masks is insensitive to certain randomized variants (layerwise shuffles), suggesting that such methods mostly find layerwise sparsity ratios. Su et al. [13] showed random tickets with the right layerwise ratios can be competitive in some regimes; Evci et al. [14] analyzed gradient flow and why tickets help optimization.

For pretrained language models, tickets exist at moderate sparsity: Chen et al. [11] found winning tickets in pretrained BERT at 40–90% sparsity for downstream tasks, and Prasanna et al. [12] found that even structured subnetworks of BERT can be winning, and that "good" and "bad" subnetworks differ little ("all tickets are winning").

LTH status at LLM scale: what is and is not established

Not established: that IMP-style winning tickets with rewinding exist and are findable for billion-parameter pretrained LLMs. IMP requires repeated full training runs; no published study repeats it at LLM pretraining scale, as far as the author is aware. **Established indirectly:** (i) one-shot methods find 50% sparse subnetworks of pretrained LLMs with small perplexity loss without retraining [42, 44], which is consistent with a weak, post-hoc form of the hypothesis; (ii) sparse pre-training with a pruning schedule can match dense pre-training at equal compute [171]. **Interpretation caution:** "subnetwork that matches the dense model on perplexity after one-shot pruning" is not the LTH, which concerns trainability from initialization. The term is used loosely in LLM papers.

3.7 Error propagation through layers

A layerwise pruning method controls the error $\varepsilon_\ell = \|(\mathbf{W}_\ell - \hat{\mathbf{W}}_\ell)\mathbf{X}_\ell\|_F$ at layer ℓ *given its inputs*, but the final output error depends on how errors compound. Let $f = f_L \circ \cdots \circ f_1$ with f_ℓ being K_ℓ-Lipschitz, and let the pruned network be $\hat{f} = \hat{f}_L \circ \cdots \circ \hat{f}_1$ where $\|f_\ell(x) - \hat{f}_\ell(x)\| \le \varepsilon_\ell$ for all x in the relevant region.

Proposition 3.4 (Error accumulation bound)**.** *Under the above assumptions,* $\|f(x) - \hat{f}(x)\| \le \sum_{\ell=1}^{L} \varepsilon_\ell \prod_{j=\ell+1}^{L} K_j$.

Proof. By induction on L. Let $a_\ell = f_\ell \circ \cdots \circ f_1(x)$ and $\hat{a}_\ell$ the pruned counterpart. Then $\|a_\ell - \hat{a}_\ell\| \le \|f_\ell(a_{\ell-1}) - f_\ell(\hat{a}_{\ell-1})\| + \|f_\ell(\hat{a}_{\ell-1}) - \hat{f}_\ell(\hat{a}_{\ell-1})\| \le K_\ell \|a_{\ell-1} - \hat{a}_{\ell-1}\| + \varepsilon_\ell$. Unrolling the recursion yields the claim. □

The bound is loose for residual networks. With residual connections $a_\ell = a_{\ell-1} + h_\ell(a_{\ell-1})$, Lipschitz constants are $1 + \mathrm{Lip}(h_\ell)$, and the bound grows at most as $\prod_j (1 + \mathrm{Lip}(h_j))$; the empirical fact that LLM blocks make small relative updates to the residual stream is why removing a single block may do little (Chapter 9) but also why errors from every layer reach the output nearly undamped. A practical corollary used by SparseGPT and Wanda pipelines is to feed each layer the activations of the *already pruned* preceding layers when computing the calibration Hessian ("sequential pruning"), which makes the layerwise objective match what the pruned network will actually see, at the cost of a sequential pass and some memory.

3.8 Sparsity allocation across layers

Given per-layer sensitivity curves $E_\ell(\rho_\ell)$ (error as function of density), allocation chooses (ρ_ℓ) to minimize $\sum_\ell E_\ell(\rho_\ell)$ subject to $\sum_\ell \rho_\ell P_\ell = \rho P$. If the E_ℓ are convex and differentiable, the KKT condition equalizes marginal cost per parameter:

$$\frac{E_\ell'(\rho_\ell)}{P_\ell} = \mu \quad \text{for all } \ell \text{ with } 0 < \rho_\ell < 1, \tag{3.8}$$

with μ the Lagrange multiplier. Uniform allocation is optimal only if all layers have the same sensitivity per parameter, which is false in LLMs. Methods differ in how they estimate E_ℓ: outlier ratio (OWL [46]), heavy-tailed spectral exponents (AlphaPruning [54]), learned per-block sparsity (BESA [53]), or Hessian-based dynamic programming (ZipLM-style [57]). Chapter 6 covers these.

Example 3.2 (Greedy allocation on toy sensitivity curves; illustrative)**.** Suppose two layers of equal size with error curves $E_1(s) = s^2$ and $E_2(s) = 4s^2$ in sparsity s, and a target mean sparsity 0.5, i.e., $s_1 + s_2 = 1$. Equalizing marginal error $2s_1 = 8s_2$ gives $s_1 = 0.8$, $s_2 = 0.2$ with total error $0.64 + 0.16 = 0.80$; uniform allocation ($s_1 = s_2 = 0.5$) gives $0.25 + 1.00 = 1.25$. Optimal allocation reduces the summed error by 36% in this toy example. Real LLMs show non-uniform sensitivities too: first and last blocks, and blocks with many activation outliers, are typically more sensitive.

3.9 Summary and a map of the theory to later chapters

Table 3.2: Where each theoretical idea reappears.

Idea	Practical consequence	Chapter
Best k-term approximation	Magnitude pruning optimal only under isotropy	5
Layerwise exact Hessian $2\mathbf{X}\mathbf{X}^\top$	OBS update is exactly optimal per weight	5
Row separability	Parallel row-wise solvers; Wanda per-row ranking	5, 6
RIP / conditioning	Damping; calibration size	5
ℓ_0 hard-concrete gates	Learned structured masks	8
Gumbel-softmax over masks	Learned N:M masks	7
LTH	Sparse-from-scratch, dynamic sparse training	13
Error accumulation	Sequential pruning; sensitivity of depth	9
Allocation KKT	OWL, AlphaPruning, BESA	6

Chapter 4

A Taxonomy of Pruning and Sparsity Methods

The literature uses overlapping vocabulary: "pruning," "sparsification," "compression," "slicing," "layer dropping," "expert skipping," "contextual sparsity." This chapter fixes terms by organizing methods along five orthogonal axes and then maps the main methods onto them. The aim is to make clear which comparisons are meaningful: a method that removes a layer and a method that zeroes half the weights of every matrix cannot be compared by "sparsity percentage" alone.

4.1 Five axes

Axis 1: What is removed (granularity). *Weights* (unstructured), *groups of weights* (N:M, blocks), *neurons or channels* (FFN intermediate channels, hidden dimensions), *attention heads* (or key-value groups), *layers or blocks* (depth), *experts* (MoE), *tokens or activations* (dynamic).

Axis 2: Static versus dynamic. *Static* pruning produces one fixed sub-model; *dynamic* (contextual, input-adaptive) pruning chooses what to skip per token or per sequence, as in Deja Vu [153], TEAL [160], early-exit [102], and MoE routing itself.

Axis 3: When the decision is made and how much training is involved. *Post-training one-shot* (calibration only), *post-training with light recovery* (LoRA or short fine-tuning), *pruning with distillation* (billions to hundreds of billions of tokens), *pruning during pre-training* (gradual schedules), *sparse training from scratch* (dynamic sparse training).

Axis 4: Criterion (how importance is scored). Magnitude; first-order (gradient, Taylor); second-order (Hessian, Fisher, OBS); activation-aware (Wanda, RIA); reconstruction-based (layerwise or blockwise error); learned (gates, mask distributions); similarity-based (layer or head redundancy); routing-statistics-based (experts).

Axis 5: Is the result executable faster on today's hardware? *Yes by construction* (smaller dense model); *yes with vendor kernels* (2:4 sparsity; custom activation-sparse GEMV kernels); *only with specialized kernels or formats* (unstructured or block-sparse); *memory only* (compressed storage); *conditional* (MoE expert pruning reduces memory and, via less routing diversity, sometimes latency).

4.2 A tree of methods

4.3 Comparison table

Table 4.1 places the main method families on the axes. The "speedup path" column states the mechanism by which the method could reduce latency or cost, with the caveats discussed in

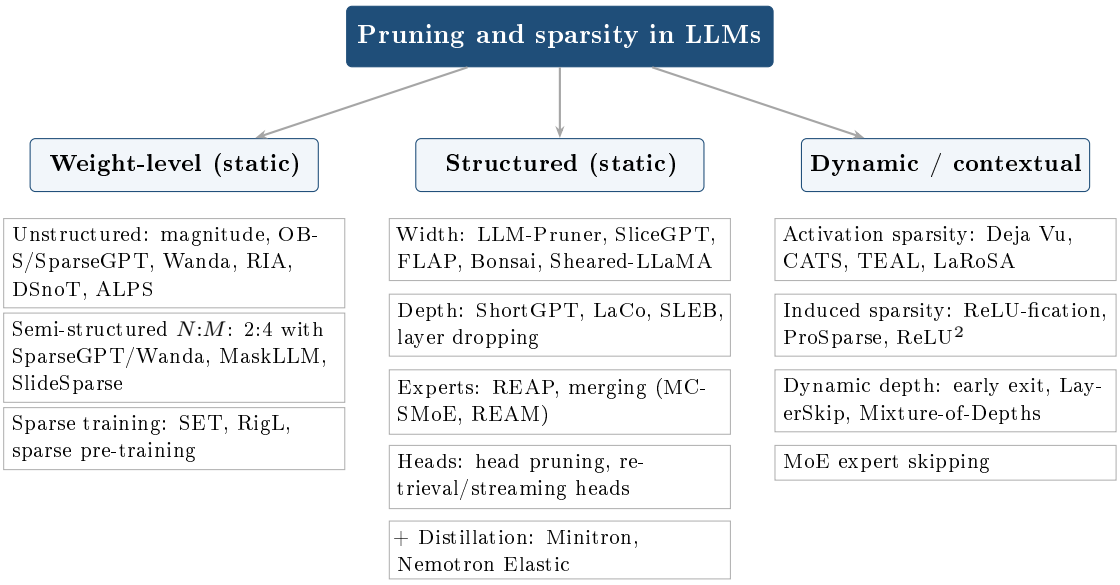


Figure 4.1: Taxonomy of pruning and sparsity techniques for LLMs. Static weight-level and structured methods change the stored model; dynamic methods change which computation is performed per input. Distillation is orthogonal and typically combined with structured removal.

later chapters.

4.4 Terminology disambiguation

"Sparsity" in MoE vs. in pruning. Mixture-of-Experts models are *conditionally computed*: only a few experts are active per token ("sparse activation"), but all experts must be stored. Pruning experts reduces the *stored* parameter count; this is a different notion of sparsity from that of weight matrices.

Activation sparsity vs. contextual sparsity. Activation sparsity is the fraction of near-zero entries in a hidden activation vector; contextual sparsity (Deja Vu [153]) is the existence, for a given input, of small sets of heads and neurons that yield approximately the same output. The former is a property of values; the latter is a property of the function that enables skipping computation if the set can be predicted cheaply.

Slicing. SliceGPT [74] performs *structured* pruning after an orthogonal change of basis, so it removes rows and columns of the transformed matrices; it does not produce an unstructured pattern.

Compression ratio. Parameter ratio, FLOPs ratio and latency ratio differ. Removing a fraction p of attention heads reduces attention parameters by about p but, for grouped-query attention with a few key-value heads, removes a smaller share of the KV cache. We always state which quantity is meant.

Pruning vs. distillation vs. shrinking. "Pruning plus distillation" produces a smaller architecture initialized from the large model's weights and trained to match the large model's output distribution; "training a smaller model from scratch" initializes randomly. The trade-offs are treated in Chapters 10 and 13.

Table 4.1: Taxonomy summary. "Training" is the typical extra training beyond calibration. "Realized speedup path" is qualitative.

Family	Granularity	Training	Static/dyn.	Realized speedup path
SparseGPT, Wanda (unstructured)	weight	none (calibration)	static	storage compression; GPU speedup rare; CPU/accelerator sparse kernels
N:M (2:4)	weight group	none to light; learned masks	static	sparse tensor cores (up to 2× peak on matmul)
Width pruning (post-training)	channel / head / hidden dim	none to LoRA	static	smaller dense GEMMs: universal
Width pruning + distillation	channel / head / hidden dim	10^9–10^{11} tokens	static	smaller dense GEMMs: universal
Depth pruning	layer	none to light	static	fewer sequential layers: universal; latency scales linearly with depth
Early exit / layer skip	layer	training-aware or none	dynamic	variable depth; batching complicates
MoE expert pruning/merging	expert	none (calibration)	static	memory reduction; fewer experts to load
Contextual activation sparsity	neuron/column	none (TEAL) to continued pretraining (ReLUfication)	dynamic	sparse GEMV kernels reading fewer weight columns
Sparse training	weight	full training	static	training memory/FLOPs only with custom kernels

4.5 Which comparisons are fair

A fair comparison among methods requires fixing (i) the *executable cost metric* (parameter count, FLOPs per token, measured latency on a named hardware/software stack), (ii) the *recovery budget* (tokens and compute for fine-tuning or distillation), (iii) the *calibration set* (source, size, and sampling), and (iv) the *evaluation suite* (including generative tasks). The following table lists common unfair comparisons, which recur in the literature and should be treated skeptically.

Takeaway

Compare methods on a common *cost* axis (measured latency or throughput, memory) and a common *quality* axis that includes generative and long-context tasks, with the recovery budget declared. Parameter counts alone are not a cost.

Table 4.2: Common sources of unfair comparison in pruning papers.

Practice	Why it misleads
Comparing unstructured 50% with structured 20% as "30 points less compression"	Unstructured 50% typically yields no runtime gain on GPUs; structured 20% does. The relevant metric is realized cost.
Reporting WikiText-2 perplexity alone	Perplexity is dominated by common tokens; it is insensitive to rare knowledge and multi-step reasoning (Chapter 15).
Calibrating on the evaluation distribution (e.g., WikiText-2 calibration, WikiText-2 evaluation)	Overstates quality; calibration-data dependence is substantial [185, 186].
Giving the structured method a fine-tuning budget and not the baseline	Recovery training can dominate the method's own contribution.
Comparing against dense models of equal size trained on less data	Hides that a large model was pruned, so total training cost includes the parent's pretraining.
Reporting "sparsity" counted over linear layers only	Embeddings and the output head can be a large share for small models.

Chapter 5

Classic Criteria and One-Shot Second-Order Pruning

This chapter covers the oldest and the most influential ideas of weight pruning: importance criteria (magnitude, gradient/Taylor, Fisher), the Optimal Brain Damage and Optimal Brain Surgeon derivations, and the layerwise reconstruction framework that made second-order pruning work at the scale of 100B-parameter models (Optimal Brain Compression and SparseGPT). We derive the closed-form updates in full, give pseudocode, and work a numerical example by hand.

5.1 Magnitude pruning

Magnitude pruning removes the weights with smallest $|w|$. Globally it compares across layers; layerwise (the default for LLMs) it removes the same fraction in each layer. By Proposition 3.1 it is optimal when the loss is isotropic in weight space. Its appeal is that it requires no data, no gradients, and $O(P \log P)$ time. For convolutional networks, iterative magnitude pruning interleaved with retraining (Han et al. [3]) and gradual pruning schedules (Zhu and Gupta [26]) work well. For pretrained LLMs without retraining, however, magnitude pruning at 50% already damages quality badly: on LLaMA-7B, the Wanda paper reports WikiText-2 perplexity rising from 5.68 (dense) to 17.29 with layerwise magnitude pruning at 50%† [44]. The reason is the activation outlier structure: a small weight attached to a high-variance input channel can matter more than a large weight attached to a quiet channel, which magnitude cannot see.

A useful way to state the failure: output perturbation from zeroing weight w_{ij} is $w_{ij}x_j$, and its expected square is $w_{ij}^2\,\mathbb{E}[x_j^2]$. Magnitude pruning implicitly assumes $\mathbb{E}[x_j^2]$ is the same for all j. In LLMs the input-channel second moments $\mathbb{E}[x_j^2]$ span several orders of magnitude after the norm layers [147, 149]. The simplest data-aware correction multiplies by the channel norm, which is Wanda's score (Chapter 6).

5.2 Gradient, Taylor and Fisher importance

First-order Taylor. For a single parameter, removing θ_q changes the loss by approximately $-g_q\theta_q$ (first-order term of (3.4) with $\delta = -\theta_q e_q$). The absolute value $|g_q\theta_q|$ is the saliency used by Molchanov et al. [31]. For a *group* S (a neuron, head, or channel), Molchanov et al. [32] recommend summing before taking the absolute value or squaring and summing: $I_S^{(1)} = \left|\sum_{q\in S} g_q\theta_q\right|$ or $I_S^{(2)} = \left(\sum_{q\in S} g_q\theta_q\right)^2$. The squared form estimates the squared loss change under a second-order view of the group gate $z_S \in \{0,1\}$, since $\partial\mathcal{L}/\partial z_S = \sum_{q\in S} g_q\theta_q$.

Fisher. With $\mathcal{L} = -\mathbb{E}\log p_\theta(y|x)$, the Fisher information $F = \mathbb{E}[\nabla \log p\, \nabla \log p^\top]$ equals the Hessian of $\mathcal{L}$ when the model is well-specified, and the *empirical* Fisher $\hat{F} = \frac{1}{n}\sum_i g_i g_i^\top$ (from per-example gradients g_i) is a positive semi-definite proxy. Diagonal Fisher pruning [33] scores weight q by $\frac{1}{2}\theta_q^2\hat{F}_{qq} = \frac{1}{2n}\theta_q^2\sum_i g_{i,q}^2$, a diagonal OBD with Fisher in place of the Hessian. Block-

diagonal inverse-Fisher approximations (WoodFisher [35]) apply the Woodbury identity to a low-rank-plus-damping estimate to get the OBS update at moderate scale.

Adaptation for LLMs. For LLMs, per-example gradients are expensive in memory. Gradient-based LLM pruners generally use (i) first-order group importance with a small calibration set (LLM-Pruner [73]), (ii) gradient-magnitude–weight-magnitude products (GBLM-Pruner [48]), or (iii) learned or evolved symbolic scores (Pruner-Zero [49]). Their added cost is a backward pass through the full model per calibration batch, which for 70B-scale models requires significant memory (activations and gradients), though no optimizer state.

Remark 5.1 (Why first-order scores are fragile at convergence). At a well-trained point $g \approx 0$ in expectation, so first-order saliency is dominated by sampling noise in g unless the group sums are large; the *second*-order term then carries the signal. This is the original rationale for OBD [1] and explains why gradient-based scores need many examples or must be combined with Hessian information.

5.3 Optimal Brain Damage and Optimal Brain Surgeon

5.3.1 Optimal Brain Damage

OBD [1] approximates (3.4) at a minimum ($g = 0$) with a diagonal Hessian: $\Delta\mathcal{L} \approx \frac{1}{2}\sum_q H_{qq}\delta_q^2$. Removing weight q ($\delta_q = -\theta_q$) costs $\frac{1}{2}H_{qq}\theta_q^2$. OBD removes the lowest-saliency weights and retrains. The diagonal approximation ignores curvature coupling between weights and, importantly, offers no compensation: after zeroing θ_q, all other weights remain unchanged.

5.3.2 Optimal Brain Surgeon

OBS [2] keeps the full Hessian and asks: if we force θ_q to zero, what is the best simultaneous adjustment of all other weights?

Theorem 5.1 (OBS saliency and update). *Let $\mathcal{L}(\theta+\delta) = \mathcal{L}(\theta) + \frac{1}{2}\delta^\top\mathbf{H}\delta$ with $\mathbf{H} \succ 0$. The minimum of $\frac{1}{2}\delta^\top\mathbf{H}\delta$ subject to $e_q^\top\delta + \theta_q = 0$ is attained at*

$$\delta^\star = -\frac{\theta_q}{[\mathbf{H}^{-1}]_{qq}}\,\mathbf{H}^{-1}e_q, \qquad \rho_q = \tfrac{1}{2}\delta^{\star\top}\mathbf{H}\delta^\star = \frac{\theta_q^2}{2[\mathbf{H}^{-1}]_{qq}}. \tag{5.1}$$

Proof. Form the Lagrangian $\mathcal{J} = \frac{1}{2}\delta^\top\mathbf{H}\delta + \lambda(e_q^\top\delta + \theta_q)$. Stationarity in δ gives $\mathbf{H}\delta + \lambda e_q = 0$, so $\delta = -\lambda\mathbf{H}^{-1}e_q$. The constraint reads $-\lambda[\mathbf{H}^{-1}]_{qq} + \theta_q = 0$, so $\lambda = \theta_q/[\mathbf{H}^{-1}]_{qq}$, which gives $\delta^\star$. Convexity ($\mathbf{H} \succ 0$) makes this the global minimizer. Substituting, $\frac{1}{2}\delta^{\star\top}\mathbf{H}\delta^\star = \frac{1}{2}\lambda^2 e_q^\top\mathbf{H}^{-1}\mathbf{H}\mathbf{H}^{-1}e_q = \frac{1}{2}\lambda^2[\mathbf{H}^{-1}]_{qq} = \theta_q^2/(2[\mathbf{H}^{-1}]_{qq})$. □

OBS therefore removes the weight with the smallest ρ_q, applies $\delta^\star$ (which zeros θ_q exactly and nudges all other weights along column q of $\mathbf{H}^{-1}$), and then *updates* $\mathbf{H}^{-1}$ before the next removal.

Lemma 5.1 (Inverse update after removing a coordinate). *If q is eliminated and $\mathbf{H}^{-1}$ is the current inverse Hessian, then the inverse Hessian of the remaining coordinates (the inverse of $\mathbf{H}$ with row and column q deleted) is the Schur complement*

$$\big(\mathbf{H}_{-q,-q}\big)^{-1} = \Big[\mathbf{H}^{-1} - \frac{\mathbf{H}^{-1}e_q e_q^\top\mathbf{H}^{-1}}{[\mathbf{H}^{-1}]_{qq}}\Big]_{-q,-q}. \tag{5.2}$$

Proof. Partition $\mathbf{H}^{-1} = \left(\begin{smallmatrix} A & b \\ b^\top & c \end{smallmatrix}\right)$ with q the last index. By block inversion, the inverse of the leading block of $\mathbf{H}$ equals $A - bb^\top/c$, which is the claimed expression restricted to indices other than q. □

Lemma 5.1 is a single step of Gaussian elimination on $\mathbf{H}^{-1}$, costing $O(d^2)$. Naively repeating OBS for k removals therefore costs $O(kd^2)$ per weight vector, and with $\mathbf{H} \in \mathbb{R}^{P\times P}$ over all parameters the method is impossible beyond tiny networks (P^2 memory). Layerwise decomposition removes this obstacle.

Example 5.1 (Worked OBS example; exact arithmetic)**.** Consider a layer row with three weights $w = (1.0, -0.9, 0.8)^\top$ and calibration statistics giving

$$\mathbf{H} = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 4 & 2 \\ 0 & 2 & 4 \end{pmatrix}, \qquad \mathbf{H}^{-1} = \begin{pmatrix} 0.375 & -0.25 & 0.125 \\ -0.25 & 0.5 & -0.25 \\ 0.125 & -0.25 & 0.375 \end{pmatrix}.$$

We want to remove one weight. Candidate scores:

	weight 1	weight 2	weight 3
magnitude $\lvert w_q\rvert$	1.00	0.90	**0.80**
OBD $\frac{1}{2}H_{qq}w_q^2$	2.00	1.62	**1.28**
OBS $w_q^2/(2[\mathbf{H}^{-1}]_{qq})$	1.333	**0.810**	0.853

Magnitude and OBD remove weight 3, incurring (without compensation) a loss increase of $\frac{1}{2}H_{33}w_3^2 = 1.28$. OBS removes weight 2 and applies

$$\delta^\star = -\frac{-0.9}{0.5}\,(-0.25,\, 0.5,\, -0.25)^\top = (-0.45,\, 0.9,\, -0.45)^\top,$$

giving new weights $(0.55, 0, 0.35)$ and loss increase exactly 0.81 (verified directly as $\frac{1}{2}\delta^\top\mathbf{H}\delta = \frac{1}{2}\cdot 0.9\cdot 1.8 = 0.81$). The OBS choice plus compensation yields a loss increase 37% lower than the magnitude choice without compensation. This is a toy, but it exhibits the two mechanisms: (i) correlated inputs (off-diagonal $\mathbf{H}$) mean other weights can absorb the removed weight's role; (ii) the criterion accounts for how easily that absorption works through $[\mathbf{H}^{-1}]_{qq}$ (large $[\mathbf{H}^{-1}]_{qq}$ means little curvature in direction e_q after accounting for the others).

5.4 Layerwise reconstruction: OBC and the exact Hessian

Frantar and Alistarh [36] observed that if we pose pruning *per layer* as a reconstruction problem, the Hessian is exact (Proposition 3.3) and small. For each linear layer with weights $\mathbf{W}$ and inputs $\mathbf{X}$, solve

$$\min_{\hat{\mathbf{W}}\ \text{sparse}} \left\|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\right\|_F^2 = \sum_i \left\|(w_i - \hat{w}_i)^\top\mathbf{X}\right\|_2^2. \tag{5.3}$$

By Proposition 3.2 the rows decouple, each with Hessian $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{d_{\text{in}}\times d_{\text{in}}}$, *shared* across all rows. This is $d_{\text{in}} \le 16\text{K}$ for most LLM layers (up to $\approx 28\text{K}$ for the largest FFN down-projections), so $\mathbf{H}$ is stored and factorized once per layer. Optimal Brain Compression (OBC) performs exact greedy OBS row by row, with cost $O(d_{\text{in}}^3)$ per row in the worst case [36], which was practical for ResNet and BERT but not for hundred-billion-parameter models, motivating SparseGPT.

5.5 SparseGPT

SparseGPT [42] solves (5.3) approximately but fast. Its ideas are best seen as three modifications of row-wise OBS.

5.5.1 Idea 1: a fixed column order and the Cholesky shortcut

OBC picks the next weight to remove *greedily* per row, so different rows follow different removal orders, and each row needs its own sequence of inverse updates. SparseGPT instead processes the columns of $\mathbf{W}$ in a *fixed* order $j = 1, \dots, d_{\text{in}}$, deciding at each step, for *every* row simultaneously, whether w_{ij} is pruned, and updating the not-yet-processed columns to compensate. The same inverse Hessian sequence then serves all rows.

Proposition 5.1 (Cholesky form of the sequential OBS update)**.** *Let $\mathbf{H}^{-1} = \mathbf{U}^\top \mathbf{U}$ with $\mathbf{U}$ upper triangular (the upper Cholesky factor of the inverse). Suppose columns $1, \dots, j-1$ have been processed (eliminated). Then (a) the inverse of the Hessian restricted to columns $j, \dots, d$ is $\mathbf{U}_{j:,j:}^\top \mathbf{U}_{j:,j:}$; (b) the OBS update that sets w_j to a target value $\tilde{w}_j$ (zero when pruned) and compensates the remaining columns $k > j$ is*

$$w_k \;\leftarrow\; w_k - \frac{w_j - \tilde{w}_j}{U_{jj}}\, U_{jk}, \qquad k > j, \tag{5.4}$$

and the associated loss increase for a single row is $(w_j - \tilde{w}_j)^2/(2U_{jj}^2)$.

Proof. (a) Partition $\mathbf{U} = \begin{pmatrix} \mathbf{U}_{11} & \mathbf{U}_{12} \\ 0 & \mathbf{U}_{22} \end{pmatrix}$ with the first block being columns $1..j-1$. Then $\mathbf{H}^{-1} = \mathbf{U}^\top\mathbf{U}$ has blocks $A = \mathbf{U}_{11}^\top\mathbf{U}_{11}$, $B = \mathbf{U}_{11}^\top\mathbf{U}_{12}$, $C = \mathbf{U}_{12}^\top\mathbf{U}_{12} + \mathbf{U}_{22}^\top\mathbf{U}_{22}$. By block inversion, the inverse of the trailing block $\mathbf{H}_{22}$ of $\mathbf{H}$ is the Schur complement $C - B^\top A^{-1} B = \mathbf{U}_{12}^\top\mathbf{U}_{12} + \mathbf{U}_{22}^\top\mathbf{U}_{22} - \mathbf{U}_{12}^\top\mathbf{U}_{11}(\mathbf{U}_{11}^\top\mathbf{U}_{11})^{-1}\mathbf{U}_{11}^\top\mathbf{U}_{12} = \mathbf{U}_{22}^\top\mathbf{U}_{22}$. (b) Applying (5.1) with the constraint $\delta_j = \tilde{w}_j - w_j$ to the trailing system gives $\delta = \frac{\tilde{w}_j - w_j}{[\mathbf{H}_{22}^{-1}]_{11}} \mathbf{H}_{22}^{-1} e_1$. Because $\mathbf{U}_{22}$ is upper triangular, $[\mathbf{H}_{22}^{-1}]_{11} = U_{jj}^2$ and the first row (and column) of $\mathbf{H}_{22}^{-1}$ has entries $U_{jj}U_{jk}$. Hence $\delta_k = \frac{(\tilde{w}_j - w_j)}{U_{jj}^2} U_{jj}U_{jk} = -\frac{(w_j - \tilde{w}_j)}{U_{jj}} U_{jk}$. The loss increase is $(w_j - \tilde{w}_j)^2/(2[\mathbf{H}_{22}^{-1}]_{11}) = (w_j - \tilde{w}_j)^2/(2U_{jj}^2)$. □

Proposition 5.1 is the computational heart of both GPTQ and SparseGPT: a single Cholesky factorization of $\mathbf{H}^{-1}$ replaces the sequence of $O(d^2)$ inverse updates. (GPTQ [43] uses the same shortcut with rounding to a quantization grid in place of $\tilde{w}_j = 0$.) The per-column compensation (5.4) is a rank-one update of all not-yet-processed columns of all rows, computable as a matrix operation: for a block of columns, one forms $\mathbf{E} = (\mathbf{W}_{:,j} - \tilde{\mathbf{W}}_{:,j})/U_{jj}$ and subtracts $\mathbf{E}\,\mathbf{U}_{j,j+1:}$.

5.5.2 Idea 2: adaptive mask selection in blocks

Using a fixed order alone does not choose *which* weights to prune. SparseGPT selects the mask in blocks of B_s columns (e.g., 128): at the start of each block, it computes OBS saliencies $\varepsilon_{ij} = w_{ij}^2/(U_{jj})^2$ for all entries of the block using the *current* (already compensated) weights, and prunes the lowest $p\%$ of the block's entries (for uniform per-layer sparsity) or the lowest 2 out of every 4 consecutive columns per row for 2:4. Saliency uses $U_{jj}^2 = [\mathbf{H}_{(j:,j:)}^{-1}]_{11}$, which *already accounts for compensation from later columns* but not from earlier ones (those are

Algorithm 3 SparseGPT layer pruning (after [42])

Require: $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, calibration inputs $\mathbf{X}$, target sparsity p (or pattern $N{:}M$), block sizes B (update) and B_s (mask), damping λ

$\mathbf{H} \leftarrow 2\mathbf{X}\mathbf{X}^\top$; $\mathbf{H} \leftarrow \mathbf{H} + \lambda \,\text{mean}(\text{diag}\,\mathbf{H})\, I$ ▷ damping; dead columns handled separately
$\mathbf{U} \leftarrow \text{Cholesky}_{\text{upper}}(\mathbf{H}^{-1})$ ▷ $\mathbf{H}^{-1} = \mathbf{U}^\top \mathbf{U}$
$\mathbf{M} \leftarrow \mathbf{1}$; $\mathbf{E} \leftarrow 0$
for $j_0 = 0, B, 2B, \cdots < d_{\text{in}}$ **do**
  $\mathbf{E} \leftarrow \mathbf{0} \in \mathbb{R}^{d_{\text{out}} \times B}$
  for $j = j_0, \ldots, j_0 + B - 1$ **do**
    if $j \bmod B_s = 0$ **then** ▷ choose mask for next B_s columns
      $\varepsilon_{:,j:j+B_s} \leftarrow \mathbf{W}^{\odot 2}_{:,j:j+B_s} \,/\, \text{diag}(\mathbf{U})^2_{j:j+B_s}$
      $\mathbf{M}_{:,j:j+B_s} \leftarrow$ prune the lowest-ε fraction p (or lowest 2 of every 4 columns for 2:4)
    end if
    $\tilde{\mathbf{W}}_{:,j} \leftarrow \mathbf{W}_{:,j} \odot \mathbf{M}_{:,j}$
    $\mathbf{E}_{:,j-j_0} \leftarrow (\mathbf{W}_{:,j} - \tilde{\mathbf{W}}_{:,j})/U_{jj}$
    $\mathbf{W}_{:,j:j_0+B} \leftarrow \mathbf{W}_{:,j:j_0+B} - \mathbf{E}_{:,j-j_0}\, \mathbf{U}_{j,j:j_0+B}$ ▷ within-block compensation
  end for
  $\mathbf{W}_{:,j_0+B:} \leftarrow \mathbf{W}_{:,j_0+B:} - \mathbf{E}\, \mathbf{U}_{j_0:j_0+B,\, j_0+B:}$ ▷ lazy batch update
end for
return $\mathbf{W} \odot \mathbf{M}$ (the compensated weights with zeros inserted)

already frozen). This asymmetry is an approximation: weights in later columns are cheaper to prune because they have more columns left to absorb the change. SparseGPT's blockwise selection partially mitigates this by comparing weights within a block.

5.5.3 Idea 3: lazy batch updates

Updating all remaining columns after each column is memory-bound. SparseGPT accumulates compensation inside the current block of B columns (processing columns within the block sequentially with updates restricted to the block), stores the accumulated error matrix $\mathbf{E}$, and applies one large matrix multiplication $\mathbf{W}_{:,j+B:} \leftarrow \mathbf{W}_{:,j+B:} - \mathbf{E}\,\mathbf{U}_{j:j+B,j+B:}$ at the end of the block. This turns many small updates into a single GEMM, and is mathematically equivalent to sequential updates because the update on later columns is linear.

Complexity. The Cholesky factorization of the $d_{\text{in}} \times d_{\text{in}}$ matrix costs $O(d_{\text{in}}^3)$. The compensation costs $O(d_{\text{out}} d_{\text{in}}^2)$ for the full sweep (each column update touches $d_{\text{out}} \times (d_{\text{in}} - j)$ entries). For a transformer of hidden size d with FFN width $\approx \alpha d$, the total over all matrices of a block is $O(d^3)$ with a constant depending on α; this is the reason the reported runtime for the largest models is hours on a single GPU rather than days. Frantar and Alistarh report pruning OPT-175B and BLOOM-176B to 50% in under about 4.5 hours on a single 80 GB GPU, processing the model block by block with 128 calibration sequences of 2048 tokens from C4[†] [42]. Memory is dominated by $\mathbf{H}$ in fp32 (e.g., for $d_{\text{in}} = 28{,}672$ in a 70B-class FFN down-projection: $28672^2 \times 4 \approx 3.3\,\text{GB}$) plus the layer's weights.

Sequential versus parallel calibration. SparseGPT feeds each block the activations produced by the *already pruned* preceding blocks (sequential error-aware mode), so that the Hessian reflects the actual inputs the pruned layer will see. Using unpruned activations is cheaper to batch but ignores error accumulation (Proposition 3.4).

5.5.4 What SparseGPT assumes and where it fails

- **Calibration sufficiency. $\mathbf{H}$** is estimated from n token vectors ($n = 128 \times 2048 = 262{,}144$ in the standard recipe). For $d_{\text{in}} = 4096$ the estimate has $d_{\text{in}}^2/2 \approx 8.4$M free parameters from 2.6×10^5 samples, so it is well determined only in the dominant subspace; damping regularizes the rest.
- **Linear reconstruction objective.** Minimizing squared output error treats all output directions equally; it ignores that the loss may be far more sensitive to certain directions. Alternatives weight outputs by gradient information (e.g., global objectives in SparseLLM [52]).
- **Distribution shift.** Mask and compensation are tuned to the calibration distribution (Chapter 15). Compensation *overfits* the calibration inputs more than Wanda-style scoring does because it changes retained weight values, not only the mask.
- **Greedy column order.** Fixed left-to-right processing introduces order dependence; permuting columns changes the result. Column order by activation magnitude ("act-order" in GPTQ) has analogues for pruning.

5.6 Optimal updates for a fixed mask and ADMM refinement

Given a fixed support (mask) S for one row, the reconstruction-optimal weights on S have a closed form.

Proposition 5.2 (Optimal weights on a fixed support)**.** *For a row w, support S with $|S| = k$ and $\mathbf{H}_{SS} \succ 0$, the minimizer of $\|(w - \hat{w})^\top \mathbf{X}\|_2^2$ over $\hat{w}$ with $\operatorname{supp}(\hat{w}) \subseteq S$ is $\hat{w}_S = \mathbf{H}_{SS}^{-1} \mathbf{H}_{S,:}\, w$, i.e., $\hat{w}_S = (\mathbf{X}_S \mathbf{X}_S^\top)^{-1} \mathbf{X}_S \mathbf{X}^\top w$.*

Proof. The objective is $\frac{1}{2}(w - \hat{w})^\top \mathbf{H}(w - \hat{w})$ up to scaling. Writing $\hat{w} = Pv$ with P the selection matrix of S and differentiating in v: $P^\top \mathbf{H}(Pv - w) = 0$, so $\mathbf{H}_{SS} v = \mathbf{H}_{S,:} w$. □

Computing Proposition 5.2 for every row costs $O(k^3)$ per row with a different S per row, which is prohibitive for unstructured masks (each row has a different support, so each needs its own factorization). SparseGPT's sequential compensation approximates this solution at lower cost. Boza [50] and Meng et al. (ALPS) [51] instead apply ADMM or penalty methods that alternate between the unconstrained least-squares solution (a shared linear system, since the full $\mathbf{H}$ is common) and a projection onto the sparsity constraint (top-k); ALPS adds a continuation of the penalty parameter and a post-processing step with preconditioned conjugate gradients on the fixed support. The reported benefit is largest at high sparsity (60–90%), where SparseGPT's greedy compensation leaves a larger gap to the optimum [51]. SparseLLM [52] decomposes the *global* objective into layerwise subproblems with auxiliary variables to improve over purely local reconstruction at high sparsity.

Table 5.1: Weight-level perplexity (WikiText-2) at 50% sparsity as reported in cited papers. Entries marked † are as recalled by the author from the Wanda paper's tables and were not re-verified; the dense LLaMA-2-7B value 5.12 and the 2:4 MaskLLM value were verified (‡) from the MaskLLM abstract. Absolute values depend on tokenizer, context length and evaluation script.

Model	Dense	Magnitude	SparseGPT	Wanda	MaskLLM 2:4
LLaMA-7B (unstructured 50%)	5.68†	17.29†	7.22†	7.26†	–
LLaMA-2-7B (unstructured 50%)	5.12‡	14.89†	6.51†	6.42†	–
LLaMA-2-7B (2:4)	5.12‡	–	≥ 10‡ (best prior)	–	6.72‡

5.7 Practical details that matter

Damping. $\mathbf{H} \leftarrow \mathbf{H} + \lambda \,\mathrm{mean}(\mathrm{diag}\,\mathbf{H})I$ with λ of order 10^{-2} is a typical setting in open-source implementations of GPTQ and SparseGPT. Too little damping makes $\mathbf{H}^{-1}$ explode along poorly sampled directions and compensation updates huge; too much reverts toward diagonal OBD and loses compensation benefit. Damping equals a Tikhonov prior on the update magnitude.

Dead input channels. Columns with $\mathbf{H}_{jj} = 0$ (never activated) are pruned outright and their diagonal set to one to keep $\mathbf{H}$ invertible.

Calibration size. The reconstruction loss is the empirical average; more samples reduce variance but with diminishing returns beyond $\sim 10^5$ tokens for standard 7B models according to ablations in the original works; for high sparsity, more data helps more [185].

Precision. Hessian accumulation and Cholesky are done in fp32 (fp16 is numerically fragile), while weights remain in bf16/fp16 between layers.

Mask patterns. The same machinery supports $N{:}M$ (choose the lowest $M - N$ of each M-group per row) and structured row/column removal at much worse quality; only the mask-selection line of Algorithm 3 changes.

5.8 Summary of second-order pruning

Key lessons of Chapter 5

1. Second-order pruning works at LLM scale because the layerwise problem has an *exact* shared Hessian $2\mathbf{X}\mathbf{X}^\top$, rows decouple, and a Cholesky factorization of the inverse turns sequential OBS into a cheap blocked update (Propositions 3.3, 3.2, 5.1).
2. *Compensation* (updating retained weights) is what lets SparseGPT beat magnitude pruning by a wide margin at 50%; the scoring criterion itself is only a part of the gain.
3. The approach is a *local* approximation; at 60–70%+ sparsity, errors accumulate, and better solvers (ADMM, ALPS), allocation (OWL), and some recovery training are needed.
4. The cost is dominated by Hessian formation and Cholesky; it is tractable for 100B-parameter models on a single GPU in hours, not days.

Chapter 6

Activation-Aware Scoring, Layerwise Allocation and Mask Refinement

SparseGPT showed that second-order reconstruction can prune LLMs, but its cost (Hessian factorization per layer) and complexity motivated a simpler question: how much of its benefit comes from the scoring rule, and how much from weight compensation? Wanda [44] and the methods in its wake (RIA [45], GBLM-Pruner [48], Pruner-Zero [49]) argue that most of the gain at 50% comes from a cheap activation-aware score. A second thread, layerwise allocation (OWL [46], AlphaPruning [54], BESA [53]), shows that *where* sparsity is placed matters at least as much as the per-weight score. A third, training-free mask refinement (DSnoT [47]), swaps entries of an initial mask to reduce reconstruction error.

6.1 Wanda: weights and activations

Score. For a linear layer with weights $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration inputs $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, Wanda scores each weight by

$$S_{ij} = |W_{ij}| \cdot \|\mathbf{X}_{j,:}\|_2, \tag{6.1}$$

the weight magnitude times the ℓ_2 norm of the j-th input channel across the calibration tokens. Within each output row i (the *comparison group*), the lowest-scoring fraction p of weights is set to zero. No weight update is performed.

6.1.1 Why the Wanda score is a diagonal reconstruction criterion

Proposition 6.1 (Wanda equals OBD on the layerwise objective)**.** *Consider the layerwise reconstruction objective* (5.3) *with Hessian* $\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$*. The single-weight OBD saliency of* W_{ij} *(diagonal Hessian, no compensation) is* $\frac{1}{2}H_{jj}W_{ij}^2 = W_{ij}^2\|\mathbf{X}_{j,:}\|_2^2$*. Ranking weights within a row by this quantity is identical to ranking by the Wanda score* (6.1)*.*

Proof. $H_{jj} = 2\sum_t X_{jt}^2 = 2\|\mathbf{X}_{j,:}\|_2^2$, so $\frac{1}{2}H_{jj}W_{ij}^2 = W_{ij}^2\|\mathbf{X}_{j,:}\|_2^2 = S_{ij}^2$. Squaring is monotone on non-negative reals, so orderings coincide. □

Proposition 6.1 identifies Wanda as OBD applied to the layer-local objective: the loss increase from zeroing W_{ij} *alone* on the calibration data is exactly S_{ij}^2 (not an approximation, by Proposition 3.3). What Wanda ignores relative to OBS/SparseGPT is (a) off-diagonal terms of $\mathbf{H}$ (correlation between input channels) and (b) the compensation update. The first matters when input channels are strongly correlated; the second matters when removed weights can be absorbed by their correlated neighbors.

Remark 6.1 (Same-row comparison matters). The per-row comparison group guarantees every output neuron has the same sparsity, which Wanda's authors found important for LLMs, relative to comparing scores across the layer or across layers. The reason is consistent with Proposition 3.2: output rows are independent subproblems, and removing all of a row's large-scale

Algorithm 4 Wanda pruning of one linear layer [44]

Require: $\mathbf{W}$, input activations $\mathbf{X}$ for the layer, sparsity p or pattern $N{:}M$

$a_j \leftarrow \|\mathbf{X}_{j,:}\|_2$ for $j = 1..d_{\text{in}}$ ▷ accumulated over calibration tokens
$S_{ij} \leftarrow |W_{ij}| \cdot a_j$
for each output row i **do**
    $\mathcal{P}_i \leftarrow$ indices of the $\lfloor p\, d_{\text{in}} \rfloor$ smallest S_{ij} ▷ or per M-group for $N{:}M$
    $W_{ij} \leftarrow 0$ for $j \in \mathcal{P}_i$
end for
return W

connections would zero out that output feature, an error that no other row can compensate. Per-row balanced pruning also matches the structure required by $N{:}M$ patterns (Chapter 7).

Example 6.1 (Worked Wanda example; exact arithmetic)**.** Let $d_{\text{in}} = 4$ with input-channel norms $a = (1,\ 5,\ 0.5,\ 0.2)$ and two rows

$$w_1 = (0.5,\ -0.2,\ 0.1,\ 0.9), \qquad w_2 = (0.35,\ 0.4,\ -0.6,\ 0.05).$$

Wanda scores: $S_1 = (0.5,\ 1.0,\ 0.05,\ 0.18)$ and $S_2 = (0.35,\ 2.0,\ 0.30,\ 0.01)$. At 50% per row we keep the top two per row: row 1 keeps channels 1,2 (weights $0.5, -0.2$), row 2 keeps channels 1,2 $(0.35, 0.4)$. Magnitude pruning keeps channels 1,4 in row 1 $(0.5, 0.9)$ and channels 2,3 in row 2 $(0.4, -0.6)$. The two methods retain *disjoint* supports in both rows. If channel 2 is a high-activation outlier channel (norm 5), magnitude pruning discards connections that process the largest-variance signal; the summed squared saliency $\sum_{j \in \text{pruned}} S_{1j}^2$ of the zeroed weights in row 1 (the loss increase if cross-terms are ignored) is $0.05^2 + 0.18^2 = 0.0349$ for Wanda, but $1.0^2 + 0.05^2 = 1.0025$ for magnitude pruning, which is about 29× larger. The illustration is deliberately extreme (a 5× larger channel norm), but real LLM layers have outlier channels with ratios well above this [147, 149].

6.1.2 Results and where Wanda is weaker

At 50% unstructured sparsity Wanda matches SparseGPT on LLaMA-family models (Table 5.1). On the Wanda paper's tables the gap appears at structured 2:4 patterns, where SparseGPT's compensation is more valuable (for LLaMA-7B, 2:4 perplexity was roughly 11.0 for SparseGPT and 11.5 for Wanda†), and at high unstructured sparsity ($\geq$60–70%), where compensation and allocation both matter. These patterns are consistent with Proposition 6.1: compensation and off-diagonal correlation matter more when each removal has fewer neighbors left to absorb it.

Speed. Wanda needs one forward pass to collect a_j and a per-row sort; no Hessian and no matrix factorization. The cost is dominated by the calibration forward pass, so Wanda prunes models in a small fraction of SparseGPT's time. It also parallelizes trivially and has negligible extra memory.

Robustness to calibration data. Because Wanda uses only the diagonal second moments (a d_{in}-vector per layer), it needs far fewer calibration tokens than a full Hessian: the statistic converges quickly. The original paper reports that Wanda is relatively insensitive to the number

of calibration samples[†]; other studies show that the *domain* of calibration data still matters for downstream behavior [185, 186], particularly for code and math (Chapter 15).

6.2 RIA and relatives: relative importance and channel permutation

Relative Importance and Activations (RIA) [45] observes that magnitude-based scores can eliminate nearly all connections of a given input or output channel. It normalizes each weight by the sums of absolute weights along its row and column, and multiplies by a power of the activation norm:

$$\mathrm{RIA}_{ij} = \Big(\frac{|W_{ij}|}{\sum_{j'} |W_{ij'}|} + \frac{|W_{ij}|}{\sum_{i'} |W_{i'j}|}\Big) \cdot \|\mathbf{X}_{j,:}\|_2^a, \tag{6.2}$$

with a a hyperparameter (the authors recommend a value around 0.5 as best of their experiments, as I recall[†]). The two normalized terms measure the weight's importance relative to its output neuron's other inputs and relative to its input channel's other outputs, which guards against *channel collapse*. RIA additionally proposes a *channel permutation* for N:M sparsity: permute input channels before applying the 2:4 constraint so that large-score weights are spread across groups, reducing the loss from forced pruning of important weights that happen to be adjacent. The permutation is an assignment problem solved approximately; since a permutation of input channels can be absorbed into the preceding layer's output channels (or the weight order), it leaves the function unchanged [45, 59].

Proposition 6.2 (Channel permutation can only help the best N:M mask)**.** *For a score matrix S and any permutation π of columns, let $V_{N:M}(S)$ be the maximum total retained score over N:M-compliant masks. Then* $\max_\pi V_{N:M}(S\,\Pi_\pi) \geq V_{N:M}(S)$.

Proof. The identity permutation is feasible, so the maximum over π is at least the value at the identity. □

The proposition is trivial but states the logic of the permutation approach: it enlarges the search space of masks to those compatible with *some* column grouping. Pool and Yu [59] proposed searching permutations by swapping channels to maximize retained magnitude under 2:4.

6.3 Gradient-informed and learned scores

- **GBLM-Pruner** [48] multiplies weight magnitudes by normalized gradient norms (and optionally activations), using calibration gradients to approximate sensitivity of the loss rather than of the layer output.
- **Pruner-Zero** [49] searches a space of symbolic expressions over weights, gradients and activations by genetic programming and finds scores that transfer across model sizes; the resulting formulas are comparable to Wanda.
- **Sparse-aware regularization of the score.** Some methods mix magnitude and activation statistics with exponents tuned on validation perplexity; they offer small improvements at the cost of tuning.

Table 6.1: Summary of layerwise allocation methods.

Method	Sensitivity signal	Needs data?	Notes
Uniform	none	no	strong baseline up to ∼50%
OWL [46]	fraction of outlier scores	calibration activations	hyperparameters M, λ; simple
AlphaPruning [54]	heavy-tail exponent of weight spectrum	no	data-free allocation
BESA [53]	learned differentiable sparsity	calibration + optimization	blockwise reconstruction
ZipLM [57]	measured error and latency tables	calibration	hardware-aware structured setting

An important caveat is that all such scores are tuned for perplexity (and, secondarily, zero-shot accuracy) at 50%. Their ranking can change on generative tasks and at higher sparsity (Chapter 15); the findings of Lucas et al. [121] on reasoning models suggest that the *calibration objective*, not the scoring rule, is the dominant factor there.

6.4 Non-uniform layerwise sparsity

By Equation (3.8), layers should be pruned in proportion to their tolerance. Empirically, uniform per-layer sparsity is suboptimal at high sparsity, and three families of allocation rules have been proposed.

OWL: outlier-weighted layerwise sparsity. Yin et al. [46] define for each layer an *outlier ratio*: the fraction of weights whose Wanda score exceeds a multiple M of the layer's mean score. Layers with more outliers receive *lower* sparsity. The allocation maps outlier ratios linearly into a band $[s - \lambda, s + \lambda]$ around the global target s while keeping the mean fixed. The authors report large perplexity reductions at 70% sparsity over uniform allocation on LLaMA models[†] [46]; both M and λ are hyperparameters (values of $M = 5$ and λ of order 0.08 are typical, as I recall[†]).

AlphaPruning. Lu et al. [54] use heavy-tailed self-regularization theory: they fit a power-law exponent α to the empirical spectral density of each layer's weight matrix. A smaller α indicates a "better trained" layer, so these receive lower sparsity. The exponent is computed from weights alone, with no calibration data.

BESA and learned allocation. BESA [53] learns per-block sparsity allocation with differentiable masks and a target-sparsity constraint, optimizing blockwise reconstruction error; ZipLM [57] derives inference-aware allocation by dynamic programming over measured per-layer latencies and errors.

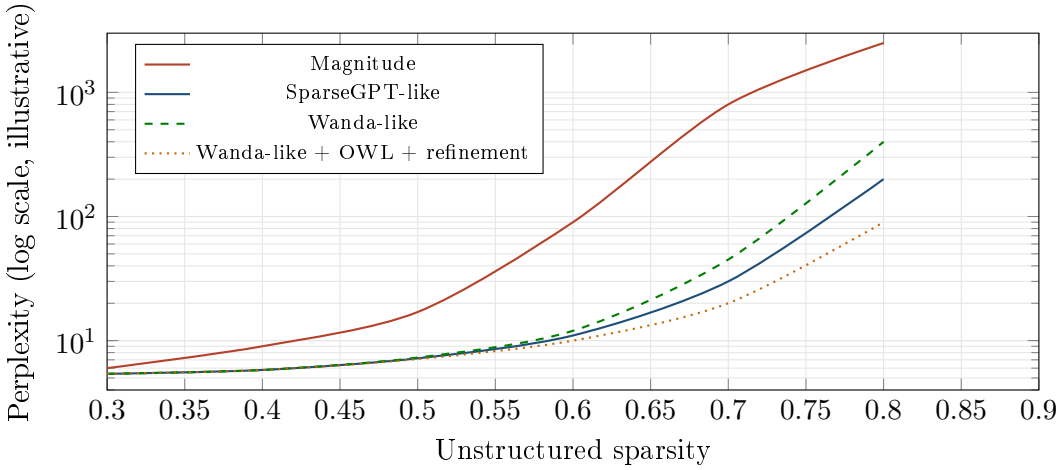


Figure 6.1: Schematic (synthetic, illustrative; not measured data) perplexity-versus-sparsity curves for a 7B-class model, encoding the commonly reported ordering and the "knee" near 50–60% where one-shot methods begin to diverge. Allocation and refinement mostly act to the right of the knee.

6.5 DSnoT and training-free mask refinement

Dynamic Sparse No Training (DSnoT) [47] starts from a Wanda or SparseGPT mask and performs iterative *prune-and-grow* swaps without weight updates or gradients: it grows back (un-prunes) a weight that most reduces reconstruction error and prunes a currently retained weight whose removal is least harmful, using the signed reconstruction error and Wanda-style scores. The procedure is a discrete local search on the mask in the spirit of RigL (Chapter 13) but with the reconstruction error of the layer as objective. It yields measurable improvements at 60–70% sparsity, at moderate extra compute, without altering retained values, so the resulting weights remain a strict subset of the original weights.

6.6 Relationship to activation-aware quantization

The central insight of Wanda—weights multiplying large activations matter more—is the same as in activation-aware weight quantization (AWQ [177]), SmoothQuant [178] and LLM.int8() [149], which treat outlier channels specially. It also coincides with the observation of "massive activations" and "super weights" [147, 148]: a handful of weights and channels are disproportionately important, and pruning them causes catastrophic degradation. Yu et al. [148] report that pruning a single such super weight can destroy the model's ability to produce coherent text, whereas pruning thousands of other weights has little effect. Practitioners therefore often protect identified outlier channels from pruning (and from aggressive quantization).

Pitfalls of simple scores

- Scores are computed on calibration activations; rare but important input channels (code, languages, long-tail facts) may have small calibration norms and get pruned (Chapter 15).
- Wanda's no-update design keeps the model's retained weights untouched, which is good

for stability but means error cannot be redistributed; at high sparsity this limits it.

- Per-row balanced sparsity forces equal sparsity in every output row, even when some outputs (e.g., rows feeding massive-activation dimensions) are critical.

6.7 Summary

Key lessons of Chapter 6

1. Wanda is the diagonal-Hessian (OBD) special case of layerwise reconstruction (Proposition 6.1); it is fast and matches SparseGPT near 50% on LLaMA-family models, with a gap opening at 2:4 and at high sparsity.
2. Non-uniform allocation (OWL, AlphaPruning, BESA) and mask refinement (DSnoT, ALPS) are the main levers beyond 50%.
3. Simple scores are tuned on perplexity; their behavior on reasoning, long-context, and bias is much less characterized (Chapters 15, 17).

Chapter 7

Semi-Structured N:M Sparsity and Sparse Tensor Cores

Unstructured sparsity offers the best accuracy per removed weight but poor hardware efficiency; structured sparsity offers perfect hardware efficiency but costs the most accuracy. N:M sparsity sits between them: in every aligned group of M consecutive weights, at most N are non-zero. With 2:4 (50% sparsity), NVIDIA introduced hardware support in the Ampere architecture, with sparse tensor cores that skip the zeros [58, 70]. This chapter explains the format, the hardware mechanics, the accuracy cost, mask-learning methods (SR-STE, transposable masks, MaskLLM), extensions beyond 2:4, and the empirical status of end-to-end speedups.

7.1 The 2:4 format and its hardware mechanics

7.1.1 Compressed representation

For each group of four consecutive elements along the reduction (K) dimension of a weight matrix, exactly two are retained. The compressed matrix stores the two retained values per group plus a 2-bit index for each, so 4 bits of metadata per group of four weights. Figure 7.1 illustrates this.

Storage arithmetic. For a group of four weights at b bits per value, the dense group takes $4b$ bits and the compressed group takes $2b+4$ bits. Table 7.1 lists the resulting size ratios; they are exact arithmetic given the 2-bit-index format.

The metadata overhead is thus a growing fraction of the stored tensor as precision falls: at 4 bits, one third of the compressed storage is metadata. The information-theoretic cost of encoding which two of four positions are kept is only $\log_2 \binom{4}{2} = \log_2 6 \approx 2.585$ bits per group, versus 4 bits stored; the format trades 1.4 bits per group of redundancy for regular decoding.

7.1.2 Sparse matrix-multiply mechanics

Sparse tensor cores execute a matrix-multiply-accumulate in which operand A (the weights) is 2:4 compressed and operand B (typically activations) is dense. For each retained weight in a group, the 2-bit metadata selects which element of the corresponding group of four in B to multiply; the hardware therefore performs half the multiplications for the same logical K-extent. This is exposed at the instruction level as a sparse MMA variant (in the PTX instruction set, the `mma.sp` family) and through libraries such as cuSPARSELt [71] and CUTLASS. NVIDIA's A100 whitepaper states a peak dense FP16 tensor throughput of 312 TFLOPS and a sparse peak of 624 TFLOPS† [70]; that is, the 2× is a peak math-throughput figure on the matmul itself.

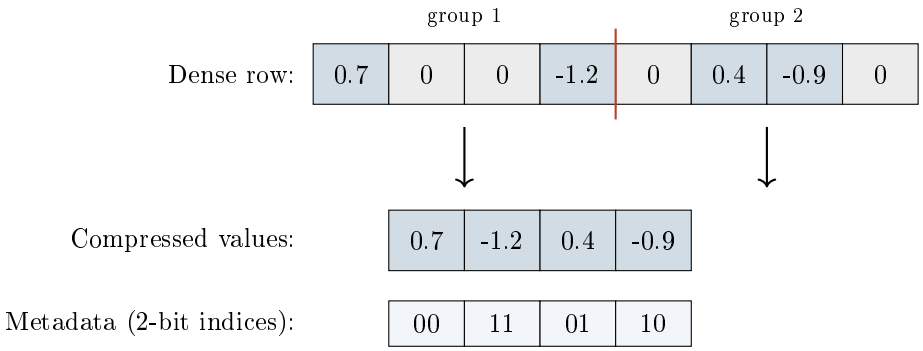


Figure 7.1: The 2:4 format. In each group of four consecutive weights (along the reduction dimension), two are kept. The compressed matrix stores kept values contiguously plus a 2-bit position index per kept value.

Table 7.1: Size of a 2:4-compressed weight tensor relative to dense, for different value precisions (computed: $(2b + 4)/(4b)$).

Value precision b	Dense bits per 4 weights	Compressed bits per 4 weights	Ratio
16-bit (FP16/BF16)	64	36	0.5625
8-bit (INT8/FP8)	32	20	0.625
4-bit (INT4/FP4)	16	12	0.75

What 2:4 speeds up and what it does not

- **Math-bound GEMMs** (prefill, large batch): up to 2× peak tensor-core throughput; realized matmul speedups are typically in the range of 1.3–1.8× for large shapes and lower for small ones (author's summary of reported kernels, approximate).
- **Memory-bound GEMV/GEMM at small batch** (decode): speedup is bounded by the weight-byte reduction—at most $1/0.5625 \approx 1.78\times$ for 16-bit values, 1.6× for 8-bit, 1.33× for 4-bit, per Table 7.1—and the activation operand is still read in full.
- **Not accelerated**: attention score computation, KV cache reads, layer norms, embeddings and sampling; Amdahl's law applies. End-to-end gains are therefore smaller than matmul gains.

Hardware generations. The Ampere A100 introduced 2:4 support. NVIDIA's later architectures (Hopper, Blackwell) retain structured-sparsity MMA support; the SlideSparse authors report running sparse kernels in FP4, INT8, FP8, BF16 and FP16 on A100, H100, B200, RTX 4090, RTX 5080 and DGX Spark‡ [64]. The author understands that other vendors have introduced comparable structured-sparse matrix instructions (for example AMD's CDNA3 generation), but has not verified details of those designs and treats them as "to be confirmed" in this survey.

Table 7.2: Mask flexibility and retained energy for i.i.d. standard Gaussian weights under magnitude selection. Mask entropy is exact. Retained energy is a Monte Carlo estimate by the author (6.4×10^6 samples, seed fixed; statistical error below 10^{-3}) of the fraction of $\sum w^2$ kept, versus 0.9287 for global 50% magnitude pruning (analytic, Table 3.1). These are properties of the random model, not of LLM weights.

Pattern (50%)	$\log_2 \binom{M}{N}$ bits/group	Mask entropy (bits/weight)	Retained energy (Gaussian)
1:2	1.00	0.500	0.8185
2:4	2.59	0.646	0.8678
4:8	6.13	0.766	0.8968
8:16	13.65	0.853	0.9124
16:32	29.16	0.911	0.9205
unstructured (global)	–	$\to 1$	0.9287

7.2 Accuracy: why 2:4 is harder than unstructured 50%

7.2.1 Counting masks

Unstructured 50% sparsity over a group of M weights allows $\binom{M}{M/2}$ masks per group *on average* (the global constraint is the only restriction), whereas N:M requires exactly N per group, reducing flexibility. The entropy per weight of the mask family, $\log_2 \binom{M}{N}/M$, quantifies the flexibility.

Table 7.2 shows, for a purely random Gaussian model, that 2:4 keeps 86.8% of weight energy versus 92.9% for unstructured 50%, i.e., it discards 13.2% of the energy versus 7.1%—nearly twice as much. Real weights differ (heavier tails and activation-weighted importance) and the compensation of retained weights can reduce the gap, but the qualitative conclusion matches observations: 2:4 post-training pruning is substantially worse than unstructured 50% (LLaMA-2-7B perplexity of the order of 10 versus about 6.5, Table 5.1).

7.2.2 Reasoning and generative tasks

Perplexity is only one measure. The SlideSparse authors note that 2:4 pruning of Qwen3 reduces its reasoning-benchmark accuracy from 54% to 15% in their setting[‡] [64], a drop that would be invisible to a perplexity-based acceptance test; they conclude that milder patterns such as 6:8 (25% sparsity) are needed to retain accuracy. We return to this phenomenon in Chapters 15 and 17.

7.3 Post-training 2:4 methods

All of the one-shot methods of Chapters 5–6 extend to N:M by restricting the mask-selection step: SparseGPT chooses the lowest-saliency $M - N$ entries per group per row with compensation; Wanda and RIA rank scores within each group (RIA additionally permutes channels, Proposition 6.2). Recovery uses (i) LoRA or low-rank adapters on the sparse model [85], (ii) full fine-tuning with the mask frozen, or (iii) logit distillation from the dense teacher. Because sparse tensor cores require the masked weights to be *exactly* zero, merging LoRA adapters into

sparse weights destroys the pattern, so adapters are kept separate (adding a dense low-rank path at inference) or are trained with mask-aware methods.

7.4 Learning the mask: SR-STE, transposable masks, MaskLLM

7.4.1 Sparse-refined straight-through estimator (SR-STE)

Zhou et al. [60] train N:M-sparse networks from scratch by projecting weights to the N:M pattern in the forward pass and using a straight-through estimator in the backward pass. Plain STE has a problem: gradients w.r.t. the masked (pruned) weights are applied to dense weights that have no effect on the loss, so the mask can fluctuate. SR-STE adds a regularizer that decays pruned weights toward zero:

$$w \leftarrow w - \eta\big(\nabla_{\tilde{w}}\mathcal{L}(\tilde{w}) + \lambda_W\,(\bar{\boldsymbol{m}} \odot w)\big), \qquad \tilde{w} = \boldsymbol{m} \odot w, \tag{7.1}$$

with $\bar{\boldsymbol{m}} = 1 - \boldsymbol{m}$. The decay term stabilizes the mask by making pruned weights smaller, reducing the churn of the top-N selection.

7.4.2 Transposable masks for training

Training with sparse tensor cores requires the *backward* pass as well: computing $\nabla_x = W^\top \nabla_y$ involves $W^\top$, whose groups of four run along a different dimension. A mask that is N:M along both dimensions ("transposable") lets both forward and backward use sparse kernels. Hubara et al. [61] give efficient algorithms to find transposable masks (min-cost flow-based) at a small quality cost relative to a one-directional mask. For inference-only LLM compression this constraint is unnecessary; it matters when pursuing training speedups (Chapter 13).

7.4.3 MaskLLM

MaskLLM [62] learns 2:4 masks end to end on a large language modeling dataset with the weights *frozen*.

Formulation. For each group of four weights there are $\binom{4}{2} = 6$ candidate masks $\{\boldsymbol{m}^{(1)}, \dots, \boldsymbol{m}^{(6)}\}$. MaskLLM parameterizes a categorical distribution with learnable logits $\boldsymbol{\pi} \in \mathbb{R}^6$ per group and samples the mask using the Gumbel-softmax reparameterization [19, 20]:

$$\tilde{\boldsymbol{y}} = \mathrm{softmax}\Big(\frac{\log \boldsymbol{p} + \boldsymbol{g}}{\tau}\Big), \quad g_k \sim \mathrm{Gumbel}(0,1), \quad \tilde{\boldsymbol{m}} = \sum_{k=1}^{6} \tilde{y}_k\, \boldsymbol{m}^{(k)}, \tag{7.2}$$

with $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{\pi})$ and τ a temperature. The forward pass multiplies the frozen weights by $\tilde{\boldsymbol{m}}$ (a convex combination of valid masks, which is a hard mask when $\tilde{\boldsymbol{y}}$ is one-hot), and the language-modeling loss is back-propagated to the logits. Because there are six candidates per group, the parameter overhead of learning is 6 logits per four weights, which can be held in memory by processing the model with gradient checkpointing and by optimizing logits only. After training, the mask is the argmax of the logits for each group.

Algorithm 5 Learnable 2:4 mask training in the style of MaskLLM [62]

Require: frozen weights $\mathbf{W}$, training data $\mathcal{D}$, temperature schedule τ_t, initial logits $\boldsymbol{\pi}$ (optionally biased toward a prior mask from SparseGPT/Wanda)

for step $t = 1, \ldots, T$ **do**
  sample a batch $(x, y) \sim \mathcal{D}$
  for each group g of 4 weights **do**
    draw Gumbel noise; compute soft mask $\tilde{\boldsymbol{m}}_g$ by Eq. (7.2) (optionally straight-through hard sample)
  end for
  loss $\leftarrow \mathrm{CE}\big(f(x; \tilde{\mathbf{M}} \odot \mathbf{W}), y\big)$ (optionally plus a weight-magnitude regularizer on the retained entries)
  update logits $\boldsymbol{\pi}$ by gradient descent (weights stay frozen)
end for
$\mathbf{M} \leftarrow$ argmax mask per group
return M

Results. MaskLLM reports 2:4 results on LLaMA-2, Nemotron-4 and GPT-3-family models from 843M to 15B parameters; on WikiText-2 with a LLaMA-2 model, dense perplexity is 5.12, leading prior 2:4 methods reach 10 or more, and MaskLLM achieves 6.72 with learned masks and frozen weights‡ [62]. The authors also report that learned mask distributions *transfer* (as priors for new domains and tasks) and can be customized for downstream tasks without loss in their experiments‡.

Remark 7.1 (Why learned masks beat one-shot masks). One-shot masks optimize a layer-local proxy. MaskLLM optimizes the actual end-to-end language-modeling loss on a large dataset, through all layers simultaneously, over the discrete structure (2:4) that matters. The price is compute: mask training requires full forward and backward passes of the model on many tokens, a cost comparable to a short fine-tuning run, rather than a calibration pass. This puts it in the "sparse training" regime of cost rather than the "post-training" regime.

7.5 Beyond 2:4: milder patterns, hybrids, and activation sparsity

7.5.1 (2N-2):2N patterns and sliding windows

A drawback of 2:4 is that it forces 50% sparsity, and few models tolerate that on reasoning. SlideSparse [64] targets patterns of the form $(2N-2)$:$2N$, e.g., 6:8 (25% sparsity) or 4:6 (33%). It decomposes each block of $2N$ weights with at most $2N-2$ non-zeros into $N-1$ overlapping windows of four consecutive elements, each of which is 2:4-compliant, so that existing 2:4 sparse tensor-core kernels can execute them, and an "activation lifting" step rearranges the dense operand at near-zero cost, folded into per-token quantization. The theoretical upper bound on speedup is $N/(N-1)$ (since $N-1$ windows of 4 replace $2N$ dense elements, i.e., $2(N-1)$ useful multiplications versus $2N$): for 6:8, $N = 4$ and the bound is 4/3. The authors report 1.33× speedup on compute-bound workloads for Qwen2.5-7B at 6:8‡, within rounding of the bound.

Table 7.3: Software paths for N:M sparse inference (summary as understood by the author; check current versions).

Component	Role
cuSPARSELt [71]	NVIDIA library for structured-sparse matrix multiply; includes compression of weights and tuned kernels.
CUTLASS	Template library with sparse tensor-core kernels; basis for several frameworks.
PyTorch semi-structured tensors	Frameworks expose conversion of 2:4 weights to a compressed representation and dispatch to cuSPARSELt/CUTLASS kernels.
vLLM and serving engines [72]	Integrations of sparse kernels, including sparse–quantized combinations (see also [64, 66]).
TensorRT-class compilers	Build-time conversion with optional sparsity fine-tuning flows.

7.5.2 Hybrid sparse–dense formats

SpenseGPT [65] splits each weight matrix into a 2:4 region and a dense region, so the effective sparsity falls below 50% while standard sparse and dense GEMM libraries execute the respective regions; the one-shot pruning algorithm decides which rows or columns remain dense. On Qwen3-32B and Seed-OSS-36B the authors report up to 1.2× end-to-end decoding speedup with FP8 on B200 GPUs while preserving accuracy[‡], and claim this is the first one-shot-pruning demonstration of real end-to-end speedups from semi-structured sparse tensor cores on recent GPUs. The modest 1.2× is itself informative: it is the realized end-to-end figure after Amdahl effects, relative to a theoretical 2× matmul peak.

7.5.3 2:4 on activations

Haziza et al. [63] apply 2:4 sparsity to *activations* in the FFN. Models using squared ReLU have naturally sparse intermediate activations; enforcing 2:4 on them lets the second FFN matmul run on sparse tensor cores in both forward and backward passes, and the paper reports up to 1.3× FFN speedup with no accuracy loss[‡]. The reuse of activations (rather than weights) as the sparse operand reduces the preprocessing cost problem—activations are produced on the fly, so they must be compressed at each step, which the authors address with fused kernels.

7.6 Software stack

7.7 A realism check on speedups

How much can a 2:4 model be expected to speed up end to end? An Amdahl-style estimate separates the fraction f of time spent in sparsifiable matmuls and the speedup s on those. For $f = 0.7$ and $s = 1.5$, the end-to-end speedup is $1/(0.3 + 0.7/1.5) = 1.30$; for $f = 0.9$, $s = 1.78$ (decode, fp16 bound), the speedup is $1/(0.1 + 0.9/1.78) = 1.65$; for $f = 0.5$ (attention- and KV-heavy long-context decoding) with $s = 1.5$ it is 1.20. These are illustrative calculations consistent with the reported 1.2× to 1.3× end-to-end results of recent work[‡] [63, 65]. Meanwhile, 4-bit quantization of a 16-bit model gives close to 4× weight-byte reduction and, for memory-bound decode, a much larger gain than 2:4 with 16-bit values. 2:4 is therefore most attractive as

a *complement* to quantization where the reduction is shared across both, or in compute-bound regimes (prefill, large batch) where quantization alone does not reduce math cost as strongly on all hardware.

2:4 pitfalls

- Perplexity-only validation can hide large generative-reasoning losses [64].
- Sparsity must be exactly 2:4 along the reduction dimension; tensor-parallel sharding, fused QKV layouts, and gated-FFN concatenation can break alignment and require care.
- Adapters (LoRA) cannot be merged without destroying the pattern.
- Speedup requires tuned kernels for the specific shape and precision; small-M decode kernels for sparse-times-dense are less mature than dense counterparts.

Key lessons of Chapter 7

1. 2:4 is the only widely supported semi-structured pattern; it halves the math and shrinks 16-bit weights to 56% of dense size.
2. It is lossier than unstructured 50% (Table 7.2); learned masks (MaskLLM) close much of the perplexity gap but at training-run cost.
3. End-to-end speedups of 1.2–1.3× are the realized figures in recent work; theoretical 2× is a peak matmul number.
4. Milder patterns (6:8) and hybrid formats extend the trade-off space but need new kernels or decompositions.

Chapter 8

Structured Width Pruning: Channels, Heads, Hidden Dimensions

Structured pruning removes whole architectural units so that the result is a smaller *dense* model, executable on any hardware with unchanged kernels. The price is accuracy: coupled structures leave less room for compensation, and units are removed at coarser granularity. This chapter treats *width* pruning (FFN channels, attention heads, hidden dimension); depth pruning is Chapter 9, expert pruning Chapter 11, and the distillation pipelines that make structured pruning competitive in production Chapter 10.

8.1 Structures, couplings and parameter accounting

8.1.1 Coupled structures

A removable unit is a set of parameters that must be deleted *together* to leave a valid network. In a LLaMA-style block:

- **FFN intermediate channel** c: row c of W_{gate}, row c of W_{up}, and column c of W_{down} (three vectors of length d each).
- **Attention head** h: the slices of W_Q (and, for multi-head attention, W_K, W_V) belonging to the head and the matching columns of W_O. For grouped-query attention (GQA) [141], a key–value head is shared by several query heads, so removing a KV head removes its entire group.
- **Hidden (residual-stream) dimension** r: row/column r of *every* matrix that reads from or writes to the residual stream, plus embedding and normalization parameters. This is the most strongly coupled structure: it spans all layers.
- **Head dimension** e **within a head**: constrained by rotary position embeddings, which rotate pairs of dimensions, so removal must be in aligned pairs and consistent between Q and K.

Systematic discovery of such groups is automated by dependency-graph tools (DepGraph [82]), used by LLM-Pruner [73].

8.1.2 Parameter accounting

Let d be the hidden size, h_q the number of query heads, h_{kv} the number of key–value heads, d_h the head dimension, d_{ff} the FFN width, L the number of blocks and V the vocabulary size. For a gated-FFN decoder without biases,

$$P = L\Big(\underbrace{d\,h_q d_h + 2d\,h_{kv} d_h + h_q d_h\,d}_{\text{attention}} + \underbrace{3\,d\,d_{\text{ff}}}_{\text{FFN}}\Big) + \underbrace{V d\,(1 \text{ or } 2)}_{\text{embeddings (tied: 1)}} + O(Ld). \tag{8.1}$$

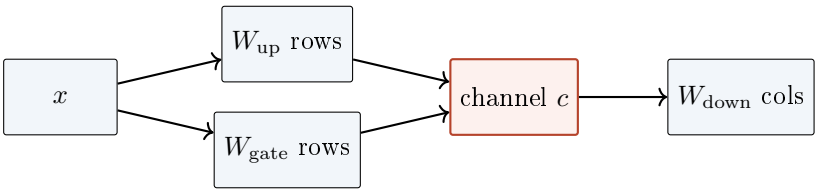


Figure 8.1: Coupled structure of one FFN channel in a gated (SwiGLU) block. Removing the channel eliminates three parameter vectors of length d.

Example 8.1 (Counting parameters; exact arithmetic from the formula)**.** *Llama-3.1-8B shape* ($d = 4096$, $h_q = 32$, $h_{kv} = 8$, $d_h = 128$, $d_{\text{ff}} = 14336$, $L = 32$, $V = 128256$, untied embeddings). Attention per block: $4096 \cdot 4096 + 2 \cdot 4096 \cdot 1024 + 4096 \cdot 4096 = 41.9$M; FFN per block: $3 \cdot 4096 \cdot 14336 = 176.2$M; per block 218.1M, so 32 blocks give 6.98B; embeddings and output head $2 \cdot 128256 \cdot 4096 = 1.05$B; total ≈ 8.03B, matching the model's nominal size [118]. The FFN holds 81% of the block parameters, so FFN channel pruning is the most productive width lever: removing 25% of channels ($d_{\text{ff}} \to 10752$) removes 1.41B parameters (17.5% of the total).

A 4B width-pruned shape. The width-pruned Llama-3.1-Minitron 4B has, as I recall from its model card[†] ($d = 3072$, $d_{\text{ff}} = 9216$, 32 heads, 8 KV heads, $L = 32$): attention per block 31.5M, FFN per block 84.9M, total blocks 3.73B plus embeddings 0.79B, about 4.5B, consistent with the "4B" name [112].

A 1B model. For the Llama 3.2 1B shape ($d = 2048$, $h_q = 32$, $h_{kv} = 8$, $d_h = 64$, $d_{\text{ff}} = 8192$, $L = 16$, tied embeddings) the blocks hold 0.97B and the tied embedding 0.26B (21% of the total) [117]. For small models the vocabulary matrix is a large share of the parameters, and pruning the vocabulary to the tokens that matter is a structured operation that deserves more attention than it receives.

8.2 LLM-Pruner

LLM-Pruner [73] is the first widely used framework for structured pruning of LLMs with limited data and compute. Its pipeline has three stages.

1. **Discovery.** Build the dependency graph; identify coupled groups (as above).
2. **Estimation.** Score each group with a first- or second-order Taylor-based importance. For a parameter w and loss on calibration data, the loss change from removal is $\Delta\mathcal{L} \approx -\frac{\partial \mathcal{L}}{\partial w} w + \frac{1}{2} w\, H_{ww}\, w + \cdots$. At a trained point the first term is not zero in expectation per-sample, and the authors use a Fisher approximation of the Hessian diagonal, $H_{ww} \approx \frac{1}{N} \sum_i (\partial \mathcal{L}_i / \partial w)^2$, to form element-wise importance, then aggregate over group members by summation, product, maximum, or last-member (the group's final parameter) depending on variant. Summation is the standard choice.
3. **Recovery.** Fine-tune the pruned model with LoRA [85] on a small instruction dataset for a couple of epochs (tens of thousands of samples).

The paper found that the first few and the last layers are particularly sensitive and left them

unpruned (as I recall[†]), and that LoRA recovery is necessary for acceptable quality at pruning ratios of 20% and above. The pipeline needs gradients through the full model for the calibration set (a modest number of samples) and is designed for single-GPU budgets.

Typical weaknesses of one-shot, gradient-scored structured pruning

- Importance estimates from few samples are noisy; aggregation choices have large effects.
- Perplexity under LoRA recovery can look acceptable while factual recall and reasoning are worse (Chapter 15).
- Coarse structure means little compensation: errors from removed channels cannot be offset by the surviving ones the way OBS compensation redistributes unstructured weights.

8.3 SliceGPT: pruning after a change of basis

SliceGPT [74] removes *rows and columns of transformed matrices*, i.e., reduces the embedding (hidden) dimension, using an exact invariance of the transformer architecture.

8.3.1 Computational invariance

Proposition 8.1 (Orthogonal invariance of RMSNorm)**.** *For $x \in \mathbb{R}^d$ and orthogonal $Q \in \mathbb{R}^{d\times d}$,* $\mathrm{RMSNorm}(Qx) = Q\,\mathrm{RMSNorm}(x)$ *when the learnable scale is a scalar (or has been folded into the adjacent matrix).*

Proof. $\mathrm{RMSNorm}(x) = x/\sqrt{\|x\|_2^2/d + \epsilon}$ (times a scale). Since $\|Qx\|_2 = \|x\|_2$, the denominator is unchanged and the numerator transforms linearly. □

LayerNorm is converted to RMSNorm by absorbing the mean subtraction (a linear projection $I - \frac{1}{d}\mathbf{1}\mathbf{1}^\top$) into the preceding weight matrices. Then inserting an orthogonal Q into the residual stream—multiply each matrix that writes to the stream by Q on the output side, and each matrix that reads from the stream by $Q^\top$ on the input side—leaves the network function unchanged. Different Q_ℓ may be used for different blocks provided a small matrix $Q_{\ell-1}^\top Q_\ell$ is inserted on the residual skip path.

8.3.2 Choosing Q by PCA and slicing

Choose Q_ℓ as the matrix of eigenvectors of the covariance $C_\ell = \sum_t x_t x_t^\top$ of the (normalized) residual-stream activations entering block ℓ on calibration data, ordered by decreasing eigenvalue. Slicing then deletes the last $d - k$ rows/columns of the transformed weights, equivalent to projecting the stream onto the top-k principal subspace.

Proposition 8.2 (Optimality of PCA slicing)**.** *Among all rank-k orthogonal projections P, the one maximizing retained activation energy $\sum_t \|Px_t\|_2^2$ projects onto the top-k eigenvectors of $C = \sum_t x_t x_t^\top$, and the discarded energy is $\sum_{i>k} \lambda_i(C)$.*

Proof. $\sum_t \|Px_t\|^2 = \mathrm{tr}(PC)$. By Ky Fan's maximum principle the maximum over rank-k orthogonal projections is $\sum_{i\le k} \lambda_i$. □

Proposition 8.2 explains the method's behavior: the loss from slicing is the discarded spectral tail of the residual-stream covariance. Since residual-stream activations in LLMs have rapidly decaying spectra with a few massive dimensions [147], substantial slicing is possible. The original paper reports that removing up to 25% of the parameters of LLaMA-2 70B, OPT 66B and Phi-2 retains about 99%, 99% and 90% of the dense model's zero-shot accuracy, respectively† [74], with reduced compute on GPUs because the result is dense.

Example 8.2 (Discarded energy from a decaying spectrum; illustrative arithmetic)**.** Suppose residual-stream covariance eigenvalues decay as $\lambda_i \propto i^{-1}$ over $d = 4096$ dimensions (a stylized power law, not measured). Total energy is $H_{4096} \approx \ln 4096 + 0.577 = 8.90$. Keeping $k = 3072$ (25% sliced) retains $H_{3072} \approx \ln 3072 + 0.577 = 8.61$, i.e., 96.8% of the energy and discards 3.2%; keeping $k = 2048$ (50% sliced) retains $H_{2048} = 8.21$, i.e., 92.2%. In this stylized model, halving the dimension discards 7.8% energy. Real LLM spectra, with outlier dimensions that carry large energy, are more concentrated in the head, but quality depends on low-energy directions that carry rare information, not only on total energy.

The costs of SliceGPT are the additional $k \times k$ (or $d \times d$ truncated) matrices on the skip path, which add compute and parameters of order Lk^2, and the fact that sliced dimensions reduce the embedding width without changing head count or FFN width (these can be combined with other pruning). Its calibration is from a single dataset, and its mid-network behavior depends on that choice.

8.4 FLAP: fluctuation-based adaptive structured pruning

FLAP [80] is a retraining-free structured method for pruning input channels of W_O (heads) and W_{down} (FFN channels). It rests on a statistic and a compensation.

Fluctuation score. For input channel j of a linear layer with weights $\mathbf{W}_{:,j}$, the score is $S_j = \mathrm{Var}(x_j)\,\|\mathbf{W}_{:,j}\|_2^2$: the variance (across calibration samples) of the channel's input feature times the squared norm of the weight column. Channels whose input barely fluctuates carry little information and can be replaced by a constant.

Bias compensation. If channel j is removed, replace its contribution $\mathbf{W}_{:,j}x_j$ by $\mathbf{W}_{:,j}\,\mathbb{E}[x_j]$, absorbed as a bias.

Proposition 8.3 (Why variance times norm)**.** *Let $\mathcal{R}$ be a set of removed input channels and use the bias $b = \mathbf{W}_{:,\mathcal{R}}\mathbb{E}[x_{\mathcal{R}}]$. Then the expected squared output error is $\mathbb{E}\|\mathbf{W}_{:,\mathcal{R}}(x_{\mathcal{R}} - \mathbb{E}x_{\mathcal{R}})\|_2^2 = \mathrm{tr}(\mathbf{W}_{:,\mathcal{R}}\Sigma_{\mathcal{R}}\mathbf{W}_{:,\mathcal{R}}^\top)$, with $\Sigma_{\mathcal{R}}$ the covariance of the removed channels. If channel covariances are diagonal, this equals $\sum_{j\in\mathcal{R}} \mathrm{Var}(x_j)\|\mathbf{W}_{:,j}\|_2^2 = \sum_{j\in\mathcal{R}} S_j$.*

Proof. The error vector is $\mathbf{W}_{:,\mathcal{R}}(x_{\mathcal{R}} - \mathbb{E}x_{\mathcal{R}})$. Its expected squared norm is $\mathrm{tr}(\mathbf{W}_{:,\mathcal{R}}\mathbb{E}[(x_{\mathcal{R}} - \mathbb{E}x_{\mathcal{R}})(x_{\mathcal{R}} - \mathbb{E}x_{\mathcal{R}})^\top]\mathbf{W}_{:,\mathcal{R}}^\top)$. Under diagonal $\Sigma_{\mathcal{R}}$ the trace splits per channel. □

Compare with Wanda's score $W_{ij}^2\|\mathbf{X}_j\|^2$ (second *moment*) from Chapter 6: FLAP replaces second moment by *variance*, which is the correct error once the mean is compensated by the bias; the bias compensation is cheap because it only adds a vector. FLAP also standardizes

scores across layers and modules (zero mean, unit variance within the module) so that a single global threshold produces a non-uniform, adaptive allocation of pruned structure across layers, answering the allocation question of Eq. (3.8) heuristically.

8.5 Gradient-free and learned structured pruning

Bonsai. Dery et al. [81] prune using only forward passes: they sample random sub-models (structured masks), evaluate them on a small set, and regress module importances from the sampled performance (a perturbative estimator), enabling pruning of models that do not fit in memory for backward passes. The method is particularly useful when gradients through the full model are not affordable.

CoFi, block pruning and Compresso. In the pre-LLM transformer literature, CoFi [76] learned masks at multiple granularities (layers, heads, FFN channels, hidden dimensions) with ℓ_0 gates (Section on hard-concrete gates in Chapter 3) plus layerwise distillation, reaching $10\times$ speedups with small accuracy loss on GLUE-type tasks. Block pruning [78] learned structured blocks with movement pruning scores, and Kwon et al. [79] proposed a fast *post-training* framework combining a Fisher-based mask search with mask rearrangement and layerwise reconstruction by least squares. Compresso [83] extended the learned-mask approach to LLaMA-7B with collaborative prompting. LoRAPrune [84] estimates importance from LoRA gradients to avoid full-gradient memory. ZipLM [57] adds inference-awareness: it prunes structures to hit a target *latency* using measured per-module timings.

8.6 Sheared-LLaMA: targeted structured pruning plus continued pre-training

Sheared-LLaMA [75] turns structured pruning into a producer of small *pretrained* models. It has two components.

8.6.1 Targeted structured pruning

The goal is a pruned model of a *prescribed* architecture (e.g., the shape of an existing 1.3B or 2.7B model), not merely a smaller one. The method learns hard-concrete masks [18] for layers (z^{layer}), attention heads (z^{head}), FFN intermediate dimensions (z^{int}) and the hidden dimension (z^{hid}), and enforces the target dimensions by a Lagrangian: for each component with expected retained size $\tilde{s}$ and target $s^\star$,

$$\mathcal{L}_{\text{prune}} = \lambda_1(\tilde{s} - s^\star) + \lambda_2(\tilde{s} - s^\star)^2, \tag{8.2}$$

with λ_1, λ_2 trained by gradient ascent, so the expected architecture converges to the target. After mask learning, the masks are discretized and the model continues with the final architecture. The structured masks are computed jointly with the language-modeling loss.

Algorithm 6 Sheared-LLaMA pipeline (after [75])

Require: source model θ_0, target architecture $\mathcal{A}^\star$, pruning data $\mathcal{D}_{\text{prune}}$, continued-pretraining data with domains $\{\mathcal{D}_i\}$, reference losses ℓ_i^{ref}

Initialize hard-concrete gate parameters and Lagrange multipliers
while expected architecture $\neq \mathcal{A}^\star$ (pruning stage) **do**
    sample batch from $\mathcal{D}_{\text{prune}}$; update weights and gates by gradient descent on $\mathcal{L}_{\text{LM}}+\mathcal{L}_{\text{prune}}$; update λ's by ascent
end while
discretize masks; instantiate pruned model θ_1 with architecture $\mathcal{A}^\star$
for $t = 1, \ldots, T$ (continued pre-training) **do**
    sample domains with probabilities w_t; update θ on LM loss; every m steps evaluate domain losses and update w by Eq. (8.3)
end for
return pruned and continued model

8.6.2 Dynamic batch loading

Pruned models recover unevenly across domains: some data domains (e.g., code or books) degrade more than others. Sheared-LLaMA continues pre-training with domain proportions adapted online. Let ℓ_i^{ref} be a reference loss for domain i (predicted by a scaling law fitted on larger models), and $\ell_i(\theta_t)$ the current loss. The weights are updated multiplicatively:

$$\Delta_t[i] = \max\big(\ell_i(\theta_t) - \ell_i^{\text{ref}}, 0\big), \qquad w_t[i] \propto w_{t-1}[i]\,\exp(\Delta_t[i]), \tag{8.3}$$

which up-weights domains where the model lags its reference and leaves already-good domains alone.

Results and cost. Starting from LLaMA-2-7B, Xia et al. produced 1.3B and 2.7B models using roughly 0.4B tokens for pruning and 50B tokens of continued pre-training, which they report as about 3% of the compute needed to train comparable models from scratch, and they report that the sheared models outperform comparably sized open models trained from scratch on much more data[†] [75]. The significance is not that structured pruning is lossless—it is not, and the 50B-token recovery phase is essential—but that a large pretrained model is a far better initialization than random weights.

8.7 Recent structured-pruning methods (2025–2026)

The post-training structured setting remains active. Representative preprints (checked against their abstracts in October 2026):

- **Putri** [87] updates remaining FFN weights to compensate for removed channels, prunes FFN layers sequentially (accounting for earlier updates), and prunes individual attention heads (including GQA) rather than whole attention layers; it claims robustness at extreme sparsity where earlier post-training methods fail[‡].
- **Iterative structured pruning with multi-domain calibration** [86] combines a hybrid multi-domain calibration set with an iterative channel-removal strategy, addressing the

Table 8.1: Qualitative comparison of width-pruning families.

Method	Structure removed	Recovery	Needs gradients?	Comment
LLM-Pruner [73]	channels, heads (grouped)	LoRA	yes (calibration)	early single-GPU recipe; layers at ends kept
SliceGPT [74]	hidden dim (after rotation)	optional	no	extra skip-path matrices; theoretically clean
FLAP [80]	channels of W_O, W_{down}	none (bias comp.)	no	fast; adaptive allocation
Bonsai [81]	modules	optional	no (forward only)	useful when backward is infeasible
Sheared-LLaMA [75]	layers, heads, FFN, hidden	50B-token CPT	yes	targeted architecture; high recovery cost
Minitron (Ch. 10)	hidden, FFN, heads, layers	distillation	forward-based importance	strongest production recipe

calibration-bias issue that we discuss in Chapter 15‡.

- **GPrune-LLM** [88] observes that neurons differ in sensitivity to the calibration distribution, groups neuron rankings into behavior-consistent sets, switches the scoring source when calibration signal is unreliable, and learns per-module sparsity under a global budget; it reports the largest gains at high sparsity‡.
- **LILA** [89] proposes a calibration-free FFN-neuron score from the Kolmogorov–Smirnov distance between singular-value distributions of the full and neuron-ablated weight matrices; on LLaMA-2-7B at 25% sparsity it reports gains over PruneNet and SliceGPT in zero-shot accuracy without fine-tuning‡.

These works share a theme: the choice and diversity of calibration data, not only the importance formula, drive post-training structured pruning quality. Since they are recent preprints, their claims have had limited independent replication.

8.8 Comparison of width-pruning methods

8.9 Design rules extracted from the literature

1. Prune FFN channels first: they hold the majority of block parameters (81% in the Llama-3.1-8B shape of Example 8.1), have the least coupling, and tolerate pruning better than the hidden dimension.
2. Treat hidden-dimension pruning as the most expensive structured operation in terms of accuracy; it requires consistent removal across all layers and strongly affects massive-activation dimensions [147].
3. Respect GQA: removing individual query heads within a group preserves the KV cache size, while removing KV groups reduces it but removes several query heads at once.
4. Use activation- or calibration-based scores computed with *forward passes* when memory limits prevent backward passes; combine with a diversified calibration set.

5. Budget for recovery: LoRA suffices for 10–20% removal; beyond 25–30% budget billions of tokens of distillation (Chapter 10).

Key lessons of Chapter 8

1. Structured pruning yields universally executable smaller dense models but loses the compensation freedom of unstructured pruning.
2. SliceGPT's rotation trick (Proposition 8.1) and PCA optimality (Proposition 8.2) give a principled hidden-dimension reduction; FLAP's variance-times-norm score with bias compensation is the exact error for mean-compensated removal under diagonal covariance (Proposition 8.3).
3. Without a recovery phase, quality degrades quickly beyond 20–30% removal; with distillation of tens of billions of tokens, pruned models are competitive with models trained from scratch at far lower cost.
4. Calibration-data diversity is a first-order design variable in 2025–2026 work.

Chapter 9

Depth Pruning, Layer Dropping and Layer Skipping

Removing entire transformer blocks is the coarsest and, in some ways, the most attractive form of structured pruning: the result is a shallower dense model whose latency falls roughly in proportion to the number of removed layers, with no change to kernels. The surprising empirical fact, documented from [92] to [94, 95], is that the *deeper* layers of pretrained LLMs are often highly redundant in the sense measured by classification-style benchmarks. The equally important and more recent finding is that this redundancy is much less benign for generative reasoning [101]. This chapter explains why, derives the standard layer-importance metric, surveys methods (static pruning, merging, and dynamic skipping or early exit), and states what is known about recovery.

9.1 The residual-stream view and why layers can be redundant

A pre-norm transformer maintains a residual stream $x_\ell \in \mathbb{R}^d$ and updates it as $x_{\ell+1} = x_\ell + f_\ell(x_\ell)$ with $f_\ell = \mathrm{attn}_\ell + \mathrm{mlp}_\ell$ (each preceded by normalization). Unrolling gives

$$x_L = x_0 + \sum_{\ell=0}^{L-1} f_\ell(x_\ell). \tag{9.1}$$

Removing block k replaces $x_{k+1} = x_k + f_k(x_k)$ by $x'_{k+1} = x_k$ and then runs later blocks on a slightly different input. If $\|f_k(x_k)\| \ll \|x_k\|$ and later blocks are not sensitive to the small shift, the output barely changes. Early work on dropping layers from BERT-family models [93] and training with structured layer dropout [92] established that dropping the top layers causes graceful degradation; for LLMs, studies show that blocks in the middle and later parts of the network make small angular updates [94, 95].

9.1.1 Block influence

ShortGPT [94] defines the *Block Influence* (BI) of block ℓ as one minus the expected cosine similarity between its input and output hidden states:

$$\mathrm{BI}_\ell = 1 - \mathbb{E}_x\Big[\frac{x_\ell^\top x_{\ell+1}}{\|x_\ell\|_2 \|x_{\ell+1}\|_2}\Big], \tag{9.2}$$

where the expectation is over tokens in a calibration set. Blocks with the smallest BI are removed.

Proposition 9.1 (BI measures relative orthogonal update size)**.** *Write $x_{\ell+1} = x_\ell + f$ and decompose $f = f_\parallel + f_\perp$ with $f_\parallel \parallel x_\ell$ and $f_\perp \perp x_\ell$. If $\|f\| \ll \|x_\ell\|$ then*

$$1 - \cos\angle(x_\ell, x_{\ell+1}) \;\approx\; \frac{1}{2}\frac{\|f_\perp\|_2^2}{\|x_\ell\|_2^2}.$$

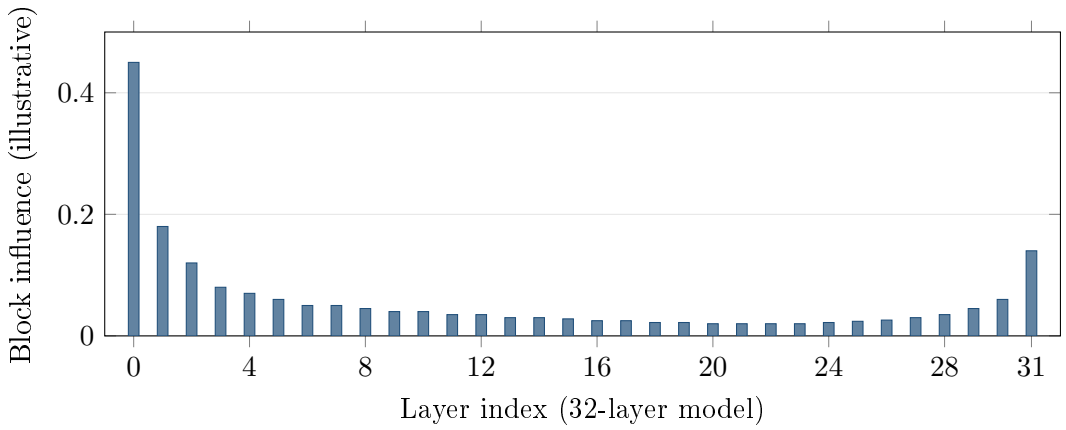


Figure 9.1: Schematic (synthetic, illustrative; not measured) BI profile reflecting the commonly reported shape: first layers and the last layer have large influence, and a broad plateau of low influence sits in the deeper middle layers.

Proof. Let $a = \|x_\ell\|$, $\alpha = \|f_\parallel\|/a$ (signed) and $\beta = \|f_\perp\|/a$. Then $\cos\theta = (1+\alpha)/\sqrt{(1+\alpha)^2+\beta^2}$. For small β, $\cos\theta \approx 1 - \frac{\beta^2}{2(1+\alpha)^2}$, and α is of the same small order as β or smaller, so the leading term is $\beta^2/2$. □

Example 9.1 (Numeric check)**.** Take $\|x_\ell\| = 10$ and an orthogonal update of norm 1. Then $\cos\theta = 10/\sqrt{101} = 0.99504$, so BI $= 0.00496$, matching $\frac{1}{2}(1/10)^2 = 0.005$. A block that rotates the residual by 25% of its norm ($\|f_\perp\|/\|x\| = 0.25$) has BI $= 1 - 1/\sqrt{1.0625} = 0.0299$.

BI therefore is, up to scaling, the *squared relative orthogonal update*. It ignores updates parallel to x_ℓ (pure rescaling) and ignores what later layers do with the update; two blocks with the same BI can differ widely in importance if one writes into a direction that a later head reads strongly. This is a heuristic metric, not a loss bound.

9.1.2 Angular distance and the Gromov et al. finding

Gromov et al. [95] compute, for a block of n consecutive layers starting at ℓ, the *angular distance* $d(x_\ell, x_{\ell+n}) = \frac{1}{\pi}\arccos\left(\frac{x_\ell^\top x_{\ell+n}}{\|x_\ell\|\|x_{\ell+n}\|}\right)$ averaged over data, find the start $\ell^\star(n)$ minimizing it (the block to remove), delete layers $\ell^\star, \ldots, \ell^\star + n - 1$, and "heal" with a small amount of parameter-efficient fine-tuning (QLoRA). Their principal findings, qualitatively:

- Question-answering benchmark accuracy (e.g., MMLU, BoolQ-style tasks) stays nearly flat as the fraction of removed layers increases up to a threshold (reported in the range of roughly 45–55% for the largest Llama-2 models and smaller thresholds for smaller models), and then collapses rapidly to near-random.
- The autoregressive validation loss, in contrast, increases smoothly and monotonically with the fraction removed, with no threshold.
- Healing reduces the loss increase but does not remove the QA collapse threshold.

They interpret this as evidence that either the deeper layers are not storing knowledge needed for QA while the shallow layers are, or that current pretraining does not use deeper layers

Algorithm 7 BI-based layer pruning with optional healing (ShortGPT-style [94])

Require: model with L blocks, calibration set $\mathcal{C}$, number of blocks to remove r

for $\ell = 0, \ldots, L-1$ **do**
    run model on $\mathcal{C}$; record x_ℓ and $x_{\ell+1}$ for all tokens
    $\mathrm{BI}_\ell \leftarrow 1 - \mathrm{mean}\big(\cos(x_\ell, x_{\ell+1})\big)$
end for
$\mathcal{R} \leftarrow$ indices of the r smallest BI_ℓ
remove blocks in $\mathcal{R}$; ▷ optionally keep the final block and first block
optional: heal with LoRA / distillation (see text)
return pruned model

efficiently. The contrast between flat QA accuracy and smoothly rising loss is itself a warning: a metric that looks insensitive does not imply that the model is unchanged.

9.2 Methods for static layer removal

ShortGPT (BI-based). Compute BI for all blocks, remove the lowest, no retraining (optionally light fine-tuning) [94]. Removal of non-consecutive layers is allowed. Its simplicity is its appeal.

SLEB. Song et al. [97] remove blocks *iteratively*, each time choosing the block whose removal least increases the perplexity-based or logit-based loss on a calibration set ("redundancy verification"), recomputing after each removal so that interactions among removals are accounted for. It costs $O(L)$ evaluations per removal, so $O(rL)$ for r removals.

Shortened LLaMA. Kim et al. [96] compare one-shot depth criteria (Taylor-based and perplexity-based) and study retraining regimes (LoRA versus continued pre-training); a central claim is that depth pruning can give better inference speedups than width pruning at small batch sizes for the same parameter reduction, with the advantage of recovery by continued pre-training when moderate amounts of data are available (as reported there). Siddiqui et al. [100] analyze which blocks matter by pruning attention and MLP sub-blocks separately and report that attention sub-layers are more removable than MLP sub-layers in later layers, in the models studied.

LaCo: layer collapse. Yang et al. [98] merge consecutive layers instead of deleting them: the parameter differences $\theta_{\ell+k} - \theta_\ell$ of subsequent layers are added to the first layer of a merged span, then the later layers are dropped, subject to a representation-similarity check. It is a depth analog of model merging.

LLM-Streamline. Chen et al. [99] replace a removed block of layers by a lightweight network (e.g., a single FFN layer) trained to map the input of the first pruned layer to the output of the last pruned layer, which partially preserves the function of the removed span for a small parameter cost.

9.3 What depth pruning does to generative reasoning

Depth pruning is easy to underestimate

Depth-pruned models can retain multiple-choice accuracy while degrading on generation. Shrestha et al. [101] report that layer pruning keeps classification performance strong with little or no fine-tuning, but that generative reasoning tasks (GSM8K, HumanEval+) recover far more poorly; pruning also erodes algorithmic abilities such as arithmetic and generating balanced parentheses. Under the constraint of a single 80 GB GPU, supervised fine-tuning on self-generated responses restores up to 90% of baseline classification performance but recovery on generative reasoning stays limited, even with full fine-tuning on self-generated GSM8K responses; a depth-pruned model trained on nearly 100B post-pruning tokens still shows deficits on simple arithmetic[‡].

Why would depth pruning hurt generation more than classification? The following account is a hypothesis consistent with the above evidence, not a theorem:

1. **Serial computation budget.** A forward pass of an L-layer transformer provides $O(L)$ sequential non-linear steps per token. Multi-token generation (e.g., arithmetic with carries, balanced parentheses) requires the model to implement multi-step algorithms across layers and across tokens, and depth bounds the number of serial operations per token. Classification from a final-token readout of a short prompt often needs fewer serial steps because the pattern is recognized in a shallow pass.

2. **Error accumulation across tokens.** A small per-token distribution shift compounds over hundreds of generated tokens (exposure bias), whereas a single-token multiple-choice readout is evaluated once.

3. **Calibration objective mismatch.** Layer scores such as BI are computed on calibration text, not on the model's own long reasoning traces; the importance of deep layers during long-chain reasoning is invisible to them. This is the same mismatch that Lucas et al. highlight for weight pruning of reasoning models, where reconstruction on on-policy traces improves results [121].

9.4 Healing and recovery

Self-data distillation [116] fine-tunes pruned models on text generated by the unpruned model, which keeps the training distribution close to the model's own and reduces catastrophic forgetting; it is cheap and effective for moderate pruning. Shrestha et al. found a similar self-generation recovery effective for classification tasks but limited for generative reasoning [101].

9.5 Dynamic depth: early exit and layer skipping

Static removal fixes depth for all inputs; *dynamic* methods let the depth depend on the token.

Table 9.1: Recovery strategies for depth-pruned models (qualitative, from the cited works).

Strategy	Typical cost	Remarks
No recovery	none	acceptable only for mild removal ($\lesssim$10–20%) and multiple-choice tasks
LoRA / QLoRA healing [95, 96]	10^7–10^8 tokens	reduces loss; does not remove QA threshold
Self-data distillation [116]	tens of thousands of samples	fine-tuning on the dense model's own outputs reduces distribution shift and forgetting
Continued pre-training [96]	10^9–10^{11} tokens	substantially better; cost approaches that of small-model training
Logit distillation (Minitron-style, Ch. 10)	10^9–10^{11} tokens	the strongest recipe in production

Early exit. DeeBERT [105] attaches classifiers to intermediate layers and exits when confidence is high. For autoregressive generation, CALM [102] trains/calibrates exit decisions per token with statistical guarantees on the output (a bound on the divergence from the full model's output, via a calibration procedure). Two problems arise in decoding: (i) later tokens' attention needs keys and values at the skipped layers, which were not computed for early-exited tokens (CALM propagates the exit-layer hidden state to deeper layers to fill the cache); and (ii) batching: tokens in a batch exit at different depths, which complicates kernel scheduling and can eliminate throughput gains.

LayerSkip. Elhoushi et al. [103] train with layer dropout (higher rates for later layers) plus an early-exit loss through the shared LM head, so that early layers produce usable token distributions. At inference, the early layers draft tokens that the remaining layers verify, reusing the shared KV cache (self-speculative decoding, related to Draft & Verify [106] and speculative decoding [107]). The paper reports speedups of up to about 2.16× on summarization, 1.82× on coding and 2.0× on a semantic-parsing task for the models and settings studied†. Because verification restores the full model's distribution, this approach is lossless in distribution (like speculative decoding) and avoids the quality loss of unverified early exit.

Mixture-of-Depths. Raposo et al. [104] route only the top-k tokens at each layer through the block (the rest skip via the residual connection), with a fixed capacity k per layer so that compute is predictable and tensor shapes are static. It trains from scratch and reports compute-matched gains; it is architecture-level rather than post-hoc pruning.

9.6 Latency arithmetic and when depth pruning pays

For decode at small batch, per-token latency is the sum over blocks of the time to read that block's weights (and KV cache) and run its kernels. Removing a fraction ϕ of identical blocks reduces both weight bytes and kernel launches by ϕ, so the ideal speedup is $1/(1-\phi)$ on the block-dependent part (embedding and output head are not reduced). For $L = 32$ blocks and $\phi = 25\%$ (8 blocks), the ideal block speedup is 1.33×; with unreduced head and embedding time, which for small models can be a sizable fraction, the end-to-end figure is lower. The

Table 9.2: Static versus dynamic depth reduction.

	Static layer removal	Early exit / LayerSkip	Mixture-of-Depths
Training needed	none to heavy recovery	training-aware (LayerSkip)	from scratch
Distribution preserved?	no (approximation)	yes with verification	no (new model)
Batching	trivial	complicated	static shapes by design
Typical benefit	latency $\propto$ remaining layers	up to $\sim 2\times$ decode†	compute-matched quality
Main risk	generative degradation	KV cache handling; training cost	requires pre-training

speedup is also *more reliable* than for unstructured sparsity because launch counts and memory traffic both shrink, and because it does not depend on specialized kernels. Width pruning to the same parameter count tends to produce smaller GEMMs that utilize hardware less efficiently, which the Minitron study found favored depth pruning for latency while width pruning gave better accuracy for the same size (Chapter 10).

Key lessons of Chapter 9

1. Many deep blocks of pretrained LLMs make small angular updates to the residual stream; BI (Eq. (9.2)) is approximately the squared relative orthogonal update (Proposition 9.1) and is a cheap pruning signal.
2. Classification-style accuracy and autoregressive loss behave differently under depth pruning: QA accuracy shows a collapse threshold while loss rises smoothly [95].
3. Generative reasoning, arithmetic and algorithmic abilities degrade more and recover poorly, even with large amounts of recovery data [101].
4. Dynamic depth with verification (LayerSkip) preserves the output distribution and can speed up decoding without permanent quality loss.
5. Use depth pruning where latency matters and tasks are retrieval- or classification-like; validate multi-step generation explicitly.

Chapter 10

Pruning plus Distillation: Compressing Large Models into Families

Structured pruning alone damages models; distillation alone requires a student architecture and a lot of training. Their combination—initialize a smaller student by pruning the teacher, then train the student to match the teacher's output distribution—has become the dominant way of producing small and medium models from large ones. This chapter derives the distillation objective used in this setting, describes the Minitron recipe and its successors (Nemotron Nano 2, Nemotron Elastic, Puzzle-derived models), summarizes what is publicly known about small-model releases derived by pruning, and works through the cost arithmetic.

10.1 Knowledge distillation for LM recovery

10.1.1 Objective

Given teacher $p_T(\cdot|x_{<t})$ and student $p_S^\theta(\cdot|x_{<t})$ over vocabulary $\mathcal{V}$, token-level logit distillation minimizes the forward KL divergence averaged over positions in sequences x drawn from a distillation corpus $\mathcal{D}$ [108]:

$$\mathcal{L}_{\mathrm{KD}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}} \frac{1}{|x|} \sum_t \mathrm{KL}\big(p_T(\cdot|x_{<t}) \,\|\, p_S^\theta(\cdot|x_{<t})\big) = \mathbb{E} \frac{1}{|x|} \sum_t \sum_{v\in\mathcal{V}} p_T(v) \log \frac{p_T(v)}{p_S(v)}. \tag{10.1}$$

With temperature τ, both distributions are computed from logits divided by τ. Minimizing (10.1) is equivalent to minimizing the cross-entropy between teacher and student distributions, since the teacher entropy is constant. Compared with next-token cross-entropy on the hard labels, each position provides a full distribution over $|\mathcal{V}| \approx 10^5$ tokens as a training signal, i.e., orders of magnitude more bits of supervision per token, which is the standard explanation for the sample efficiency of distillation.

Proposition 10.1 (Gradient of forward-KL distillation)**.** *For logits z of the student and $q = \mathrm{softmax}(z)$, the gradient of $\mathrm{KL}(p\|q)$ with respect to z is $q - p$.*

Proof. Write $\mathrm{KL}(p\|q) = \sum_v p_v \log p_v - \sum_v p_v \log q_v$ with $\log q_v = z_v - \log \sum_u e^{z_u}$. Then

$$\frac{\partial}{\partial z_k}\Big(- \sum_v p_v \log q_v \Big) = -p_k + q_k \sum_v p_v = q_k - p_k.$$

□

The gradient has the same form as ordinary cross-entropy with the one-hot label replaced by the teacher distribution: the student is pushed toward the teacher's *entire* distribution, including probabilities of near-miss tokens.

Variants. Reverse KL and generalized divergences emphasize mode-seeking behavior; on-policy distillation samples sequences from the student and uses teacher probabilities as feedback, reducing the train–inference mismatch [110]. Intermediate losses (matching hidden states or attention maps through projections) have been used for BERT-scale models [109]; for LLM compression pipelines the Minitron authors report that logit-only distillation was sufficient or best in their experiments† [111], which is convenient because hidden-state matching complicates architectures with different widths.

10.2 The Minitron recipe

Muralidharan et al. [111] compress Nemotron-4 15B into 8B and 4B models and distill the practical lessons; Sreenivas et al. [112] apply the recipe to Llama 3.1 8B (to 4B) and Mistral NeMo 12B (to 8B), comparing depth pruning against joint width pruning of hidden, attention and MLP dimensions‡.

10.2.1 Importance estimation from activations

A distinctive element is a purely *activation-based, forward-pass* importance estimate, computed on a small calibration set (on the order of a thousand sequences), which replaces gradient-based scores. As I understand the formulation†, the importance of

- an **FFN neuron** is the average magnitude of its activation over batch and sequence positions,
- an **attention head** is the average norm of the head's contribution (aggregated over positions),
- an **embedding (hidden) channel** is the average magnitude of the normalized-layer outputs on that channel, aggregated over all layers,
- a **layer** is measured by the change in LM loss (or by cosine-based BI, Chapter 9) when removed.

Because activations need no backward pass, importance estimation runs with modest memory on the teacher. The authors report that one-shot estimation was adequate (iterating prune–retrain cycles gave no benefit in their studies†).

10.2.2 Pipeline

10.2.3 Reported findings and lessons

The papers report the following (figures as recalled, hence marked †):

- For a family of sizes, train the largest model once, then derive the others by pruning and distillation; the paper reports using up to about 40× fewer training tokens for the 8B and 4B models than training them from scratch, a 1.8× compute saving for training a whole family (15B, 8B, 4B), and up to about 16% MMLU improvement over from-scratch training at equal size† [111].

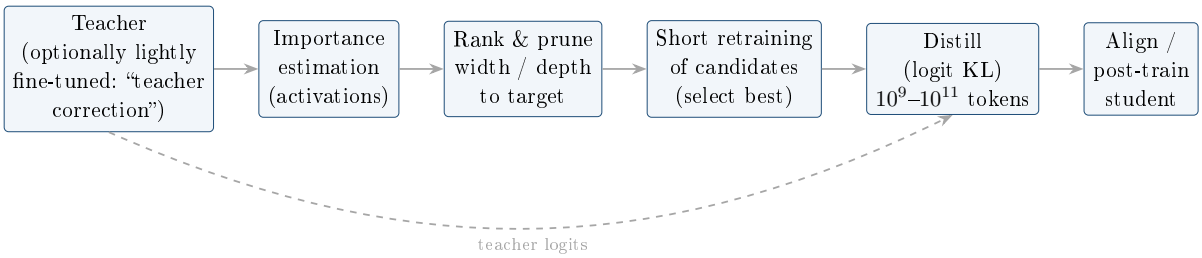


Figure 10.1: Pruning-plus-distillation pipeline in the style of Minitron [111, 112]. Teacher correction is used when the teacher's original pre-training data is unavailable.

Algorithm 8 Pruning and distillation pipeline (Minitron-style [111, 112])

Require: teacher T, calibration data, distillation corpus $\mathcal{D}$, target architectures $\{\mathcal{A}_k\}$

(optional) fine-tune T on $\mathcal{D}$ for a small number of tokens (teacher correction)
compute activation-based importance scores (neurons, heads, channels) and layer importance on calibration data
for each candidate architecture $\mathcal{A} \in \{\mathcal{A}_k\}$ **do**
    prune T to $\mathcal{A}$ by keeping highest-importance units (re-ordering weights accordingly)
    retrain briefly with $\mathcal{L}_{\text{KD}}$; record validation loss
end for
select the best candidate $\mathcal{A}^\star$
distill $\mathcal{A}^\star$ for N tokens with $\mathcal{L}_{\text{KD}}$ (teacher forward in the loop)
return student; optionally align (SFT, preference or RL stages)

- Width pruning gave better accuracy than depth pruning at the same parameter count in their comparison for Llama 3.1 8B→4B, while depth pruning gave larger inference speedups; both were released as base models [112].
- Distillation retraining beats conventional LM-loss retraining at the same token count.
- When the teacher's original pre-training data is not available (as for Llama and Mistral NeMo), slightly fine-tuning the teacher on the distillation dataset ("teacher correction") improves the result[‡] [112].

10.3 From Minitron to many-in-one: Nano 2, Elastic, Puzzle

Nemotron Nano 2 (9B). NVIDIA pre-trained a 12B hybrid Mamba–Transformer base model on 20 trillion tokens with an FP8 recipe and, after alignment, applied the Minitron strategy to compress and distill it to 9B parameters so that inference on contexts up to 128k tokens fits on a single A10G GPU with 22 GiB of memory in bf16[‡]. The report says the resulting model matches or exceeds similarly sized models such as Qwen3-8B on reasoning benchmarks and gives up to 6× higher throughput in reasoning settings (for example, 8k input and 16k output tokens)[‡] [113]. Two points matter for this survey: the architecture is a hybrid with mostly state-space layers (cf. Nemotron-H [114]), so the throughput gain is not due to pruning alone, and the pruning was applied to an *already aligned reasoning* model, with distillation designed to preserve long-chain-of-thought behavior.

Table 10.1: Public models whose derivation involves pruning of a larger model (as understood by the author). ‡ marks details seen in an abstract or page in October 2026; † marks details recalled from papers or announcements.

Model	Parent	Pruning axis	Notes
Minitron 8B, 4B (2024) [111]	Nemotron-4 15B	width and depth variants	distillation retraining; ∼40× fewer tokens†
Llama-3.1-Minitron 4B (2024) [112]	Llama 3.1 8B	width and depth variants‡	base weights released permissively‡
Mistral-NeMo-Minitron 8B [112]	Mistral NeMo 12B	width‡	teacher correction‡
Sheared-LLaMA 1.3B, 2.7B (2023) [75]	LLaMA-2-7B	layers, heads, FFN, hidden	50B-token continued pre-training†
Llama 3.2 1B, 3B (2024) [117]	Llama 3.1 8B	structured pruning	logits of 8B and 70B as distillation targets, per Meta†
Nemotron Nano 2 9B (2025) [113]	Nemotron-Nano-12B-v2	Minitron strategy‡	hybrid Mamba-Transformer; 128k context on A10G‡
Nemotron Elastic 9B, 6B (2025) [115]	Nemotron Nano V2 12B	nested elastic extraction‡	110B tokens; nested weights‡
Llama-3.1-Nemotron-51B and Llama-Nemotron family [198, 199]	Llama 3.1 70B and others	Puzzle NAS (block-level)	throughput-oriented†

Nemotron Elastic. Taghibakhshi et al. [115] train a single parent model that contains nested sub-models (hybrid Mamba–attention), extractable at deployment time without extra training. Applied to Nemotron Nano V2 12B, the procedure yields 9B and 6B models simultaneously using 110B training tokens, which the authors report as a more than 360× cost reduction versus training each family member from scratch and about 7× versus state-of-the-art compression techniques; the nested models share weights, so deployment memory does not grow with the number of family members‡. This builds on the idea of elastic architectures such as MatFormer [90] and Flextron [91], which train nested FFN or router-selected sub-networks end to end.

Puzzle and Llama-Nemotron. Puzzle [198] performs distillation-based neural architecture search over per-block alternatives (attention variants, FFN widths, or no-ops), with block-wise local distillation and an integer-programming search under a throughput/memory constraint. It produced a 51B model from Llama-3.1-70B-Instruct that was reported to run about 2.17× faster on a single H100 while retaining most of the parent's measured accuracy†. The Llama-Nemotron family [199] used Puzzle-based compression for its larger reasoning models, as I recall†.

10.4 Small models released through pruning

Many other widely used small models (for example Gemma 2 2B and 9B, and the smaller Qwen3 models) are *not* pruned from larger models; they are trained from scratch, with distillation from a larger teacher in the training objective [120, 200]. This is the closest alternative to pruning-plus-distillation and is compared in Chapter 13.

10.5 Cost arithmetic: when is pruning plus distillation cheaper?

Example 10.1 (Compute comparison under stated assumptions; illustrative arithmetic)**.** Use the standard approximation that training costs $6ND$ floating-point operations for N parameters and D tokens, and that a forward pass of the teacher costs $2N_T$ per token. Compare producing a 4B student:

- *From scratch* on $D = 15$T tokens: $6 \cdot 4{\times}10^9 \cdot 1.5{\times}10^{13} = 3.6 \times 10^{23}$ FLOPs.
- *Prune from an 8B teacher and distill* for $D' = 100$B tokens: per token, teacher forward $2 \cdot 8{\times}10^9 = 1.6 \times 10^{10}$ plus student forward-backward $6 \cdot 4{\times}10^9 = 2.4 \times 10^{10}$, total 4×10^{10}; times 10^{11} tokens gives 4×10^{21} FLOPs, about 1.1% of the from-scratch cost.

These ignore the cost of pretraining the teacher (the 8B teacher itself needs $\sim 7 \times 10^{23}$ FLOPs at the same 15T tokens) and the cost of importance estimation and candidate search (small). The saving is real for the *marginal* model derived from an existing teacher and compounds for a family of several sizes: the teacher cost is amortized, which is why the Minitron papers frame the benefit as a family-level compute reduction [111]. If only one small model is needed and no teacher exists, pruning offers no advantage over training the small model directly.

Equation-level intuition: the pruned student starts at a point already close to the teacher function, so most of the learning that from-scratch training spends on acquiring basic capabilities is inherited. The residual cost is repairing damage and re-aligning the reduced capacity, which is why 10^{10}–10^{11} tokens suffice, rather than 10^{13}.

10.6 Limits and failure modes

What distillation after pruning cannot fix

- **Capacity limits.** A student with fewer parameters has less capacity to store long-tail facts; distillation can transfer *behavior on the distillation distribution*, but memorized knowledge outside it may be lost for good. See Chapter 15.
- **Inherited flaws.** The student inherits the teacher's biases, errors, and safety behavior *as expressed on the distillation distribution*; behaviors absent from that distribution (refusals, rare languages, tool-use formats) can regress.
- **Data requirements.** Distillation needs a large, diverse corpus (billions of tokens) and a teacher forward pass for every training token; for 70B-class teachers this dominates cost.
- **Licensing and provenance.** Derived models inherit the license terms of the parent, and the use of a teacher's outputs may be restricted by terms of service.
- **Architecture lock-in.** The student keeps the teacher's design (tokenizer, head dimension, attention type), which may be suboptimal for small sizes (large vocabulary share, Example 8.1).

Key lessons of Chapter 10

1. Distillation after pruning is the practical recipe for turning structured pruning into competitive models; its cost is of order 10^{10}–10^{11} tokens, one to two orders of magnitude below from-scratch training.
2. Activation-based forward importance, short candidate retraining, logit-only KL, and teacher correction are the reported recipe ingredients.
3. The approach extends to hybrid Mamba–Transformer reasoning models and to nested elastic families (110B tokens for two sizes at $> 360\times$ claimed savings[‡]).
4. The benefit is largest for families derived from a teacher that already exists; it does not remove the teacher's pre-training cost.

Chapter 11

Mixture-of-Experts Pruning and Merging; Attention-Head Pruning

Mixture-of-Experts (MoE) models decouple *stored* parameters from *active* parameters: only k of E experts run per token, so quality scales with total parameters while per-token compute scales with active ones [123, 124]. The cost is memory: all experts must be resident (or offloaded). This chapter treats compression of the *expert* dimension—pruning, merging and dynamic skipping—and then attention-head pruning, whose redundancy and specialization have been studied since the BERT era.

11.1 MoE basics and why experts are a compression target

An MoE feed-forward layer with experts $E_1, \dots, E_E$ and a router producing logits $r(x) \in \mathbb{R}^E$ computes

$$y(x) = \sum_{j \in \mathcal{T}(x)} g_j(x)\, E_j(x), \qquad \mathcal{T}(x) = \mathrm{TopK}\big(r(x), k\big), \quad g = \mathrm{softmax}\left(r(x)_{\mathcal{T}(x)}\right), \tag{11.1}$$

(some models normalize gates over the selected set, others over all experts; others add always-on *shared* experts [126]). Modern open MoE models have many fine-grained experts: Mixtral 8x7B has 8 experts with top-2 routing and about 47B total/13B active parameters [125]; DeepSeek-V3 has 671B total/37B active parameters [127]; Kimi K2 has about 1T total/32B active [128]; Qwen3-30B-A3B has 128 experts per layer with 8 active, about 30B total/3B active [120].

Example 11.1 (Expert-union under batching; computed under a uniform-routing assumption)**.** If each of B tokens in a decode batch independently picks k of E experts uniformly at random, the expected number of distinct experts touched in a layer is $E\big(1 - (1 - k/E)^B\big)$. For $E = 128, k = 8$: $B = 1 \to 8$; $B = 8 \to 51.6$; $B = 16 \to 82.4$; $B = 32 \to 111.8$; $B = 64 \to 125.9$. In reality routing is skewed (some experts are hot), which lowers these numbers, but the lesson holds: at moderate batch sizes a decode step touches nearly all experts, so *memory traffic per step approaches the total parameter count, not the active parameter count.* MoE throughput at scale is therefore limited by total weight bytes, which makes expert pruning (which reduces total bytes) directly valuable, whereas it reduces nothing about the per-token FLOPs.

11.2 Expert importance and pruning

11.2.1 Frequency, gate and activation statistics

Calibration passes record, for each expert j in a layer, the set of tokens routed to it $\mathcal{X}_j$, gate values $g_j(x)$, and output norms $\|E_j(x)\|$. Simple scores:

- **Frequency**: $|\mathcal{X}_j|/N$ (SEER-MoE-style heavy-hitter counts [135]).

- **Mean gate**: $\mathbb{E}_{x\in\mathcal{X}_j}[g_j(x)]$.
- **Activation norm**: $\mathbb{E}_{x\in\mathcal{X}_j}\|E_j(x)\|_2$.
- **Router-weighted activation** (REAP [131]): the product of gate and activation norm averaged over routed tokens (derived below).
- **Weights × router × activation** (MoE-Pruner [137]): a Wanda-like unstructured score for the weights *inside* experts, weighted by router weights.

11.2.2 Derivation of a router-weighted expert score

Proposition 11.1 (First-order error of removing an expert)**.** *Consider one MoE layer and a token x routed to expert j. If expert j is removed and its gate mass is simply dropped (no re-routing and no renormalization), the output changes by exactly $\Delta y(x) = -g_j(x)E_j(x)$, so the squared error is $g_j(x)^2\|E_j(x)\|_2^2$. Averaged over the tokens routed to j, and over the token distribution (tokens not routed to j contribute zero), the expected squared layer error is*

$$\mathbb{E}_x\|\Delta y\|_2^2 = \Pr(j\in\mathcal{T})\ \mathbb{E}_{x\in\mathcal{X}_j}\left[g_j(x)^2\|E_j(x)\|_2^2\right].$$

Proof. From (11.1), the layer output is a sum over selected experts. Deleting term j for tokens that select it gives $\Delta y = -g_jE_j$, and tokens that do not select j are unchanged. Taking expectations over the mixture of "routed" and "not routed" tokens gives the formula. □

REAP's score is a close relative: it averages $g_j(x)\|E_j(x)\|_2$ over the routed tokens (the root-mean-square or mean variant of the same quantity), in order to minimize a reconstruction-error bound, and the authors emphasize that the score uses *both* router gate values and expert activation norms[‡] [131]. Dropping the frequency factor $\Pr(j\in\mathcal{T})$ as REAP does gives a score for *how much expert j matters when it is used*; keeping it gives the average over all tokens. Proposition 11.1 ignores re-routing: when expert j is deleted from the router, tokens that would have selected j instead select their next-best experts, which adds a second, typically smaller error term and can overload them.

Remark 11.1 (Gate renormalization and load). Mixtral-style gates normalize over the top-k set, so deleting an expert and re-running top-k changes gate values on the substituted expert. Pruned models also shift the load-balance statistics seen by the remaining experts; serving systems that rely on balanced expert parallelism may see imbalance after pruning.

11.2.3 Exhaustive reconstruction search and skipping

Lu et al. [133] observe that for models with few experts per layer (Mixtral's eight), one can enumerate subsets of experts to retain and choose the subset minimizing the Frobenius reconstruction error of the layer output on calibration data, layer by layer, with no gradient computation. They also propose *dynamic expert skipping* at inference: when the second expert's gate weight is small relative to the first, skip the second expert, thereby reducing active compute (a per-token, input-dependent decision). The combination of static pruning and dynamic skipping yields speedups on Mixtral-class models with modest accuracy loss in their study; as for all dynamic skipping, speedups depend on batching behavior (tokens in a batch skip different experts).

Table 11.1: Generative-suite scores reported by Jha et al. [132] (verified from the paper's summary, ‡). REAM scores are for the best calibration mix reported; REAP is the pruning baseline of [131]. The suite is the paper's own aggregate over six generative benchmarks; absolute values are not comparable across rows with different suites.

Model (experts before → after)	Uncompressed	REAP	REAM
Qwen3-30B-A3B-Instruct-2507 (128→96)	70.9	68.6	69.8
Qwen3-Coder-Next (512→384)	72.9	70.7	72.9
Qwen3-Next-80B-A3B-Instruct (512→384)	72.9	69.6	71.5
GLM-4.5-Air (128→96)	77.1	71.9	73.9

11.2.4 Task-specific pruning

Chen et al. [136] progressively prune non-professional experts during fine-tuning on a downstream task, down to a single expert per layer for some tasks, retaining the benefits of the MoE-pretrained model while serving a dense-like model. Such task-specific specialization is compatible with domain-specific deployments but does not retain generality.

11.3 Merging experts

Rather than discard expert j, merging combines groups of experts into one, in the hope of retaining some of the removed experts' function.

MC-SMoE. Li et al. [134] use routing statistics to pick *dominant* experts (by usage), assign each non-dominant expert to the most similar dominant one, align hidden units by permutation (weight matching), and merge by a usage-frequency-weighted average, followed by compression (low-rank and sparse decomposition of the merged weights).

REAM. Router-weighted Expert Activation Merging [132] creates a small number of large groups of experts plus many untouched experts, uses a combined similarity of experts, aligns neurons with a cost matrix that uses both activations and weights, and recomputes calibration statistics sequentially per layer. On Qwen3 and GLM-4.5 MoE models at 25% and 50% expert reduction, it reports that it often beats pruning baselines and in many cases approaches uncompressed models, and that a trade-off between multiple-choice and generative performance depends on the calibration mix‡. Table 11.1 lists specific results from that paper.

11.3.1 Why merging may be intrinsically lossy

The REAP authors argue that merging introduces an irreducible error because the merged expert loses the router's fine-grained control over mixing [131]. A simple calculation makes the point for two experts.

Proposition 11.2 (Merge error with a fixed mixing coefficient)**.** *Let two experts have outputs $a = E_a(x)$, $b = E_b(x)$ and gates g_a, g_b for a token x that selects both. Replace the pair by one merged expert with output $m = \alpha a + (1-\alpha)b$ for a constant $\alpha \in [0,1]$, carrying total gate mass*

$g_a + g_b$. *The output error is* $(g_a + g_b)\,(r - \alpha)\,(a - b)$ *with* $r = g_a/(g_a + g_b)$ *the router's mixing ratio, and the expected squared error is*

$$\mathbb{E}\big[(g_a + g_b)^2 (r - \alpha)^2 \|a - b\|_2^2\big].$$

It vanishes only if $a = b$ *(experts functionally identical on the data) or* $r \equiv \alpha$ *(the router's ratio is constant).*

Proof. The unmerged contribution is $g_a a + g_b b$; the merged one is $(g_a + g_b)(\alpha a + (1-\alpha)b)$. Their difference is $[g_a - (g_a + g_b)\alpha]a + [g_b - (g_a + g_b)(1 - \alpha)]b$; the coefficient of a is $(g_a + g_b)(r - \alpha)$ and that of b is its negative, giving the first claim. The second follows by squaring and taking expectations. □

So merging is lossless only when the router does not need to separate the two experts; any input dependence of r combined with functional difference $a - b$ produces error that grows with how much the router varies its mixing. Pruning, by contrast, discards one expert but *keeps the remaining experts intact with the router able to choose among them*; the error is confined to tokens that relied on the removed expert (Proposition 11.1). Which effect dominates is empirical: REAP finds pruning prevails on generative benchmarks at 50%[‡], while REAM reports that merging with better alignment and grouping can close or reverse the gap in some settings[‡]. These two preprints disagree in emphasis and both rely on specific calibration mixes; readers should treat the pruning-versus-merging question as open.

11.3.2 Reported scale of results

REAP evaluates on SMoE models from 20B to 1T parameters, reports consistent gains over merging and other pruning methods on generative benchmarks with the largest advantage at 50% compression, and states near-lossless code generation for Qwen3-Coder-480B and Kimi-K2 even after pruning 50% of experts[‡] [131]. Note that "near-lossless on code generation" refers to a specific task family with a calibration set matched to it; the paper's own framing (and REAM's calibration trade-off) implies that general-purpose behavior of a pruned model depends on calibration breadth.

MoE compression risks

- **Calibration-domain dependence.** Experts specialized in domains absent from calibration (languages, rare tool-use formats, safety-related behaviors) appear unimportant and are pruned. REAM reports an explicit trade-off between multiple-choice and generative performance as a function of calibration mix[‡].
- **Router mismatch.** The router was trained with all experts; after removal it may route to substitutes poorly. Router fine-tuning or logit adjustment is sometimes used.
- **Load imbalance** under expert parallelism after non-uniform pruning.
- **No FLOP reduction.** Top-k is unchanged; speedups come from memory capacity and bandwidth, not compute.

11.3.3 Systems consequences

Removing 50% of experts halves expert parameter memory. For a 1T-parameter MoE this can change the number of nodes needed; for a 30B model it can allow single-GPU deployment of a model that otherwise requires two. Batched decode benefits from lower weight traffic (Example 11.1). Alternative strategies that reduce memory without permanent removal include *expert offloading* (keep rarely used experts in CPU memory or flash, fetch on demand) and low-bit quantization of experts; these compose with pruning and share the same routing statistics.

11.4 Attention-head pruning and redundancy

11.4.1 Classical findings

Voita et al. [140] analyzed heads in a Transformer translation model, finding specialized roles (positional, syntactic, rare-token), and that a small set of important heads can be retained by pruning the rest via relaxed ℓ_0 gates (the hard-concrete machinery of Chapter 3) with small quality loss. Michel et al. [139] showed that many heads can be removed at test time without significant degradation, and that in some layers all but one head can be ablated, with the importance of head h estimated by a first-order sensitivity to a head-mask variable ξ_h that scales the head's output:

$$I_h = \mathbb{E}_x \Big| \operatorname{Att}_h(x)^\top \frac{\partial \mathcal{L}(x)}{\partial \operatorname{Att}_h(x)} \Big| = \mathbb{E}_x \Big| \frac{\partial \mathcal{L}(x)}{\partial \xi_h} \Big|, \tag{11.2}$$

evaluated at $\xi_h = 1$, which is exactly the first-order Taylor group importance of Chapter 5 applied to the head output. These are consistent with the finding that redundancy in heads is substantial for classification and translation tasks of the era.

11.4.2 Heads in LLMs: GQA, retrieval heads, streaming heads

Architectural redundancy. Multi-query attention [142] and grouped-query attention [141] reduce the number of *key–value* heads (to shrink the KV cache and decoding bandwidth) while keeping many query heads, an architecture-level admission that K/V heads are redundant; GQA checkpoints can be created by mean-pooling KV heads from a multi-head checkpoint and uptraining on a small fraction of pretraining compute [141]. Post-hoc head pruning in a GQA model must respect group structure (Chapter 8).

Retrieval heads. Wu et al. [143] identify a small subset of attention heads (a few percent) that implement copying of information from long contexts ("retrieval heads"); masking them destroys long-context retrieval and factuality while leaving short-context perplexity largely intact. DuoAttention [144] builds on the distinction: *retrieval heads* need the full KV cache, while *streaming heads* can use a short window plus attention sinks [145], which reduces KV memory and latency for long-context inference. This is a form of head-level *dynamic or heterogeneous* sparsity rather than pruning.

Table 11.2: Head-level redundancy results and caveats.

Source	Finding	Caveat
Voita et al. [140]	few specialized heads carry most function in NMT	encoder–decoder translation, pre-LLM
Michel et al. [139]	many heads removable at test time; I_h via gradient	BERT/NMT era; short sequences
GQA/MQA [141, 142]	KV heads can be shared with small loss	architecture design, needs uptraining
Wu et al. [143]	a few percent of heads are retrieval heads	short-context scores insensitive
DuoAttention [144]	streaming heads need only sink + window	requires head classification, custom kernels

Head pruning and long context

Head importance measured on short calibration sequences misses retrieval heads, whose role appears only when the answer lies far back in the context. Head pruning should be validated on long-context retrieval (needle-in-a-haystack-style and multi-document QA) before deployment, and calibration data for head scoring should include long sequences.

Key lessons of Chapter 11

1. In MoE models, memory scales with total experts; at moderate batch sizes decode touches almost all experts (Example 11.1), so expert pruning delivers memory and bandwidth savings but no per-token FLOP reduction.

2. Router-weighted expert activation (gate $\times$ output norm) is a principled score from a first-order reconstruction argument (Proposition 11.1); merging has an intrinsic mixing-ratio error (Proposition 11.2), explaining why pruning often wins on generation, while better alignment (REAM) narrows the gap.

3. Calibration breadth determines which experts survive; results at 25–50% expert reduction are strong for matched domains (code) and show benchmark-type trade-offs elsewhere.

4. Head pruning is attractive but must be checked for retrieval heads; GQA already exploits KV-head redundancy by design.

Chapter 12

Contextual and Activation Sparsity

Weight pruning removes parameters permanently. *Activation* (or *contextual*) sparsity leaves the weights intact and skips computation that, for the current token, would multiply by (near-)zero activations. Because the saving is realized by *not reading* weight columns, it targets exactly the bottleneck of small-batch decoding: weight memory traffic. This chapter develops the idea from first principles, derives the optimal skipping rule and its error, covers predictor-based methods (Deja Vu), training-free thresholding (CATS, TEAL, LaRoSA), induced sparsity by activation-function changes and continued training (ReLU-fication, ProSparse, ReLU^2, Q-Sparse), and shows why the benefit decays with batch size.

12.1 Definitions and sources of sparsity

Let a linear map be $y = \mathbf{W}x$ with $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and input $x \in \mathbb{R}^{d_{\text{in}}}$. If $x_j = 0$ for $j \in \mathcal{Z}$, the columns $\mathbf{W}_{:,\mathcal{Z}}$ need not be read: $y = \sum_{j \notin \mathcal{Z}} x_j \mathbf{W}_{:,j}$. Three sources of such zeros exist.

1. **Exact zeros from ReLU-family activations.** In models with ReLU FFNs (OPT, original T5, some others) the FFN intermediate vector has many exact zeros—often more than 90% in large OPT models according to Deja Vu and subsequent studies[†] [153, 154]. Squared-ReLU, used in some recent models, retains this property [63].
2. **Near-zeros from smooth gated activations.** In SwiGLU/GeLU models (LLaMA, Mistral, Qwen), intermediate activations are rarely exactly zero, but their magnitudes are heavily concentrated near zero; thresholding small magnitudes produces a sparse approximation (CATS, TEAL).
3. **Sparsity created by training.** Replace the activation function (ReLU-fication) or add activation-sparsity regularization during continued pre-training, producing intrinsic sparsity (ProSparse, ReLU^2, Q-Sparse).

Contextual versus static. The set of inactive columns depends on the input x (hence *contextual*); a different token activates different columns. A static method would remove the same columns for every input (equivalent to structured pruning). Contextual sparsity can be much higher than static sparsity at equal quality because each token only needs its own small subset of neurons, but exploiting it requires a per-token decision.

12.2 Thresholding theory: which activations may be skipped?

12.2.1 Optimal skipping under an error bound

Suppose we choose a set $\mathcal{Z}$ of $|\mathcal{Z}| = s\, d_{\text{in}}$ coordinates of x to zero. The output error is $e(\mathcal{Z}) = \sum_{j \in \mathcal{Z}} x_j \mathbf{W}_{:,j}$.

Proposition 12.1 (Activation-and-weight-norm rule)**.** *Let $e(\mathcal{Z}) = \sum_{j\in\mathcal{Z}} x_j \mathbf{W}_{:,j}$ be the output error from zeroing the coordinates in $\mathcal{Z}$.*

(a) $\|e(\mathcal{Z})\|_2 \le \sum_{j\in\mathcal{Z}} |x_j|\,\|\mathbf{W}_{:,j}\|_2$, with $\|e(\mathcal{Z})\|_2^2 = \sum_{j\in\mathcal{Z}} x_j^2 \|\mathbf{W}_{:,j}\|_2^2$ when the skipped columns are mutually orthogonal.

(b) Among all sets of a given size, the coordinates with smallest $|x_j|\,\|\mathbf{W}_{:,j}\|_2$ minimize the bound in (a).

(c) If all column norms are equal, this rule reduces to dropping the smallest $|x_j|$ (magnitude thresholding).

Proof. (a) The triangle inequality gives the bound; for orthogonal columns Pythagoras gives equality. (b) Minimizing $\sum_{j\in\mathcal{Z}} c_j$ over sets of fixed size with $c_j = |x_j|\|\mathbf{W}_{:,j}\|$ is achieved by taking the $|\mathcal{Z}|$ smallest c_j. (c) Constant $\|\mathbf{W}_{:,j}\|$ makes the ordering of c_j the ordering of $|x_j|$. □

Proposition 12.1 expresses a *duality* with Wanda (Chapter 6): Wanda scores weight W_{ij} by $|W_{ij}| \cdot \|\mathbf{X}_{j,:}\|_2$ (static weight pruning using the calibration norm of input channel j), while activation sparsity scores *input coordinates* by $|x_j|\cdot\|\mathbf{W}_{:,j}\|_2$ (dynamic skipping using the current activation and the weight column norm). Both multiply a weight-side quantity by an activation-side quantity, because the output contribution of a connection is their product. Within one matrix, column norms vary little compared to activation magnitudes (which span orders of magnitude), so thresholding $|x_j|$ alone is nearly optimal—the choice made by TEAL [160]. Some later methods, including *weight-informed* activation selection, incorporate the column norms explicitly.

12.2.2 Thresholds from the activation distribution

To realize a target sparsity s, choose a threshold t such that $\Pr(|x_j| < t) = s$. If activations are zero-mean with a unimodal shape, closed forms follow. For Gaussian $\mathcal{N}(0,\sigma^2)$, $t = \sigma\,\Phi^{-1}\big(\frac{1+s}{2}\big)$ (e.g., $t = 0.674\sigma$ for $s = 0.5$). For Laplace with scale b (density $\frac{1}{2b}e^{-|x|/b}$), $t = -b\ln(1-s)$ (e.g., $t = 0.693b$ for $s = 0.5$). TEAL observes (as I recall[†]) that hidden states entering attention and MLP blocks are approximately zero-mean and Gaussian-shaped while intermediate states are Laplacian-shaped, which justifies per-matrix thresholds estimated from calibration data via empirical quantiles and applied with a single fixed comparison at inference [160].

Example 12.1 (Energy dropped by thresholding; analytic)**.** By Example 3.1 and Table 3.1, zeroing the smallest-magnitude 50% of i.i.d. Gaussian coordinates discards 7.1% of the squared norm, and for Laplace coordinates 3.3%; at 30% sparsity the figures are 1.5% and 0.6%. In relative ℓ_2-norm, 50% sparsity is a perturbation of order $\sqrt{0.033} \approx 18\%$ (Laplace) to $\sqrt{0.071} \approx 27\%$ (Gaussian) of the input norm *if* the weight matrix treated all directions equally. That models tolerate this with 1–2% accuracy loss suggests strong robustness to noise in activations, consistent with the noise-stability view of Chapter 3; I flag this as an observation, not a proof.

12.3 Deja Vu: predicting contextual sparsity

Deja Vu [153] defines *contextual sparsity* as the existence, for each input, of small subsets of attention heads and MLP neurons that yield approximately the same layer output as the full layer, and shows empirically that such subsets are small (large fractions of heads and neurons

are unnecessary per input) in OPT models. To exploit it one must know the subset *before* computing the layer, which requires a cheap predictor.

Key observation: slowly changing embeddings. Because of residual connections, the input $x_{\ell+1} = x_\ell + f_\ell(x_\ell)$ to layer $\ell+1$ is close to x_ℓ (cosine similarity near one; cf. Proposition 9.1). Hence the active set for layer $\ell+1$ can be predicted from the input of layer ℓ, so the prediction for layer $\ell+1$ can run *asynchronously* in parallel with the computation of layer ℓ, hiding predictor latency.

Predictor. A small two-layer MLP (low-rank in the hidden dimension) per layer, trained offline on activations to predict, as multi-label classification, which heads/neurons are in the top set, with a recall-oriented target. At inference, the predictor outputs an index set; the layer computes only those heads and neurons.

Proposition 12.2 (Predictor economics)**.** *Let an FFN with hidden size d and width d_{ff} use gated structure with three matrices ($3\,d\,d_{\text{ff}}$ weights), and let a predictor with inner rank r have $r(d + d_{\text{ff}})$ weights. If the predictor identifies exactly the fraction s of channels that can be skipped (skipping all three matrices' corresponding rows/columns), the net weight-traffic saving is $3s\,d\,d_{\text{ff}} - r(d + d_{\text{ff}})$, positive iff $r < \dfrac{3s\,d\,d_{\text{ff}}}{d + d_{\text{ff}}}$.*

Proof. Dense FFN reads $3dd_{\text{ff}}$ weights; with skipping it reads $3(1-s)dd_{\text{ff}}$; the predictor reads $r(d + d_{\text{ff}})$. Net saving $= 3sdd_{\text{ff}} - r(d + d_{\text{ff}})$. □

Example 12.2 (Predictor budget; exact arithmetic)**.** For $d = 4096$, $d_{\text{ff}} = 14336$ and $s = 0.5$, the break-even rank is $r < 3 \cdot 0.5 \cdot 4096 \cdot 14336/18432 \approx 4778$. A predictor of rank 1024 reads $1024 \cdot 18432 = 18.9$M weights against a gross saving of $0.5 \cdot 3 \cdot 4096 \cdot 14336 = 88.1$M, leaving a net saving of 69.2M (78% of the gross). Prediction errors reduce the real saving: false negatives hurt quality, false positives waste compute, and the predictor itself is an additional model to train and version per checkpoint.

Results. The paper reports up to a 2× wall-clock latency reduction relative to an optimized FasterTransformer baseline for OPT-175B without degrading accuracy on the tasks tested, and compatibility with quantization[†] [153]. Limits: the evidence is strongest for ReLU models whose FFN activations are truly sparse; for SwiGLU models the predictor must target magnitude-based importance, and predictors have a nontrivial false-negative rate.

12.4 Training-free thresholding: CATS, TEAL, LaRoSA

CATS. Contextually-Aware Thresholding for Sparsity [159] sparsifies the *gate* activation of SwiGLU FFNs: $\text{CATS}_t(x) = \text{SiLU}(\mathbf{W}_{\text{gate}}x)$ if its magnitude is at least t, else 0, with t set from the empirical distribution to hit a target sparsity. Because the gate is computed first, zeros in the gate output allow skipping the corresponding rows of $\mathbf{W}_{\text{up}}$ and columns of $\mathbf{W}_{\text{down}}$ (the gate itself is computed densely). The paper reports that CATS models stay within roughly 1–2% of the base models even at 50% activation sparsity on Mistral-7B and Llama-2-7B without fine-tuning, and a custom GPU kernel yields about a 15% reduction in token-generation latency[‡]

Table 12.1: Training-free activation sparsity methods (numbers verified from abstracts, ‡, from different papers and setups; not directly comparable).

Method	What is sparsified	Reported sparsity/quality	Reported speed
CATS [159]	SwiGLU gate activation	within 1–2% at 50% (Mistral-7B, Llama-2-7B)	∼15% lower token latency
TEAL [160]	input of every matrix	40–50% model-wide, minimal loss (7B–70B)	up to 1.53× (40%), 1.8× (50%)
LaRoSA [161]	rotated inputs, Top-K	40%: PPL gap 0.17; 0.54% acc. gap (LLaMA2-7B)	1.30× (40%)

[159]. The limited speedup reflects that only the FFN is sparsified and the gate matrix is still dense: with gate computed densely, the FFN weight traffic at sparsity s is $(1 + 2(1 - s))/3$ of dense, which at $s = 0.5$ is 0.667 (an ideal 1.5× on the FFN alone, then diluted by attention and other costs).

TEAL. Training-free Activation Sparsity [160] applies magnitude thresholding to the *input of every matrix* in the model (hidden states entering $W_{Q,K,V}$, W_O, gate/up, and the intermediate state entering W_{down}), not only the FFN. It achieves 40–50% model-wide activation sparsity with minimal degradation on Llama-2, Llama-3 and Mistral models from 7B to 70B, with wall-clock decoding speedups of up to 1.53× at 40% sparsity and 1.8× at 50% using improved sparse kernels, and works alongside weight quantization‡. A per-matrix sparsity allocation is chosen by a greedy procedure that distributes the sparsity budget to minimize error (as I recall†).

LaRoSA. Layerwise Rotated Sparse Activation [161] applies layerwise orthogonal rotations to the inputs, then Top-K selection, so that rotated activations are more amenable to sparsification and sparsity is consistent across tokens (a fixed K rather than a threshold, which gives predictable speedups). Because rotations can be absorbed into weights by the orthogonal invariance of RMSNorm networks (Proposition 8.1), they add no inference cost in many placements. On LLaMA2-7B at 40% sparsity the authors report a perplexity gap of 0.17, a 1.30× wall-clock speedup, and a zero-shot accuracy gap of 0.54%, outperforming TEAL by 1.77% and CATS by 17.14% in their comparison‡ [161].

12.5 Inducing sparsity: ReLU-fication and related training

ReLU-fication. Mirzadeh et al. [154] ("ReLU strikes back") show that replacing the activation function of pretrained non-ReLU models (e.g., GELU or SiLU) with ReLU and fine-tuning recovers most of the quality while creating high activation sparsity, in two stages: (1) replace the FFN activation; (2) additionally insert ReLUs after normalization layers so the inputs to attention projections and up-projections are sparse. They report FLOP reductions of up to roughly threefold in some models with small quality change†. The mechanism is not free: it needs continued training (fine-tuning on a modest number of tokens in their study), and quality

recovery is not guaranteed at larger scale or for models whose pretraining depended on gating dynamics.

ReLU^2. Zhang et al. [155] compare activation functions for *pretraining* sparse-activation LLMs on 1B-scale models and find that squared ReLU gives a favorable trade-off between sparsity and performance (and ReLU alone more sparsity but slightly lower quality); squared ReLU appears in Nemotron-family models, and Haziza et al. exploit its sparsity with 2:4 activation kernels [63].

ProSparse. Song et al. [156] convert Llama-2-family models to ReLU-activated models with *progressive* sparsity regularization (an ℓ_1-type penalty on intermediate activations whose strength increases along a schedule) followed by activation-threshold shifting (a shifted ReLU that zeroes small positive activations), reaching very high intermediate activation sparsity (reported around 89% for the 7B and 13B models†) with performance comparable to the original, and demonstrating acceleration with sparse-aware inference.

Q-Sparse and sparse-by-design training. Wang et al. [162] train models with top-K sparsification of activations (straight-through gradient) in all linear layers, report scaling-law-like behavior of sparsely-activated models, and show compatibility with low-bit models. Luo et al. [163] study how activation sparsity depends on activation function, training data amount and architecture ratios ("sparsing law"); as I recall†, ReLU-type models become sparser with more training data while SiLU-type models do not.

Costs of induced sparsity

Changing the activation function and continuing training alters the model; evaluation must cover the full capability suite, since fine-tuning data can shift behavior. Regularizing activations toward sparsity can hurt performance on tasks that depend on dense features (e.g., long-tail knowledge). The cost is a training run, whereas TEAL-style thresholding has none; the benefit is higher achievable sparsity (80–90%) rather than 40–50%.

12.6 Systems: kernels, batching and offloading

12.6.1 Sparse GEMV

For $y = \mathbf{W}x$ with sparse x, a kernel reads only the columns $\mathbf{W}_{:,j}$ with $x_j \neq 0$. Achieving bandwidth efficiency requires column-contiguous storage (so each active column is a contiguous read of length d_{out}); the index set is dynamic, so gather overhead and load-balancing across thread blocks matter. TEAL's kernel work is what turns 50% activation sparsity into 1.8× rather than 1.0×‡ [160]; CATS's custom kernel gives $\sim$ 15% because of the dense gate matrix [159].

12.6.2 The batching problem

Proposition 12.3 (Batch union of active columns)**.** *Suppose each of B tokens in a batch independently zeroes each input coordinate with probability s (sparsity). A coordinate's weight*

Table 12.2: Fraction of weight columns that must be read at batch size B under independent per-token sparsity s (computed from Proposition 12.3). Real activations are correlated across tokens (hot channels), which lowers the numbers, but the trend is robust.

Sparsity s	$B = 1$	2	4	8	16	32
0.3	0.700	0.910	0.992	1.000	1.000	1.000
0.4	0.600	0.840	0.974	0.999	1.000	1.000
0.5	0.500	0.750	0.938	0.996	1.000	1.000
0.7	0.300	0.510	0.760	0.942	0.997	1.000

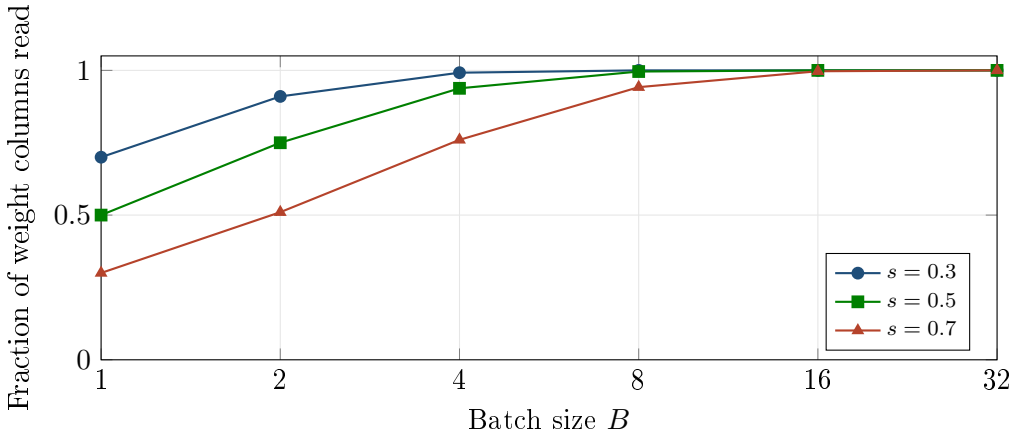


Figure 12.1: Computed from Proposition 12.3 (independence assumption). Contextual activation sparsity helps most at batch size one to four; at serving batch sizes it largely vanishes unless activations are strongly correlated across tokens or the system is organized around hot/-cold neurons.

column must be read if any *token keeps it, which occurs with probability* $1 - s^B$*. The expected fraction of columns read is* $1 - s^B$*.*

The consequence is the same as for MoE (Example 11.1): the benefit of input-dependent sparsity is concentrated at low batch sizes—on-device inference, latency-critical single-stream serving, speculative-decoding verification at small widths—and is weak for high-throughput batched serving, where dense, quantized kernels dominate. Correlation helps: PowerInfer [158] exploits a power-law distribution of neuron activation frequencies by keeping frequently ("hot") activated neurons on the GPU and rarely activated ("cold") neurons on the CPU, with an adaptive predictor and neuron-aware operators; “LLM in a flash” [157] uses activation sparsity to load only needed rows from flash storage, with windowing and row–column bundling, so that models larger than DRAM can be run (the paper reports large speedups over naive loading, of order 4–5× on CPU and 20–25× on GPU†).

Table 12.3: Activation sparsity versus weight-side compression for small-batch decoding.

Aspect	Contextual activation sparsity	Weight pruning / quantization
Model modified?	no (training-free variants)	yes
Saves memory footprint?	no (weights remain)	yes
Saves per-token bandwidth?	yes, at small batch	yes, any batch
Composes with quantization?	yes (TEAL)	yes
Batched serving benefit	small (Table above)	unchanged
Quality risk	token-dependent noise; kernel-specific	static; calibration-dependent

12.7 Positioning relative to other compression families

Key lessons of Chapter 12

1. The optimal activation-skipping rule scores $|x_j| \, \|\mathbf{W}_{:,j}\|$ (Proposition 12.1); magnitude thresholding is nearly optimal because column norms vary little. It is the dynamic dual of Wanda.

2. Training-free thresholding (TEAL, CATS, LaRoSA) reaches 40–50% activation sparsity at small quality cost with $\sim$ 1.3–1.8$\times$ kernel-level decoding speedups; induced sparsity (ReLU-fication, ProSparse, ReLU^2) reaches 80–90% but needs training.

3. The benefit decays with batch size because active sets from different tokens union quickly (Proposition 12.3); this is a latency/on-device technique rather than a throughput technique.

4. Predictor-based approaches (Deja Vu) add a model and a recall/precision trade-off; thresholding approaches avoid predictors.

Chapter 13

Sparse Training, Scaling Laws, and Sparsity with Quantization

Chapters 5–12 treated pruning as post-training compression. This chapter asks the economic questions: can we *train* sparse models directly; how do sparsity and model/data size trade off in scaling laws; when is pruning a large model better than training a small one; how does sparse fine-tuning fit; and how does sparsity combine with quantization, where rate–distortion theory gives a surprisingly sharp (and sobering) answer.

13.1 Dynamic sparse training

13.1.1 The SET–RigL lineage

Dynamic sparse training (DST) maintains a fixed parameter budget (density ρ) throughout training and periodically changes *which* weights are active. The lineage:

- **Deep Rewiring** [166] reallocates connections stochastically under a Bayesian-flavored sampling scheme.
- **SET** (Sparse Evolutionary Training) [165] starts from a random sparse topology (Erdős–Rényi) and at intervals removes the smallest-magnitude weights and adds the same number of *random* connections.
- **Dynamic sparse reparameterization** [167] adds adaptive global thresholds and cross-layer reallocation; **Sparse Networks from Scratch** [168] regrows by momentum.
- **RigL** [169] regrows connections by the largest *gradient magnitude*: the topology is updated using a dense gradient computed only at update steps, so training cost stays proportional to the sparse model between updates.

The drop fraction is annealed, typically $f_t = \frac{f_0}{2}\big(1 + \cos(\pi t/T_{\text{end}})\big)$, so the topology stabilizes late in training.

Proposition 13.1 (Growth by gradient magnitude is a steepest-descent choice)**.** *Among inactive weights (currently zero), activating weight i with a small step η in the direction of descent changes the loss to first order by $-\eta|\nabla_{\theta_i}\mathcal{L}|$. Hence activating the k weights with the largest gradient magnitudes maximizes the first-order loss reduction among all choices of k weights to activate.*

Proof. For a newly activated weight initialized at zero, the loss change under a step $-\eta\,\mathrm{sign}(g_i)$ of size η is $-\eta|g_i| + O(\eta^2)$. Summing over k chosen weights is maximal for the k largest $|g_i|$. □

The Liu et al. "in-time over-parameterization" view [170] explains DST's success: over the course of training, the *union* of weights that have been active at some time covers a much larger fraction

Algorithm 9 RigL topology update [169]

Require: weights θ, mask $\mathbf{M}$ (density ρ per layer), update interval ΔT, drop fraction schedule f_t

for training step $t = 1, 2, \ldots$ **do**
  sparse forward/backward; update active weights with the optimizer
  if $t \bmod \Delta T = 0$ and $t < T_{\text{end}}$ **then**
    $k \leftarrow f_t \cdot \text{nnz}(\mathbf{M})$
    drop: deactivate the k active weights with smallest $|\theta_i|$
    compute dense gradient $\nabla_\theta \mathcal{L}$ on the current batch (for inactive weights)
    grow: activate the k inactive weights with largest $|\nabla_{\theta_i} \mathcal{L}|$; initialize them to zero
  end if
end for

of the dense network than the instantaneous density, so a sparse network explores a dense-like space of parameters over time while paying sparse costs at each step.

13.1.2 Sparse training at LLM scale

For LLMs the state of the art is more recent and less settled. Two 2026 papers address stability and data regimes:

- **SMET** (Sparse Memory-Efficient Training, Xiao et al. [173]) identifies a cold-start problem: newly regrown parameters under Adam receive overly large updates (because their second-moment estimates are initialized near zero), producing loss spikes after topology updates. It proposes optimizer warm-up and density-aware learning-rate scaling, and stores gradients and optimizer states only for active parameters to save memory[‡].
- **Sparse models under data scarcity** (Wu et al. [174]) studies sparse LMs when unique data must be repeated, up to 93.75% sparsity and models up to 1.92B parameters (held-out checks to 7.68B dense-equivalent), proposing a scaling law in active parameters, unique tokens, repetition and sparsity; findings include delayed data saturation under sparsity and that, at fixed data, the *loss-optimal* sparsity is moderate (about 50%) while the *compute-optimal* sparsity is higher and grows with data scale[‡].

Training with unstructured sparsity rarely saves wall-clock time

Most DST research *simulates* sparsity with dense kernels and masks, so reported FLOP savings are theoretical. On GPUs, unstructured sparse backward passes are not faster than dense ones at 50–90% sparsity for the matrix shapes of LLMs (Chapter 14). Memory savings are real only if parameters, gradients and optimizer states are stored compressed (as in SMET's active-parameter storage). Structured variants (N:M with transposable masks [61], 2:4 activation sparsity [63]) can give genuine, modest training speedups (the latter up to 1.3× on FFN[‡]).

13.2 Scaling laws with sparsity

13.2.1 Dense baseline

The Chinchilla-style parametric law [175, 176] models loss as

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \tag{13.1}$$

with N parameters and D training tokens; compute-optimal training takes roughly 20 tokens per parameter in the original fits (approximate; fits vary by setup). Frontier open models are routinely trained far beyond this ratio—for example, Llama 3 8B was trained on about 15T tokens, roughly 1900 tokens per parameter or about 94× the compute-optimal ratio [118]—because inference cost, not training cost, dominates the lifetime budget.

13.2.2 Sparse extensions

Frantar et al. [172] fit scaling laws for sparsely connected foundation models (vision transformers and encoder–decoder language models, not decoder LLMs) with a multiplicative effect of sparsity on the effective parameter count, finding that the optimal sparsity grows with the amount of training data. Jin et al. [171] study sparse *pre-training*, in which pruning occurs during training, across 80 pruning schedules, sparsity levels and durations: they find that starting to prune at 25% of the compute budget and finishing at 75% gives near-optimal final loss, and propose replacing N in (13.1) by the *average parameter count over pre-training* $\bar{N}$, reporting that this modified law fits both dense and sparse runs[‡], and that sparse pre-training can reach the same final quality as dense pre-training at equal compute while producing smaller models.

$$L \approx E + \frac{A}{\bar{N}^\alpha} + \frac{B}{D^\beta}, \qquad \bar{N} = \frac{1}{T}\int_0^T N(t)\,dt. \tag{13.2}$$

Example 13.1 (Average parameter count for a gradual-pruning schedule; computed)**.** Take the cubic schedule of Zhu and Gupta [26]: density falls from N_0 to N_f over the window $[t_0, t_1]$ as $N(t) = N_f + (N_0 - N_f)(1-u)^3$, $u = (t - t_0)/(t_1 - t_0)$. The window average of $(1-u)^3$ is $1/4$. With dense training for the first 25% of steps, pruning over the next 50%, and the final 25% at N_f, with $N_f = 0.25N_0$ (75% sparsity), the average is $\bar{N} = 0.25N_0 + 0.5\,(N_f + 0.25(N_0 - N_f)) + 0.25N_f = 0.25 + 0.5 \cdot 0.4375 + 0.25 \cdot 0.25 = 0.531N_0$. Under (13.2) the run behaves like a dense run of 53% of the starting size, while the final model has 25%. This is the sense in which sparse pre-training is "cheaper than dense at the final model quality" and also "larger than the final model".

13.3 Prune a large model or train a small one?

Three routes produce a model with N_f inference parameters:

1. **(A) Train dense at N_f.** Cost $\approx 6N_fD$.
2. **(B) Prune from a teacher** of size $N_0 \gg N_f$ and recover with D_r tokens of distillation: cost $\approx 6N_0D_0 + (6N_f + 2N_0)D_r$, where the first term is the (possibly sunk) teacher training.

Table 13.1: Pruning versus training a smaller model: when each route wins (author's synthesis of the cited evidence; not a measured comparison).

Situation	Favors	Reason
Teacher exists; need one or several smaller sizes	(B) pruning + distillation	marginal cost ∼1–10% of from-scratch (Example 10.1); family amortization
No teacher; single target size; budget ≥ trillions of tokens	(A) train small, heavily over-trained	no teacher cost; best quality per inference parameter at very large D
Target size within ∼25–30% of teacher	(B), structured + light recovery	damage small; recovery cheap
Target ≪ 25% of teacher (e.g., 70B → 1B)	(A) or distillation into a smaller architecture	pruned initialization may be a poor fit for drastic shape changes (open question)
Data-limited regime	(C) sparse training with moderate sparsity	sparsity delays data saturation[‡] [174]
Need hardware-executable sparsity (2:4)	(B) with learned masks or sparse CPT	one-shot 2:4 is lossy; training closes the gap

3. **(C) Sparse pre-train** (pruning during training): cost between the two, as in Example 13.1.

The evidence for (B) over (A) in the regime where it has been tested (derived models of 40–60% of the teacher's size) is strong: Minitron and Sheared-LLaMA report large token savings and better accuracy than from-scratch baselines at the same size (Chapter 10)[†]. The evidence for the extreme-shrink regime is thinner, and for (A) with very long training the gap closes: a well-trained small dense model benefits from 10^{13} tokens that no distillation run can supply. A sober reading is that pruning plus distillation is a *compute-efficient* way to produce good models, not a way to beat the best-trained dense model at the same size given unlimited budget.

13.4 Sparse fine-tuning and recovery

After post-training pruning, quality can be recovered by fine-tuning that respects sparsity.

- **Mask-frozen full fine-tuning**: gradients are masked so pruned weights stay zero; memory is that of full fine-tuning unless optimizer states are stored for active weights only.
- **LoRA and relatives** [85]: parameter-efficient, but adapters generally cannot be merged into sparse weights without destroying the pattern; LoRAPrune-type methods co-design pruning and adaptation [84].
- **Distillation-based sparse fine-tuning**: Kurtić et al. [202] study sparse fine-tuning for inference acceleration with a distillation loss on intermediate representations; sparse continued pre-training has been used to release 2:4 sparse variants of open models [203].
- **Self-data distillation** [116]: fine-tune on the dense model's own outputs.
- **On-policy reconstruction**: for reasoning models, reconstruct activations on the model's own reasoning traces during pruning [121].

13.5 Joint sparsity and quantization

13.5.1 Shared machinery

SparseGPT and GPTQ share the same Hessian-based compensation (Proposition 5.1); applying pruning and quantization in the same sweep ("prune then quantize within the column loop") is natural and was demonstrated by the SparseGPT authors for OPT-175B, who report that 50% sparsity combined with 4-bit weights compares favorably with 3-bit dense at similar storage† [42]. SpQR [179] stores a small fraction of outlier weights in higher precision (sparse outliers) alongside low-bit dense weights, a different joint use of sparsity. SLiM [55] combines one-shot quantization, sparsity and a low-rank correction term.

13.5.2 Storage arithmetic: effective bits per original weight

With value precision b bits and a representation of density ρ, the effective bits per *original* weight (counting index overhead) are:

- 2:4: $(2b + 4)/4 = b/2 + 1$ (Table 7.1);
- unstructured with a 1-bit/weight bitmap: $\rho\, b + 1$;
- dense: b.

13.5.3 Rate–distortion view: is sparsity ever the right way to spend bits?

For a source of independent Gaussian components with variances σ_i^2, Shannon's rate–distortion function under mean-squared error is given by reverse water-filling [201].

Theorem 13.1 (Reverse water-filling). *For independent $X_i \sim \mathcal{N}(0, \sigma_i^2)$ and total rate R bits, the minimum MSE distortion is $D = \sum_i \min(\theta, \sigma_i^2)$ where the water level θ satisfies $R = \sum_i \max\left(0, \frac{1}{2}\log_2(\sigma_i^2/\theta)\right)$. Components with $\sigma_i^2 \le \theta$ receive zero bits ("are pruned") and contribute distortion σ_i^2; all others contribute exactly θ.*

Corollary: for *i.i.d.* Gaussian weights ($\sigma_i^2 \equiv \sigma^2$), every component should receive the same rate, so $D(R) = \sigma^2 2^{-2R}$ per weight and *no* component should be pruned (set to zero) unless the rate is below the point where $\theta = \sigma^2$, i.e., rates near zero. Pruning is rate–distortion-optimal only when component variances are heterogeneous, so low-variance components lie below the water level.

In this toy model, at three effective bits per weight a dense 3-bit quantizer has a weight MSE of 0.0345, whereas unstructured-50% plus 4-bit survivors has about 0.08 and 2:4 plus 4-bit about 0.14: sparsity wastes bits when weights look i.i.d. Gaussian, and the discarded energy of 2:4 (13%) dwarfs the quantization noise at 4 bits ($< 1\%$). This is a *weight-space* calculation, and real LLM compression is evaluated in function space, where (i) activation outlier structure makes some weights far more important than others (heterogeneity, which water-filling says is exactly when pruning helps), and (ii) OBS-style compensation corrects pruning error. The practical implications are nevertheless clear and consistent with experience: **on GPUs, 4-bit (or lower) dense quantization is usually the better first use of a bit budget; sparsity**

Table 13.2: Weight-space distortion, in units of the weight variance, at equal storage in an i.i.d. Gaussian toy model (computed; ignores activations, compensation, and real weight structure). Scalar Lloyd–Max quantizer MSE values for the Gaussian are the standard tabulated ones (†): 4-bit 0.0095, 3-bit 0.0345, 2-bit 0.1175. Pruning distortions: unstructured 50%, 0.0713 (analytic); 2:4, 0.1322 (Table 7.2); surviving weights add quantization noise on the retained fraction. Theoretical limit: 2^{-2R}.

Scheme	Bits per original weight	Relative MSE
Dense 4-bit	4.0	0.0095
Dense 3-bit	3.0	0.0345
Dense 2-bit	2.0	0.1175
Unstructured 50% + 4-bit survivors, bitmap index	3.0	≈ 0.080
2:4 + 4-bit survivors	3.0	≈ 0.140
2:4 + 8-bit survivors	5.0	≈ 0.132
Shannon limit at 3 bits (Gaussian)	3.0	0.0156

pays off where it enables execution savings that quantization cannot (compute-bound math via sparse tensor cores; structured removal; MoE expert memory; contextual skipping at small batch), or where weight heterogeneity is extreme.

Key lessons of Chapter 13

1. Dynamic sparse training (SET→RigL) works by exploring many weights over time (in-time over-parameterization); LLM-scale DST needs optimizer fixes for regrown weights (SMET‡), and unstructured sparse training seldom saves wall-clock time on GPUs.

2. Sparse pre-training can match dense pre-training at equal compute with an average-parameter-count scaling law (pruning from 25% to 75% of training‡); under data scarcity moderate sparsity helps‡.

3. Pruning plus distillation is compute-efficient when a teacher exists; training a small model heavily is competitive when no teacher exists and the budget is large.

4. By reverse water-filling, sparsity is a poor use of bits for homogeneous weights; combine it with quantization only where it unlocks execution savings or exploits genuine heterogeneity.

Chapter 14

Systems: Kernels, Formats, and the No-Speedup Trap

A pruning method is only as valuable as the speed, memory, or cost it actually saves on a real software and hardware stack. This chapter explains, from first principles, why unstructured sparsity rarely yields speedups on GPUs; quantifies index overhead and break-even densities for common formats; describes block-sparse and semi-structured alternatives and the "load sparse, compute dense" technique; covers CPU and accelerator opportunities; discusses the interaction with the KV cache, serving engines and dimension alignment; and ends with a measurement checklist.

14.1 The roofline view of LLM inference

The roofline model bounds attainable throughput by $\min\left(F_{\text{peak}},\, B_{\text{mem}} \cdot I\right)$, where I is arithmetic intensity (FLOPs per byte moved). For a weight matrix of P_{w} parameters applied to B tokens at once (batch of decode steps), the FLOPs are $2P_{\text{w}}B$ and the weight bytes are $P_{\text{w}}\, b/8$, so $I \approx 16B/b$ FLOPs per byte, ignoring activation traffic; in bf16 ($b = 16$), $I \approx B$.

Example 14.1 (Where is the ridge? Approximate vendor figures†)**.** A current high-end GPU has a dense bf16 tensor-core peak of order 10^{15} FLOP/s (about 989 TFLOP/s dense, with a nominal 2× for 2:4 sparse) and about 3.35 TB/s of HBM bandwidth, so the ridge point is $I^{\star} \approx 989/3.35 \approx 295$ FLOP/byte. With bf16 weights, $I \approx B$, so single-GPU decode stays *memory-bound up to batch sizes of order* 10^2. For 4-bit weights $I \approx 4B$, so the ridge moves to $B \approx 74$. In the memory-bound regime, speed is determined by weight bytes read (and KV bytes), not by FLOPs; so a method helps only if it reduces bytes read *efficiently* (high achieved bandwidth).

14.2 Index overhead and break-even density

14.2.1 Formats

- **CSR/COO-like**: each non-zero stores a value (b bits) and a column index (16 bits for $d_{\text{in}} \le 65536$), plus row pointers (negligible).
- **Bitmap**: one bit per original weight marks non-zeros, followed by packed values.
- N:M: fixed density with a few metadata bits per group (4 bits per group for 2:4).
- **Block-sparse**: dense blocks (e.g., 16×16 to 128×128) with a block index; overhead amortized over the block.

Proposition 14.1 (Break-even density for storage)**.** *Compared with a dense matrix of b-bit values, a CSR layout with 16-bit indices is smaller iff density $\rho < b/(b+16)$; a bitmap layout is smaller iff $\rho < (b-1)/b$.*

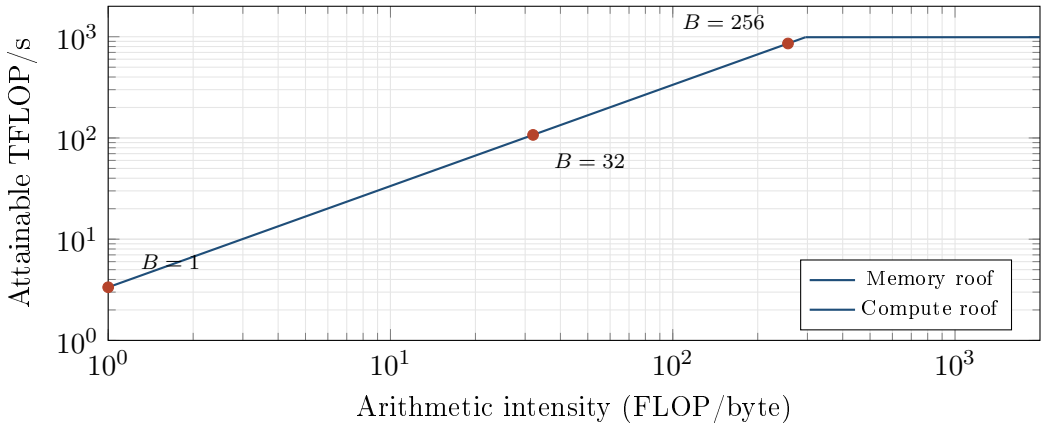


Figure 14.1: Illustrative roofline using approximate vendor figures for a high-end GPU (989 TFLOP/s dense bf16, 3.35 TB/s). Points mark attainable throughput for bf16 weight-bound matmuls at decode batch sizes 1, 32, 256 with $I \approx B$. At low batch, throughput is set by bandwidth, so byte reduction is what matters.

Table 14.1: Break-even densities (*) from Proposition 14.1 (exact arithmetic) and the sparsity required for a storage saving, by value precision b.

b (bits)	CSR (16-bit idx)		Bitmap	
	density*	sparsity	density*	sparsity
16	0.500	$> 50\%$	0.9375	$> 6.25\%$
8	0.333	$> 66.7\%$	0.875	$> 12.5\%$
4	0.200	$> 80\%$	0.750	$> 25\%$

Proof. CSR stores $\rho(b + 16)$ bits per original weight versus b; bitmap stores $\rho b + 1$ versus b. Solve the inequalities. □

Two observations follow. First, index overhead scales inversely with precision: for the 4-bit weights that dominate modern deployment, naive CSR needs $> 80\%$ sparsity merely to break even on storage. Second, bitmaps are far cheaper than explicit indices at moderate sparsity (and are what the best unstructured GPU formats use), but decoding a bitmap on the fly costs instructions, which matter when the kernel is bandwidth-bound and tensor-core-fed.

14.3 Why unstructured sparsity rarely speeds up GPUs

The following effects compound.

1. **Tensor cores want dense tiles.** The peak math throughput is available only to dense (or 2:4) tiles; scalar sparse kernels run on CUDA cores at a fraction of tensor-core throughput. For compute-bound shapes this alone forecloses unstructured sparsity unless sparsity exceeds the compute ratio (tens of times) between tensor and scalar paths.

2. **Irregular access.** A column-index gather defeats coalescing; achieved bandwidth falls well below peak, so even when bytes are halved, time need not fall by half.

3. **Low reuse for skinny matrices.** At decode, the weight matrix is streamed once, with no reuse to amortize decode overhead.

4. **Load imbalance.** Rows with different non-zero counts cause warp divergence and tail effects.

5. **Dense baselines are excellent and moving.** Low-bit dense kernels (e.g., MARLIN-class kernels [66]) achieve near-ideal bandwidth with 4-bit weights, so a sparse kernel must beat a baseline that has already cut bytes by 4×.

6. **Amdahl.** Attention, KV reads, normalization and sampling do not benefit.

Kernels designed for deep-learning-sized sparse matrices [68] narrowed the gap on earlier hardware but, as I understand, still needed high sparsity (roughly in the range of 70–80% or more for FP32 on V100-era hardware†) to beat dense kernels, and the break-even rises when the dense baseline uses tensor cores.

14.3.1 Load-as-sparse, compute-as-dense

Flash-LLM [67] targets the memory-bound skinny-matmul regime of LLM decoding with unstructured sparsity: sparse weights are loaded from global memory in a compressed format (saving bandwidth), expanded into dense tiles in shared memory/registers, and fed to tensor cores for dense computation. This "load-as-sparse, compute-as-dense" design reduces global memory traffic without trying to skip the arithmetic. It makes unstructured sparsity useful on GPUs at moderate-to-high sparsity for memory-bound shapes; the dense expansion cost and decode complexity limit the gain, and the technique competes with low-bit quantization, which gets comparable bandwidth reduction with simpler dequantization.

14.4 Block sparsity, N:M, and the flexibility–efficiency spectrum

Block-sparse kernels [69] skip entire dense blocks, achieving close to proportional speedups when blocks are large enough to keep tensor cores busy, but requiring that the pruning *select* blocks (rather than individual weights), which costs accuracy: the coarser the block, the more the mask must discard useful weights adjacent to useless ones. Applied to LLM weights, whose important entries are scattered, block pruning at useful block sizes is damaging unless the model is retrained with the block constraint.

14.5 CPUs, edge devices and accelerators

CPUs. On CPUs, decoding is also bandwidth-bound but with lower bandwidth (tens to a few hundred GB/s) and with large caches; unstructured sparsity kernels (including those with bitmap or run-length formats) can turn byte reduction into speed more readily than on GPUs, since scalar and vector units are the compute path anyway and there are no tensor cores to forgo. Industrial CPU inference runtimes have long exploited unstructured sparsity for this reason†. Edge deployment more often uses 4-bit quantization plus structured size reduction (smaller dense models) because the toolchains are mature.

Table 14.2: Sparsity patterns ordered from most flexible/least efficient to least flexible/most efficient.

Pattern	Accuracy at equal sparsity	Hardware path	Notes
Unstructured	best	custom kernels (Flash-LLM-type); CPUs; some accelerators	index overhead; GPU speedups rare
N:M (2:4)	between	vendor sparse tensor cores	fixed 50%; Chapter 7
Block-sparse (16–128) [69]	worse	block-sparse kernels, dense tiles inside blocks	well-supported in frameworks; accuracy drops with block size
Structured (channels, heads, layers)	worst	any dense kernel	smaller dense model; universal

Dataflow accelerators and ASICs. Hardware designed around sparsity can exploit unstructured patterns: the EIE accelerator [204] executed compressed sparse networks with weight sharing; wafer-scale dataflow systems advertise native support for unstructured weight sparsity (the author has not verified current claims and cites this as a hardware-vendor assertion). The general lesson is that unstructured sparsity needs hardware co-designed for it; commodity GPUs are optimized for dense and 2:4 math.

Flash and memory hierarchies. Activation sparsity lets a runtime fetch only the needed rows from flash (LLM-in-a-flash [157]) and place hot neurons in fast memory (PowerInfer [158]); these are memory-hierarchy designs where sparsity acts as a prefetch/caching signal rather than as a compute skip.

14.6 The KV cache: what weight pruning does not touch

For long contexts, the KV cache rivals or exceeds the weights. The KV cache per token is $2\,L\,h_{kv}\,d_h$ values.

Example 14.2 (KV cache arithmetic; exact)**.** For the Llama-3.1-8B shape ($L = 32$, $h_{kv} = 8$, $d_h = 128$) in 16-bit, KV bytes per token $= 2{\cdot}32{\cdot}8{\cdot}128{\cdot}2 = 131{,}072\,\text{B}$ (128 KiB). At a 128k-token context the cache of one sequence is 16 GiB, equal to the bf16 weights (≈ 16 GB). A decode step at 128k context therefore reads about 32 GB; *halving the weights by any method reduces step traffic by only 25%*, and only layer or KV-head removal reduces KV bytes. Width/FFN pruning and unstructured weight sparsity leave the KV cache untouched.

Methods that shrink the KV cache (GQA, KV quantization, token eviction such as H2O [146], streaming windows with sinks [145], retrieval/streaming head splits [144]) are complementary and, at long context, more valuable than weight pruning. A complete deployment plan reports the speedup as a function of context length.

14.7 Serving-engine integration

- **Smaller dense models** (width/depth pruning) run unmodified, but non-uniform per-layer widths (different heads or FFN sizes per layer, as produced by adaptive pruning like FLAP or Puzzle) are not supported by every engine; check support for heterogeneous layer shapes, or constrain pruning to uniform widths.
- **Alignment.** Round pruned dimensions to hardware-friendly multiples (a common guideline is multiples of 8 for 16-bit tensor-core use and larger multiples such as 64 or 128 for best efficiency; consult the vendor guidance for the target GPU), since odd sizes cause padding or fallback kernels.
- **Tensor parallelism.** Pruned head and channel counts must remain divisible by the tensor-parallel degree; GQA group structure must be consistent across shards.
- **Sparse kernels** (2:4, activation sparse) are integrated in a subset of engines [64, 66, 72]; fused attention and quantized kernels often have no sparse variants, so enabling sparsity may disable other optimizations.
- **MoE.** Expert-pruned models with fewer experts per layer need consistent expert-parallel placement; non-uniform expert counts per layer complicate placement.

14.8 Measurement checklist

The no-speedup trap: what to measure before claiming a speedup

1. Report **end-to-end** tokens/s (and time-to-first-token and time-per-output-token separately) on a named GPU/CPU with software versions, not matmul microbenchmarks.
2. Sweep **batch size** (1, 4, 16, 64, 256) and **context length** (short, 8k, 32k, 128k); activation sparsity and MoE pruning behave very differently across batch sizes (Proposition 12.3, Example 11.1).
3. Compare against the **best dense baseline at equal memory**: a 4-bit dense model, not the 16-bit baseline.
4. Include **prefill** and **decode** separately: 2:4 helps compute-bound prefill, byte-reduction helps decode.
5. Check **power and cost per token**, not only latency, when comparing across hardware.
6. Verify **accuracy under the same kernels** (some fused sparse kernels use lower precision accumulation).

Key lessons of Chapter 14

1. Decoding is memory-bound up to batch sizes of order 10^2, so only *efficient byte reduction* speeds it up.
2. Index overhead is exact and large for low precision: CSR with 16-bit indices breaks even

at 80% sparsity for 4-bit values (Table above).

3. Unstructured sparsity on GPUs loses to dense low-bit kernels for six compounding reasons; load-as-sparse designs and CPU/ASIC targets are the exceptions.
4. The KV cache is untouched by weight pruning and dominates at long context (Example 14.2); report speedups across context lengths.
5. Structured pruning wins on deployability: it needs no new kernels, only alignment and engine support for the resulting shapes.

Chapter 15

Evaluation Methodology and What Pruning Removes

Pruning papers conventionally report WikiText-2 perplexity and a handful of zero-shot accuracies. This chapter explains why that protocol systematically understates damage, derives what perplexity can and cannot reveal, gives a transfer bound that explains calibration-data dependence, and synthesizes the evidence on *what* pruning removes: knowledge versus skill, tails versus head, and side effects on safety, bias and calibration. It ends with a recommended evaluation protocol.

15.1 Perplexity: what it measures and what it hides

Perplexity on a corpus is exp of the average per-token cross-entropy: PPL = exp(CE) [192, 193]. The change in cross-entropy between a dense model p_D and a pruned model p_P on data from distribution $p^\star$ is

$$\Delta\mathrm{CE} = \mathbb{E}_{p^\star} \log \frac{p_D}{p_P} = \mathrm{KL}(p^\star \| p_P) - \mathrm{KL}(p^\star \| p_D). \tag{15.1}$$

Thus perplexity change measures the difference in KL to the data distribution, *averaged over all tokens*, and so is dominated by the bulk of high-frequency, low-surprise tokens. The relationship between PPL and loss is exponential, which makes ratios intuitive.

Example 15.1 (From perplexity ratios to nats; computed)**.** A LLaMA-2-7B model going from 5.12 to 6.72 perplexity (MaskLLM 2:4, Table 5.1) has $\Delta\mathrm{CE} = \ln(6.72/5.12) = 0.272$ nats per token (0.39 bits). Magnitude pruning of LLaMA-7B from 5.68 to 17.29 is $\Delta\mathrm{CE} = 1.113$ nats; SparseGPT's 5.68 to 7.22 is 0.240 nats. A perplexity increase from 5.12 to 6.72 may look modest as a ratio (1.31×), but it equals 0.39 extra bits for every token.

Example 15.2 (A model can forget facts without perplexity noticing; illustrative arithmetic)**.** Suppose 0.5% of the tokens in an evaluation corpus are "fact-bearing" tokens (named entities, numerical values), on which the dense model assigns probability 0.5, and the pruned model has forgotten the facts and assigns 0.05 to the correct token (a tenfold drop), while behaving identically elsewhere. The cross-entropy change is $0.005 \cdot \ln 10 = 0.0115$ nats, i.e., perplexity rises by 1.2% ($\exp(0.0115) = 1.0116$) while exact-match recall of those facts falls by an order of magnitude. This is an assumption-driven toy, but it shows why a 1–2% perplexity increase is compatible with losing a large share of rare factual knowledge.

15.2 Downstream benchmarks and their blind spots

- **Zero-shot multiple choice** (ARC, HellaSwag, PIQA, WinoGrande, BoolQ; harness of [188]) scores by comparing log-likelihoods of options. These are mostly insensitive to pruning at moderate sparsity because they test common-sense patterns that the bulk of the model retains; differences of 1–2 points are within noise (the standard error of an accuracy near 50% on a 1000-item benchmark is $\sqrt{0.25/1000} = 1.6$ points, so a 95% interval is about ±3.1 points for a single model).

Table 15.1: Sensitivity of evaluation categories to pruning (author's synthesis; *not* a measured ranking).

Evaluation	Typical sensitivity	Why
WikiText-2 / C4 perplexity	low–moderate	dominated by frequent tokens; Example 15.2
Zero-shot multiple choice	low at $\leq$50%	likelihood ranking among 2–4 options; small signal
MMLU / closed-book QA	moderate–high	depends on stored facts; capacity-limited
GSM8K / MATH / code	high	multi-step, compounding errors; algorithmic skills
Long-chain reasoning models	very high	long on-policy traces; distribution shift from calibration
Long-context retrieval	varies (head/layer-dependent)	localized retrieval circuits
Safety / bias / calibration	unknown a priori	side effects not targeted by objective

- **Knowledge-intensive tasks** (MMLU [189], closed-book QA) probe stored knowledge. The LLM-KICK benchmark [180] was designed precisely to show that perplexity-preserving compression can mask large losses on knowledge-intensive tasks, finding in its study that pruned LLMs degrade markedly at modest sparsity on such tasks whereas quantized models are more robust (as I recall the headline finding†).

- **Generative reasoning** (GSM8K [190], HumanEval [191], long chain-of-thought tasks). Errors compound across generated tokens, making these the most pruning-sensitive tasks: Lucas et al. report that pruning hurts reasoning models more than standard LMs and that pruned models may emit more thinking tokens while performing worse, making them slower‡ [121]; Shrestha et al. report weak recovery on GSM8K and HumanEval+ after layer pruning‡ [101]; SlideSparse reports 2:4 pruning of Qwen3 dropping a reasoning metric from 54% to 15%‡ [64].

- **Long-context tasks** (needle retrieval, multi-document QA). Specific heads implement retrieval [143]; head or layer pruning evaluated only at short context can miss their loss (Chapter 11).

15.3 Calibration-data dependence

All calibration-based methods (SparseGPT, Wanda, FLAP, Minitron-style importance, REAP) estimate statistics from a calibration set Q and are then evaluated on a distribution P. Two results make the dependence precise and one cautionary study shows its size.

Proposition 15.1 (Transfer of reconstruction error across distributions)**.** *Let $H_Q = 2\mathbf{X}_Q\mathbf{X}_Q^\top$ and $H_P = 2\mathbf{X}_P\mathbf{X}_P^\top$ be the layerwise Hessians for the calibration distribution and the deployment distribution, $H_Q \succ 0$. For a weight-error vector $\delta = w - \hat{w}$ of one row, the deployment*

reconstruction error satisfies

$$\tfrac{1}{2}\delta^\top H_P \delta \;\le\; \kappa \cdot \tfrac{1}{2}\delta^\top H_Q \delta, \qquad \kappa = \lambda_{\max}\big(H_Q^{-1} H_P\big),$$

and the bound is tight for some δ. If H_Q has (near-)zero energy in a direction used by P, κ is unbounded.

Proof. The generalized Rayleigh quotient $\delta^\top H_P \delta / \delta^\top H_Q \delta$ is bounded by the largest generalized eigenvalue of the pencil (H_P, H_Q), which equals $\lambda_{\max}(H_Q^{-1} H_P)$, and equality is attained at the corresponding eigenvector. □

The proposition formalizes calibration coverage: a method that minimizes error on Q (as SparseGPT does explicitly, Wanda implicitly) can leave arbitrarily large error on P in directions Q never excites.

Example 15.3 (Coverage failure; exact arithmetic)**.** Take two input channels with $H_Q = \mathrm{diag}(100, 0.01)$ (calibration activates channel 1 only, e.g., English prose) and $H_P = \mathrm{diag}(1, 1)$ (deployment uses both, e.g., code that excites channel 2). Then $\kappa = \max(1/100, 1/0.01) = 100$: an error that is invisible on Q can be 100× larger on P. A Wanda-style score ranks channel 2's weights by $|w| \cdot \sqrt{0.01} = 0.1|w|$, ten times below the same weight on channel 1, so the weights serving channel 2 are pruned preferentially.

Empirical evidence. Williams and Aletras [185] study how calibration data affect post-training pruning and quantization and find that the choice of calibration data can change downstream results more than perplexity would suggest; Bandari et al. [186] question whether the C4 corpus used as the default is optimal for pruning, examining alternatives (including data with arithmetic content for reasoning tasks). 2025–2026 work addresses the issue directly: GPrune-LLM groups neurons by calibration-sensitivity and learns per-module sparsity‡ [88]; multi-domain calibration with iterative pruning [86]‡; REAM reports a calibration-mix trade-off between multiple-choice and generative performance‡ [132]; and RAC reconstructs on the model's own reasoning traces‡ [121]. Practical guidance: use at least 10^5 tokens (the standard 128×2048 recipe is 2.6×10^5) drawn from a mixture covering the deployment domains, including code, math, multilingual text, instruction/chat formats, and, for reasoning models, on-policy generations; report variance over calibration seeds.

15.4 What does pruning remove?

15.4.1 Knowledge capacity

Allen-Zhu and Li [205] find in controlled synthetic settings that language models store roughly 2 bits of knowledge per parameter when sufficiently trained. If this scaling is a reasonable guide for real LLMs, removing a fraction ϕ of parameters (and not retraining) removes up to a fraction ϕ of the stored bits, *unless* the model did not saturate capacity, as is the case for rarely-seen facts, which are stored with lower reliability. Pruning cannot create capacity; at best it removes capacity the model was not using, and distillation can reinforce high-frequency knowledge but cannot restore bits that no longer fit. This provides a theoretical reason why rare factual knowledge is the first casualty, and why structured width pruning of FFNs, which hold the key–value memories [150, 151], hits recall more than it hits syntax.

Table 15.2: Hypotheses linking what is removed to what is lost. These are the author's synthesis of the evidence cited in this chapter and in Chapters 9, 11; each row is a hypothesis with partial support, not an established law.

Removed	Most-affected capability	Supporting evidence
Unstructured weights ≤50% (with compensation)	small broad degradation; rare facts	small PPL change but bias/knowledge tasks diverge [180, 187]
Unstructured weights ≥70%	everything; sharp cliff	one-shot methods diverge beyond the knee (Chapter 6)
2:4 (50%)	generative reasoning	Qwen3 54%→15% in one report‡ [64]
FFN channels (width)	factual recall	FFNs as key–value memories [151]; capacity argument
Hidden dimension	broad; massive-activation dims	[147]
Deep layers (depth)	multi-step generation, arithmetic	[101]; flat QA vs rising loss [95]
Attention heads	long-context retrieval	retrieval heads [143]
Experts	domain/language/task skills absent from calibration	[131, 132]
Activations (contextual)	noise-like, widespread, small	[159, 160]

15.4.2 Evidence on tails

- Hooker et al. [181] show for image classifiers that compression disproportionately affects a small subset of examples (pruning-identified exemplars), typically atypical or underrepresented inputs, even when overall accuracy is almost unchanged; later work shows compression can amplify fairness gaps [182] and that pruning has a disparate impact on groups with lower accuracy [183], an effect linked to larger gradient norms for the underrepresented groups.
- In LLMs, analogous tail effects appear in knowledge-intensive tasks [180], in bias measures that diverge from perplexity (Wanda-pruned models in [187]), and in domain-specialized experts removed from MoE models by calibration mismatch [132].
- Super weights and massive activations [147, 148] are a different failure type: a handful of parameters are *individually* critical, and removing them is catastrophic regardless of average sparsity.

15.4.3 Knowledge versus skill

The terms are not sharply separable, but the evidence suggests different pruning types damage different capabilities.

15.4.4 Side effects: safety, alignment, fairness, calibration

Decoding Compressed Trust [184] evaluated compressed (quantized and pruned) chat models across several trust dimensions (toxicity, stereotype bias, privacy, adversarial robustness, fairness, ethics, truthfulness); its headline conclusions, as I recall them†, were that moderate quantization is generally safer for trustworthiness than pruning at comparable efficiency, that

Table 15.3: Minimum evaluation protocol for accepting a pruned/sparse LLM.

Category	Contents
Language modeling	Perplexity on ≥ 2 corpora (one not used for calibration), plus KL to the dense model on held-out text
Likelihood benchmarks	Zero-shot multiple choice (ARC, HellaSwag, PIQA, WinoGrande); report confidence intervals
Knowledge	MMLU or equivalent; closed-book QA; a long-tail knowledge slice
Reasoning/generation	GSM8K/MATH, HumanEval/MBPP-like code, instruction-following, long chain-of-thought where relevant; count generated tokens
Long context	Needle retrieval and multi-document QA at the deployed context length
Multilingual / domain tails	Slices for each language and domain in deployment
Safety and fairness	Refusal/jailbreak suites, toxicity, BBQ-style bias, privacy probes; compare to dense baseline
Calibration	ECE / Brier on multiple-choice or verbalized confidence
Systems	End-to-end latency and throughput across batch sizes and context lengths on the target hardware
Robustness of the procedure	$\geq$3 calibration seeds; calibration-domain ablation; recovery-budget stated

extreme compression degrades trust, and that benign-looking compression can change trust dimensions in non-monotonic and unexpected ways. A 2026 multi-method study of bias under pruning (Gemma-2-9b-it, Mistral-7B-Instruct-v0.3, Phi-3.5-mini-instruct; random, magnitude and Wanda pruning at 10–70%; 12,148 BBQ items; 5 seeds) found that Wanda kept perplexity nearly unchanged (a 3.5% increase at 50% for Mistral-7B) while its Stereotype Reliance Score rose by 83.7%, with 47–59% of previously unbiased items turning stereotypical at 70% sparsity, whereas random pruning wrecked language ability but left bias at chance[‡] [187]. The authors name this the "smart pruning paradox". These are single-study results with specific models and a single bias benchmark, but they illustrate that quality-preserving pruning is not automatically behavior-preserving.

Calibration of predictive confidence (expected calibration error) and refusal behavior of aligned models under pruning are less systematically studied, and the author is not aware of definitive results for current-generation chat models; they should be measured rather than assumed.

15.5 A recommended evaluation protocol

Common evaluation errors

- Calibrating and evaluating on the same corpus (inflates results by construction).
- Reporting a single seed for the calibration sample and claiming differences below the benchmark noise (about ± 3 points on 1000-item sets).
- Evaluating generation with greedy decoding on one prompt template; pruned models can be more sensitive to prompt format.
- Ignoring generated length for reasoning models: a pruned model that thinks longer is

slower, which erases speedups [121].

- Comparing to a dense baseline of the same size trained on less data.

Key lessons of Chapter 15

1. Perplexity change is an average over mostly easy tokens; a few-percent increase can coexist with large losses on rare knowledge (Example 15.2).
2. Calibration coverage is a precise quantity (Proposition 15.1); unexcited directions are pruned and then unrecoverable on deployment data.
3. Different pruning types damage different capabilities (Table above); the most damaged are generative reasoning, long-tail knowledge, long-context retrieval and behaviors not covered by calibration.
4. Side effects on bias, safety and calibration are real and not predicted by perplexity.
5. Adopt a multi-axis protocol with confidence intervals and calibration-seed variance.

Chapter 16

Recent Advances Timeline (2022–October 2026) and Current Capabilities

This chapter first lays out a dated timeline of the main developments, then summarizes what is achievable today: sparsity levels versus quality retention, and real speedups, with every sourced number tagged by how it was verified. All numbers come from the cited papers; none were produced by the author's experiments, and numbers from different papers use different models, hardware, and evaluation harnesses and should not be compared as if they were a single benchmark.

16.1 Timeline

Months refer to the first arXiv submission or announcement as best known to the author; venues are given where known. Items marked ‡ were seen on abstract or paper pages in October 2026.

16.1.1 Trends visible in the timeline

1. **From weights to systems-aware structure.** 2022–2023 emphasized one-shot unstructured pruning (OBC, SparseGPT, Wanda); from 2024 the emphasis shifts to structured removal with distillation (Minitron, Elastic), semi-structured sparsity that hardware supports (MaskLLM, SlideSparse, SpenseGPT) and dynamic sparsity (TEAL and successors).
2. **MoE becomes the dominant frontier architecture,** so expert compression (REAP, REAM) emerges in late 2025–2026.
3. **Reasoning models change evaluation.** From mid-2025, pruning research increasingly tests on long-chain reasoning, finding larger damage than on classic benchmarks (RAC, layer-pruning limits, SlideSparse).
4. **Calibration and side effects receive attention** (GPrune-LLM, multi-domain calibration, bias studies) as the cheap-method frontier saturates.
5. **Hybrid and non-transformer architectures** need pruning recipes of their own (Minitron-SSM, Nano 2, Elastic).
6. **Training-free activation sparsity is an active area** in 2026: WiSparse and Prox report new end-to-end speedups beyond TEAL-era numbers.

Table 16.1: Timeline of notable pruning and sparsity work for LLMs, 2022–2026.

When	Development
2022	Optimal Brain Compression, exact layerwise OBS framework [36]; fast post-training pruning for transformers [79]; CoFi structured pruning with distillation [76]; oBERT second-order pruning [37]; CALM confident early exit for LMs [102]; Chinchilla scaling [176].
Jan 2023	SparseGPT: one-shot 50% pruning of 100B+ models [42] (ICML 2023); GPTQ shares the Hessian machinery [43].
Mid 2023	Deja Vu contextual sparsity [153]; LLM-Pruner [73] (May); Wanda [44] (Jun); Flash-LLM unstructured kernels [67] (Sep).
Oct–Dec 2023	Sheared-LLaMA [75]; ReLU Strikes Back [154]; FLAP [80]; LLM in a flash [157] and PowerInfer [158]; Mixtral MoE [125] (Jan 2024).
Jan–Mar 2024	SliceGPT [74]; ReLU^2 study [155]; ProSparse [156]; ShortGPT [94]; unreasonable ineffectiveness of deeper layers [95]; Bonsai [81]; OWL [46], RIA [45], DSnoT [47] (ICLR/ICML 2024 cycle).
Apr–Jul 2024	CATS [159]; LayerSkip [103]; Mixture-of-Depths [104]; Not-all-experts-are-equal [133]; Minitron [111]; Llama 3 herd [118].
Aug–Dec 2024	Llama-3.1-Minitron 4B and Mistral-NeMo-Minitron 8B [112]; TEAL [160] (Aug); MaskLLM [62] (Sep); Llama 3.2 1B/3B via pruning and distillation [117] (Sep); MoE-Pruner [137]; super weights [148]; Puzzle [198] (Nov); DeepSeek-V3 [127] (Dec); Sparse-Llama-3.1-8B-2of4 release [203].
Jan–Jun 2025	Average-parameter-count sparse scaling law [171]; DeepSeek-R1 [119]; 2:4 activation sparsity [63] (Mar); Minitron-SSM [210] and Nemotron-H [114] (Apr); Qwen3 [120] and Llama-Nemotron [199] (May).
Jul–Sep 2025	LaRoSA [161]; Kimi K2 [128]; Nemotron Nano 2 [113]; GLM-4.5 [129]; gpt-oss [130]; reasoning-aware pruning (RAC) [121]‡; N:M activation sparsity benchmark [209]‡.
Oct–Dec 2025	REAP expert pruning [131]‡ (ICLR 2026); Nemotron Elastic [115]‡ (Nov).
Jan–Mar 2026	Multi-domain iterative structured pruning [86]‡; limits of layer pruning for reasoning [101]‡; WiSparse [206]‡; SlideSparse [64]‡ (ICML 2026); GPrune-LLM [88]‡.
Apr–Jun 2026	REAM [132]‡; SparseForge [208]‡; Putri [87]‡; bias under pruning [187]‡; SMET sparse training [173]‡ (ICML 2026); sparse LMs under data scarcity [174]‡; SpenseGPT [65]‡.
Jul–Oct 2026	Prox training-free FFN sparsity [207]‡; LILA calibration-free structured pruning [89]‡ (Sep).

Table 16.2: One-shot weight-pruning quality reported in the literature (WikiText-2 perplexity; see Table 5.1 for sources). Other 2:4 evidence: learned masks close most of the gap at training-run cost.

Setting	Reported	Source / status
LLaMA-7B, 50% unstructured	5.68→7.22 (SparseGPT), 7.26 (Wanda); magnitude 17.29	[44]†
LLaMA-2-7B, 50% unstructured	5.12→6.51 (SparseGPT), 6.42 (Wanda)	[44]†; dense 5.12‡
LLaMA-2-7B, 2:4	dense 5.12; prior one-shot $\geq$ 10; MaskLLM 6.72	[62]‡
LLaMA-2-7B, 2:4 + 5B-token recovery	57.27% avg. zero-shot vs. dense 56.43%	SparseForge [208]‡
Llama-3.1-8B, 2:4 + 13B-token distillation	98.37% recovery of OpenLLM-v1 average (62.16 vs. 63.19); 97.3% on Mosaic Gauntlet	model card [203]‡
OPT-175B / BLOOM-176B, 50%	negligible perplexity increase; $<$ 4.5 h on one GPU	[42]†

16.2 What is achievable today

16.2.1 Weight sparsity: levels and quality

16.2.2 Structured and distilled models

16.2.3 MoE expert compression

REAP reports near-lossless code generation for Qwen3-Coder-480B and Kimi-K2 after pruning 50% of experts‡ [131]; REAM at 25% expert reduction reports generative-suite scores of 69.8 (Qwen3-30B-A3B-Instruct-2507; dense 70.9), 72.9 (Qwen3-Coder-Next; dense 72.9), 71.5 (Qwen3-Next-80B-A3B-Instruct; dense 72.9), and 73.9 (GLM-4.5-Air; dense 77.1)‡ [132]. Restoring generative quality on general (non-code) suites at 50% remains harder (Table 11.1).

16.2.4 Real speedups

Two patterns stand out: activation-sparsity and semi-structured methods report end-to-end gains between about 1.2× and 2× in current work, and none of the *unstructured weight* methods in this table reports a GPU wall-clock gain, which is consistent with Chapter 14. Larger gains come from *architecture-level* changes (hybrid architectures, smaller dense models, NAS), where pruning is one ingredient among several.

Table 16.3: Structured/distilled compression results with sourced numbers.

Result	Numbers	Source / status
Minitron (Nemotron-4 15B→8B,4B)	up to ∼40× fewer tokens than from scratch; ∼1.8× family compute saving; up to ∼16% MMLU gain	[111]†
Llama-3.1-8B→4B; NeMo-12B→8B	depth and width variants; base weights released	[112]‡
Sheared-LLaMA 1.3B, 2.7B	∼3% of from-scratch compute; 50B tokens continued pre-training	[75]†
SliceGPT	up to 25% parameters removed; ∼99%/99%/90% zero-shot retention (LLaMA-2 70B, OPT 66B, Phi-2)	[74]†
Minitron-SSM (Nemotron-H 8B→4B)	up to 40× fewer tokens; ∼2× faster inference	[210]‡
Nemotron Nano 2 (12B→9B)	128k context on one A10G (22 GiB); up to 6× throughput vs. Qwen3-8B in reasoning settings (architecture-dependent)	[113]‡
Nemotron Elastic (12B→9B,6B)	110B tokens; > 360× cost reduction vs. from-scratch per size; ∼ 7× vs. prior compression	[115]‡
Puzzle (Llama-3.1-70B → 51B)	∼ 2.17× throughput on one H100 (reported)	[198]†

16.2.5 Rules of thumb

The following summary expresses the author's judgment from the above evidence (estimates, not measured values).

Takeaway

What is achievable today: roughly half of the weights of 7B+ models can be zeroed after training with a modest perplexity penalty; half of experts of large MoEs can be removed on matched domains; half of activations can be skipped at small batch; quarter-to-half of dense width/depth can be removed given a distillation run. What is *not* yet demonstrated robustly: lossless 2:4 on reasoning models without retraining; general-purpose 50% expert pruning; sparsity-based speedups at high batch size.

Table 16.4: Reported wall-clock gains from sparsity-based methods. Not comparable across rows: different models, hardware, precision, batch size and baselines. Rows marked † were recalled and not re-verified.

Method	Setting	Reported gain	Source
TEAL	Llama-2/3, Mistral 7B–70B; 40% / 50% activation sparsity	up to 1.53× / 1.8× decode	[160]‡
LaRoSA	LLaMA2-7B, 40%	1.30×	[161]‡
CATS	Llama-2-7B, Mistral-7B, 50%	∼15% lower token latency	[159]‡
WiSparse	Llama3.1, 50% (97% of dense retained)	21.4% end-to-end	[206]‡
Prox	10 LLMs, 70% FFN sparsity	up to 1.99× end-to-end decode	[207]‡
SlideSparse	Qwen2.5-7B, 6:8, compute-bound	1.33× (bound 4/3)	[64]‡
SpenseGPT	Qwen3-32B, Seed-OSS-36B, B200, FP8	up to 1.2× end-to-end decode	[65]‡
2:4 activation sparsity	FFN, training and inference	up to 1.3× FFN	[63]‡
LayerSkip	self-speculative decoding	up to $\sim$ 2.16× (summarization)	[103]†
Deja Vu	OPT-175B vs. FasterTransformer	up to $\sim$ 2×	[153]†

Table 16.5: Rule-of-thumb regimes (author's estimates informed by the cited sources).

Technique	Comfortable range	Recovery budget	Realized benefit
One-shot unstructured	≤50%	none	memory/storage only unless custom kernels
One-shot 2:4	50% fixed	learned masks or 10^9–10^{10} tokens	∼1.2–1.3× end-to-end
Post-training width pruning	≤20–25%	LoRA or small KD	proportional to size
Width pruning + KD	25–50%	10^{10}–10^{11} tokens	proportional to size; best quality
Depth pruning	≤20–25% layers for MC tasks; less for reasoning	KD for generation	proportional to depth
Expert pruning (MoE)	25–50% experts	none to light	memory; no FLOP change
Activation sparsity (training-free)	40–50%, 70% FFN in latest work	none	1.3–2× at small batch

Chapter 17

Pros, Cons, and Risks

17.1 Advantages and disadvantages by family

17.2 Risk register

Each risk below is stated with the mechanism, evidence, early-warning signals, and mitigations. Evidence tags: ‡ verified in preparation; † recalled.

17.2.1 R1: Accuracy cliffs

One-shot methods exhibit a knee: gentle degradation up to about 50–60% unstructured sparsity or 20–30% structured removal, then rapid collapse (Figures 2.2, and the illustrative curves in Chapter 6). Depth pruning exhibits a threshold in QA accuracy even while loss rises smoothly [95]. The cliff location is model-, task- and calibration-dependent, so a threshold validated on one model does not transfer to another.

R1 mitigations

Sweep sparsity in small steps around the target and plot task curves, not just perplexity; keep a safety margin (back off one step from the observed knee); re-run the sweep for each model version and task suite; prefer methods with non-uniform allocation to move the knee right (OWL, AlphaPruning).

17.2.2 R2: Compounding of small per-token errors in generation

Example 17.1 (Per-token degradation compounds; illustrative arithmetic)**.** Suppose a task requires $T = 200$ consecutive generated tokens to be right (e.g., a short derivation or code block) and the dense model gets each critical token right with probability 0.999, so success is $0.999^{200} = 0.82$. If pruning lowers per-token success by only 0.2 points to 0.997, the sequence success becomes $0.997^{200} = 0.55$, a 33% relative drop, though the per-token change is almost invisible in perplexity. For $T = 2000$ (long reasoning chains), $0.999^{2000} = 0.135$ versus $0.997^{2000} = 0.0025$. Long chains turn tiny per-token distribution shifts into large accuracy losses, which is one mechanism behind the sensitivity of reasoning models to pruning [101, 121].

17.2.3 R3: Brittleness

Pruned models can be more sensitive to prompt format, sampling temperature and decoding parameters, because the error margins that absorbed such perturbations have shrunk; instruction-tuned and aligned behavior may be partly erased. Mitigate by evaluating under several prompt templates and temperatures, and by distillation from the *aligned* teacher rather than from base-model likelihoods when the deployed model is aligned.

Table 17.1: Pros and cons of the main pruning and sparsity families (author's synthesis of the evidence in Chapters 5–14).

Family	Pros	Cons
One-shot unstructured (SparseGPT, Wanda)	No training; fast (minutes to hours); strong quality at 50%; simple; model unchanged otherwise; research substrate	Rarely any GPU speedup; index overhead; cliff beyond ∼60–70%; calibration-dependent; compensation overfits calibration
N:M / 2:4	Real hardware support; ∼44% smaller weights at 16-bit; composes with FP8/INT8; learned masks near-lossless on perplexity	Fixed 50%; lossy for reasoning; modest end-to-end gains; adapters break pattern; hardware-generation dependent
Structured width (LLM-Pruner, SliceGPT, FLAP)	Smaller dense model; universal speedup; no special kernels; composes with quantization	Fast quality loss beyond 20–30% without recovery; coupling constraints; GQA/RoPE constraints; non-uniform shapes may not be engine-supported
Width + distillation (Minitron)	Best quality per token spent; family-level amortization; works for hybrids and reasoning models	Needs teacher forward passes and 10^{10}–10^{11} tokens; license/provenance; capacity limits; architecture lock-in
Depth pruning	Latency drops linearly; trivial to implement; no kernels	Generative reasoning and arithmetic degrade; QA collapse threshold; weak recovery
MoE expert pruning/merging	Cuts memory of huge models; one-shot; near-lossless on matched domains	No FLOP reduction; calibration-domain dependent; router mismatch; load imbalance
Head pruning	Reduces attention params/KV (if KV heads removed)	Retrieval heads; long-context risk; GQA coupling
Contextual activation sparsity	Training-free variants; model intact; composes with quantization; 1.3–2× at small batch	Weights remain in memory; benefit vanishes with batching; needs custom kernels; per-token noise
Induced activation sparsity (ReLU-fication, ProSparse)	Very high sparsity (80–90%); good on-device fit	Needs continued training; behavior changes; evaluation burden
Sparse training (DST)	Potentially cheaper training and smaller models; scaling-law support	Mostly simulated sparsity; unstable at scale without fixes; no wall-clock gains on GPUs

17.2.4 R4: Calibration-set bias

Importance estimates reflect the calibration distribution; unexcited directions are pruned and are unrecoverable on deployment data (Proposition 15.1). Evidence: [86, 88, 132, 185, 186]‡. Mitigation: multi-domain calibration; calibration seeds and domain ablations; include on-policy generations for reasoning models.

17.2.5 R5: Safety, alignment, fairness side effects

Pruning can raise bias even when perplexity is flat (83.7% increase in a stereotype-reliance score for Wanda at 50%, with perplexity up only 3.5%, in one 2026 study‡ [187]) and affects some trustworthiness dimensions differently from quantization [184]; disparate impact on low-accuracy groups is established for classifiers [182, 183]. Mitigation: evaluate refusal, toxicity, bias and privacy suites before and after; use safety-inclusive calibration and distillation data; red-team the pruned model, not only the teacher.

Table 17.2: Order-of-magnitude recovery costs (illustrative arithmetic; assumes $\sim 40\%$ utilization of a $\sim 10^{15}$ FLOP/s accelerator, i.e., $\sim 4 \times 10^{14}$ FLOP/s effective, and the $6ND$ training approximation).

Recovery step	Tokens	FLOPs	GPU-hours (approx.)
One-shot SparseGPT/Wanda (7B)	$\sim 2.6{\times}10^5$ calibration	tiny	minutes
LoRA on 50k short samples	$\sim 2.5 \times 10^7$	$\sim 10^{17}$–10^{18}	< 1
Distill 4B student from 8B, 10B tokens	10^{10}	4×10^{20}	~ 280
Distill 4B student from 8B, 100B tokens	10^{11}	4×10^{21}	$\sim 2{,}800$
Train 4B from scratch, 15T tokens	1.5×10^{13}	3.6×10^{23}	$\sim 250{,}000$

17.2.6 R6: Loss of rare knowledge and tail behavior

Capacity arguments and empirical tail studies indicate that rare facts, low-resource languages, and unusual formats are lost first [180, 181, 205]. Perplexity does not show this (Example 15.2). Mitigation: tail-slice evaluation; domain-balanced calibration; retrieval augmentation to externalize long-tail facts.

17.2.7 R7: Recovery cost

(The distillation rows use 4×10^{10} FLOPs per token: teacher forward $2 \times 8 \times 10^9$ plus student forward–backward $6 \times 4 \times 10^9$, as in Example 10.1.) The key budgeting insight is that the recovery step, not the pruning algorithm, dominates cost for anything beyond mild compression, and that depth pruning and 2:4 pruning on reasoning models may not recover even with 100B tokens [101].

17.2.8 R8: Hardware and software dependence

Speedups depend on GPU generation, precision, kernel availability and engine versions. A 2:4 model gains nothing on hardware without sparse tensor cores; activation-sparsity kernels may exist for one stack only. Mitigation: define the deployment target first, then choose the sparsity pattern; keep a dense fallback.

17.2.9 R9: The no-speedup trap

Unstructured sparsity at 50% yields smaller *theoretical* FLOPs but typically no GPU latency reduction (Chapter 14); several papers report parameter or FLOP reductions without measured latency. Mitigation: require end-to-end latency measurements against a quantized dense baseline in any acceptance review.

Table 17.3: Risk matrix (author's subjective judgment of severity and detectability; ⋆ low to ⋆⋆⋆ high).

Risk	Most affected	Severity	Detectability with standard evals	Primary mitigation
R1 Accuracy cliffs	all, high sparsity	⋆⋆	⋆⋆⋆	fine sweeps
R2 Compounding in generation	depth, 2:4, reasoning	⋆⋆⋆	⋆ (PPL)	generative suites
R3 Brittleness	all	⋆⋆	⋆	prompt/temperature sweeps
R4 Calibration bias	Hessian/activation-based	⋆⋆⋆	⋆	multi-domain calibration
R5 Safety/bias	all, esp. Wanda-like	⋆⋆⋆	⋆	trust suites
R6 Tail knowledge loss	structured width, experts	⋆⋆⋆	⋆	tail slices
R7 Recovery cost	structured, 2:4	⋆⋆	⋆⋆⋆	budget planning
R8 Hardware dependence	2:4, kernels	⋆⋆	⋆⋆⋆	fix target first
R9 No-speedup trap	unstructured	⋆⋆⋆	⋆⋆⋆ if measured	latency baseline
R10 Reproducibility	all	⋆⋆	⋆⋆	seeds, versions
R11 Security/provenance	all	⋆⋆	⋆	re-run security review

17.2.10 R10: Reproducibility

- Calibration sample selection (seed, shuffling, sequence length) changes results by amounts comparable to method differences; report seeds and variance.
- Library versions (kernel numerics, Hessian precision) alter outcomes at high sparsity.
- Models released without masks or exact pipelines cannot be reproduced; the 2025–2026 preprints cited here are recent and largely unreplicated.
- Results across papers use different tokenizers/context lengths for perplexity, making cross-paper perplexity comparisons unreliable.
- Closed-weight or restrictively licensed teachers limit the reproducibility and legality of distilled students.

17.2.11 R11: Security and provenance

Compression changes a model's behavior; a pruned or distilled model should be treated as a new model for security review (jailbreak resistance, data extraction, backdoor behavior inherited from the teacher). The author is not aware of definitive studies on pruning-specific attack surfaces for current LLMs, and the open question is listed in Chapter 20.

Key lessons of Chapter 17

The risks that matter most are the ones that standard evaluation does not detect: compounding in long generation, calibration coverage, tails, and side effects. They are manageable with a protocol (Chapter 15) and with realistic recovery budgeting, but they are

the main reason that aggressive pruning of general-purpose, aligned, or reasoning models should be validated more heavily than perplexity-based papers suggest.

Chapter 18

Practical Decision Guide

This chapter converts the preceding analysis into decision aids: a flowchart, decision tables keyed by constraint, workload and hardware, four "recipe cards" with concrete parameters drawn from the cited literature, and a pre-deployment checklist. The guidance is the author's synthesis and reflects the evidence available in October 2026; it should be re-examined as kernels and models change.

18.1 Flowchart

18.2 Decision tables

18.2.1 By primary constraint

18.2.2 By workload

18.2.3 By hardware

18.3 Recipe cards

Recipe A: Structured pruning + distillation (smaller dense model)

1. Choose the target shape (round dimensions to multiples of 64/128; respect GQA and tensor-parallel divisibility).
2. Compute activation-based importance on a diverse calibration set ($\sim 10^3$ sequences; include code, math, multilingual, chat) [111].
3. Prune FFN channels first; then heads and hidden dimension; consider depth only if latency requires it [112].
4. If the teacher's pretraining data is unavailable, lightly fine-tune the teacher on the distillation corpus (teacher correction) [112].
5. Distill with forward-KL on logits for 10^{10}–10^{11} tokens from the aligned or base teacher according to the target.
6. Align (SFT/preference/RL) if needed; run the full protocol of Chapter 15.

Recipe B: MoE expert pruning/merging

1. Collect routing statistics on calibration data that mirrors deployment (code, chat, multilingual, tool use); use generative samples, not only multiple-choice text [131, 132].
2. Score experts by router-weighted activation (REAP-style) and prune 25% first; examine per-domain scores; try 50% only if domain slices hold.
3. Optionally try REAM-style merging for sensitive domains; compare on both multiple-choice and generative suites.

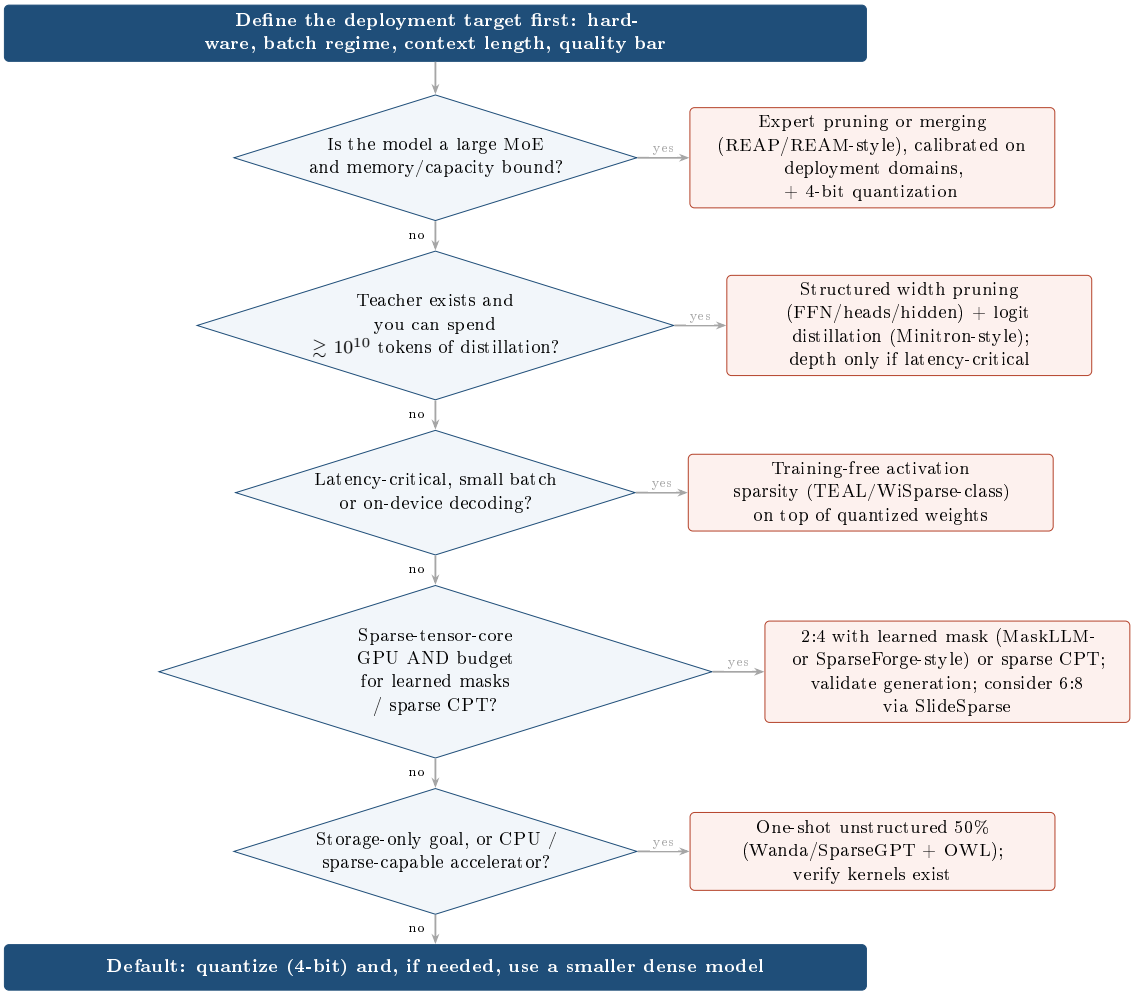


Figure 18.1: Decision flowchart (author's synthesis). Answers are read top-down; the first "yes" gives the primary recommendation, and techniques compose (for example, quantization plus any branch). The final fallback is dense quantization, which is the default baseline that every sparsity technique must beat on measured cost.

Table 18.1: Primary constraint → ranked techniques. "Q" = 4-bit weight quantization as baseline.

Constraint	First choices	Avoid / caveats
Weight memory (fit on fewer devices)	Q; expert pruning (MoE); structured + KD to a smaller model	unstructured sparsity without compressed format
Single-stream decode latency	Q; activation sparsity; smaller dense model; depth reduction (with validation)	expecting speedup from unstructured sparsity
High-throughput batched serving	Q; smaller dense model (KD); MoE expert pruning (fit larger batches)	activation sparsity (batch union), dynamic skipping
Prefill / compute-bound	smaller dense model; 2:4 on sparse-tensor-core hardware	speedup from weight-only quantization alone is limited when compute-bound
Long-context serving	KV-cache reduction (GQA, KV quantization, streaming heads) and depth/KV-head reduction	weight-only pruning (leaves KV untouched, Example 14.2)
Training cost of small model family	pruning + KD from one large teacher	single small model with no teacher
Edge / on-device	Q + structured smaller model; activation sparsity; flash-aware loading	GPU-centric sparse formats
Storage / download size	Q + entropy coding; unstructured sparsity with compressed storage	–

4. Fine-tune the router lightly if load imbalance or quality loss appears; re-balance expert-parallel placement.
5. Combine with weight quantization; measure memory and batch-size scaling.

Recipe C: Training-free activation sparsity for latency

1. Start from the quantized or bf16 model; collect per-matrix activation quantiles on calibration data.
2. Set per-matrix thresholds for 40% model-wide sparsity first; allocate non-uniformly (greedy or evolutionary search) [160, 206].
3. Validate generation tasks; increase toward 50% (and 70% FFN sparsity with newer methods [207]) only if quality holds.
4. Benchmark at batch sizes 1–8 with the actual kernels; disable if serving batches exceed the useful range (Proposition 12.3).

Recipe D: 2:4 on sparse-tensor-core hardware

1. Confirm kernel support for your precision, shape and tensor-parallel layout.
2. Initialize masks with SparseGPT or Wanda (+ channel permutation); then learn masks (MaskLLM-style) or anneal (SparseForge-style), or run sparse continued pre-training with KD [62, 203, 208].
3. Validate generative reasoning explicitly; if it fails, consider milder patterns (6:8 via SlideSparse) or hybrid sparse–dense formats [64, 65].
4. Measure end-to-end speed against a 4-bit dense baseline; proceed only if the gain justifies the training and validation cost.

Table 18.2: Workload → risk profile and suggested validation emphasis.

Workload	Technique suitability	Validation emphasis
Chat / assistant (aligned)	KD-based structured pruning from the aligned teacher; light 2:4 at most	safety/refusal, bias, calibration, prompt robustness
Reasoning / long CoT	Avoid aggressive weight pruning, depth pruning, 2:4 post-training; prefer KD with on-policy data (RAC-style)	GSM8K/MATH, generated length, accuracy per token
Code generation	Expert pruning calibrated on code (REAP: near-lossless reported for code); structured + KD	HumanEval-like, repository-level tasks
RAG / retrieval-based QA	Depth pruning and width pruning more tolerable (knowledge externalized)	long-context retrieval, grounding
Classification / extraction	Depth pruning, structured pruning, distillation to small models	task-specific accuracy
Multilingual	Calibration must cover all languages; avoid expert pruning without language-balanced calibration	per-language slices
Long context	Do not prune heads without retrieval-head checks	needle and multi-document QA at target length

18.4 Pre-deployment checklist

1. Target hardware, batch regime and context length written down before method choice.
2. Baseline: best dense quantized model at equal memory, with measured latency.
3. Calibration set documented (sources, size, seeds); at least three seeds evaluated.
4. Recovery budget declared; teacher and licensing reviewed.
5. Evaluation protocol of Table 15.3 completed, including safety, bias, calibration, long context and tails.
6. End-to-end speed measured across batch sizes and context lengths on the target stack.
7. Monitoring plan in production for drift in refusal rate, output length, and task accuracy against the dense reference.
8. Rollback path to the dense model.

Table 18.3: Hardware → what sparsity can realistically deliver.

Hardware / stack	Realistic sparsity benefit
Sparse-tensor-core NVIDIA GPUs (A100 and later)	2:4 kernels (math up to 2×; end-to-end ∼1.2–1.3× reported); activation-sparse GEMV kernels (TEAL-class); structured models always
Consumer GPUs	activation sparsity and smaller dense models; sparse kernel availability varies; 2:4 supported on recent consumer generations per [64][‡]
Other GPU vendors	check vendor documentation for structured-sparsity instructions and library maturity (unverified by the author)
CPUs	unstructured sparsity kernels can pay off; activation sparsity for decode; smaller dense models
Mobile NPUs	usually dense low-bit only; smaller dense models; flash-aware activation-sparse loading in research systems
Dataflow / wafer-scale systems	vendor-claimed native unstructured sparsity (unverified)

Chapter 19

Estimated Priority in the LLM Engineering Community

Read this first: what these scores are

The scores in this chapter are the **author's subjective judgments**, informed by the evidence cited throughout this document. They are **not measured data**: no survey of practitioners, no download or deployment statistics, and no systematic bibliometric count underlie them. The author's literature sample comes from a focused set of searches in October 2026 and from prior knowledge, which are biased toward work that is well indexed and toward NVIDIA-adjacent and arXiv-centric literature. Readers who disagree should change the scores and weights; the machinery is intentionally transparent so that they can.

19.1 Rubric

Each sub-topic is scored from 1 (lowest) to 5 (highest) on five criteria. "Priority" means: how much should a team building or serving LLMs in late 2026 pay attention to this sub-topic, weighing present use, future potential and cost of getting it wrong.

Table 19.1: Rubric criteria, definitions, anchors, and base weights.

Criterion	Weight	Definition	Anchors (1 / 3 / 5)
C1 Production adoption	0.30	Evidence of use in released or deployed models and serving stacks	1: none known / 3: some released artifacts or vendor-supported paths / 5: multiple flagship releases derived by the technique
C2 Research momentum	0.15	Volume and novelty of 2025–2026 work; new results rather than incremental tuning	1: stagnant / 3: steady / 5: many new methods in 2026
C3 Hardware tailwind	0.20	Current and expected hardware/software support for realizing the saving	1: none, or adverse trend / 3: partial, needs custom kernels / 5: universal, runs on any stack
C4 Marginal benefit left	0.20	Headroom: potential further gain from additional work	1: exhausted / 3: moderate / 5: large untapped
C5 Risk-adjusted reliability	0.15	Inverse of risk: predictability, side-effect profile, validation burden (5 = low risk)	1: frequent surprises / 3: manageable with protocol / 5: low risk

The priority score is $\sum_i c_i\, w_i$ with the weights above, giving a value between 1 and 5. Production adoption has the largest weight because it is the best available evidence that a technique works under real constraints; hardware tailwind and headroom come next because they determine the future value of effort. The weights themselves are a judgment, so we report sensitivity to four alternative weightings: equal weights, adoption-heavy, research-heavy and risk-heavy (Table 19.3).

19.2 Scoring rationale

The justification below is brief; it points to the chapters and citations that provide the evidence.

Structured width pruning + distillation (5/4/5/4/3). Adoption 5: multiple flagship derived models (Minitron, Llama 3.2 1B/3B per Meta, Nemotron Nano 2, Elastic, Puzzle-derived models) [111–113, 115, 117]. Momentum 4: hybrids, elastic nesting and reasoning-aware compression in 2025–2026. Hardware 5: output is dense. Headroom 4: NAS, elastic families, reasoning-model-specific recipes. Risk 3: heavy recovery cost and tail loss, but a well-understood protocol.

MoE expert pruning/merging/skipping (3/5/4/4/3). Adoption 3: emerging but real use on very large open MoE checkpoints (the author counts this as "some released artifacts" without a verified census). Momentum 5: REAP, REAM and related in late 2025–2026 [131, 132]. Hardware 4: memory reduction benefits all GPU stacks. Headroom 4: calibration, router adaptation, reasoning models. Risk 3: calibration-domain sensitivity.

Training-free contextual activation sparsity (2/5/3/4/4). Adoption 2: kernels and research systems, few flagship deployments known. Momentum 5: TEAL, LaRoSA, WiSparse and Prox in rapid succession [160, 161, 206, 207]. Hardware 3: needs custom kernels; best at small batch. Headroom 4: higher sparsity (70% FFN reported). Risk 4: model unchanged, tunable at runtime.

Depth pruning (3/3/5/2/2). Adoption 3: used as a variant (Minitron depth models; Puzzle block no-ops). Momentum 3. Hardware 5. Headroom 2: reasoning limits [101]. Risk 2: generative degradation.

Attention-head pruning/head-level KV (2/3/4/3/3). Adoption 2 for post-hoc head pruning (GQA is architectural, not pruning). Hardware 4 for KV reduction. Risk 3 (retrieval heads).

Joint sparsity + quantization (2/3/4/3/3). Real prospects where low-precision sparse kernels exist [55, 64]; rate–distortion view cautions (Chapter 13).

Semi-structured N:M/2:4 (2/4/4/3/2). Adoption 2: few released models (Sparse-Llama-3.1-8B-2of4); Momentum 4: learned masks, SlideSparse, SpenseGPT, SparseForge, activation N:M; Hardware 4: A100→B200 support, flexible patterns requested by the community [209]; Risk 2: reasoning loss and modest speedups.

Post-training structured pruning without distillation (2/3/5/2/2). Superseded in practice by distillation-based recipes at the compression levels that matter; still useful for mild compression.

Dynamic depth (2/3/3/3/3). LayerSkip-type approaches are attractive but entangled with batching and training changes.

Induced activation sparsity (2/3/3/3/3). Works but requires continued training; squared ReLU is an architectural choice in some model families.

Sparse training/DST/sparse pre-training (1/3/2/3/3). Promising scaling-law evidence [171, 174]; little production use and no GPU wall-clock benefit for unstructured patterns.

Table 19.2: Scores on criteria C1–C5 (author's subjective judgments, not measured data), base-weight priority score, rank range across the five weightings of Table 19.3, tier (1: ≥ 3.3, 2: 2.8–3.3, 3: < 2.8), and confidence in the tier assignment. Ranks 5, 6 and 7 are tied on the base score (2.90).

#	Sub-topic	C1	C2	C3	C4	C5	Score	Rank range	Tier	Conf.
1	Structured width pruning + distillation	5	4	5	4	3	4.35	1–1	1	High
2	MoE expert pruning / merging / skipping	3	5	4	4	3	3.70	2–2	1	Med
3	Training-free contextual activation sparsity	2	5	3	4	4	3.35	3–4	1	Med
4	Depth pruning / layer dropping	3	3	5	2	2	3.05	3–7	2	Med
5	Attention-head pruning / head-level KV	2	3	4	3	3	2.90	4–5	2	Low
6	Joint sparsity + quantization	2	3	4	3	3	2.90	5–6	2	Low
7	Semi-structured N:M / 2:4 sparsity	2	4	4	3	2	2.90	4–9	2	Med
8	Post-training structured pruning (no distillation)	2	3	5	2	2	2.75	8–10	3	Med
9	Dynamic depth (early exit, layer skip)	2	3	3	3	3	2.70	7–9	3	Low
10	Induced activation sparsity (ReLU-fication etc.)	2	3	3	3	3	2.70	8–10	3	Low
11	Sparse training / dynamic / sparse pre-training	1	3	2	3	3	2.20	11–12	3	Low
12	One-shot unstructured weight pruning	2	3	1	2	3	2.10	11–12	3	Med

One-shot unstructured weight pruning (2/3/1/2/3). Scientifically central and a strong research substrate, but with no GPU hardware tailwind and little unclaimed benefit.

19.3 Scores and final ranking

19.3.1 Final ranking with confidence

1. **Structured width pruning + distillation** — robustly first under every weighting tested. *Confidence: high* (adoption evidence is verified in several flagship releases; rank is insensitive to weights).

2. **MoE expert pruning/merging/skipping** — consistently second. *Confidence: medium*: the evidence is two to three recent, largely unreplicated preprints and a trend in architecture, not a verified deployment census.

3. **Training-free contextual activation sparsity** — third or fourth. *Confidence: medium*: fast-moving, strongly dependent on whether batch-1/on-device decoding matters to the reader's workload.

4. **Depth pruning** — mid-ranked (rank 3–7 across weightings); sensitive to the adoption weight and to the risk weight. *Confidence: medium.*

5–7. **Head-level methods, joint sparsity + quantization, and N:M/2:4** — statistically

Table 19.3: Sensitivity of scores (and ranks, in parentheses) to the weighting: base (0.30/0.15/0.20/0.20/0.15), equal (0.2 each), adoption-heavy (0.45/0.10/0.20/0.10/0.15), research-heavy (0.15/0.30/0.15/0.30/0.10), risk-heavy (0.25/0.10/0.15/0.15/0.35).

Sub-topic	Base	Equal	Adoption	Research	Risk
Structured width pruning + distillation	4.35 (1)	4.20 (1)	4.50 (1)	4.20 (1)	4.05 (1)
MoE expert pruning / merging / skipping	3.70 (2)	3.80 (2)	3.50 (2)	4.05 (2)	3.50 (2)
Training-free contextual activation sparsity	3.35 (3)	3.60 (3)	3.00 (4)	3.85 (3)	3.45 (3)
Depth pruning / layer dropping	3.05 (4)	3.00 (7)	3.15 (3)	2.90 (7)	2.80 (6)
Attention-head pruning / head-level KV	2.90 (5)	3.00 (5)	2.75 (5)	3.00 (5)	2.90 (4)
Joint sparsity + quantization	2.90 (6)	3.00 (6)	2.75 (6)	3.00 (6)	2.90 (5)
Semi-structured N:M / 2:4 sparsity	2.90 (7)	3.00 (4)	2.70 (7)	3.20 (4)	2.65 (9)
Post-training structured pruning (no distillation)	2.75 (8)	2.80 (10)	2.70 (8)	2.75 (10)	2.55 (10)
Dynamic depth (early exit, layer skip)	2.70 (9)	2.80 (8)	2.55 (9)	2.85 (8)	2.75 (7)
Induced activation sparsity (ReLU-fication etc.)	2.70 (10)	2.80 (9)	2.55 (10)	2.85 (9)	2.75 (8)
Sparse training / dynamic / sparse pre-training	2.20 (11)	2.40 (11)	1.90 (12)	2.55 (11)	2.35 (11)
One-shot unstructured weight pruning	2.10 (12)	2.20 (12)	2.05 (11)	2.25 (12)	2.30 (12)

tied in the middle. *Confidence: low for ordering, medium for tier.* N:M has the widest rank range (4–9) because it is highly sensitive to the risk weight (reasoning losses) and to hardware trends.

8–10. **Post-training structured pruning without distillation, dynamic depth, induced activation sparsity** — lower tier. *Confidence: low to medium.*

11–12. **Sparse training/DST and one-shot unstructured pruning** — lowest as *deployment* priorities, ranks 11–12 under all weightings. *Confidence: medium for one-shot unstructured; low for sparse training* (a breakthrough in sparse hardware or sparse-training stability would move it).

Cross-cutting enabler: evaluation and trustworthiness of pruned models

Not a compression technique but arguably the highest-leverage investment: scored on the same rubric as a sub-topic (3/4/3/5/5, base score 3.85) it would rank second. Better evaluation (Chapter 15) raises the value of every technique above by reducing risk R4–R6 of Chapter 17. We keep it out of the ranking of techniques because it is a methodology rather than an alternative.

19.4 Interpreting and stress-testing the ranking

Robustness. The top two ranks are invariant over the five weightings; the top three tier membership is invariant. The middle tier is a near tie, and no reader should read much into the order of ranks 4–7. This is a feature of the evidence: many middle-tier techniques have comparable adoption and hardware stories, with different risks.

What would change the ranking.

- A mainstream GPU generation with flexible N:M (for example 8:16) or unstructured sparse tensor-core support would raise N:M and unstructured pruning substantially (C3 from 4 to 5 for N:M, from 1 to 3 for unstructured). The community has asked for flexible patterns [209]; the author has no information on vendor roadmaps.
- Robust methods for reasoning-model 2:4 recovery at low cost would lift N:M's C5 and adoption.
- Evidence of widespread deployment of activation-sparse kernels in major serving engines would lift activation sparsity's C1.
- A shift of serving workloads toward long reasoning chains at small batch (agentic, on-device) would favor activation sparsity and depth-latency techniques; a shift toward high-throughput batch inference would disfavor them.
- A validated, replicated result that expert pruning hurts safety or multilinguality would lower MoE pruning's C5.

Forecast, with low confidence. The author expects, over the next 12–24 months and without strong evidence, that (i) pruning-plus-distillation will remain the main route to model families, increasingly via elastic/nested training; (ii) MoE expert compression and low-bit expert quantization will converge into standard post-training toolkits for open large MoEs; (iii) activation sparsity will see wider kernel support for on-device use; and (iv) unstructured weight sparsity will remain a research tool unless hardware changes. These are opinions, not predictions backed by data.

Takeaway

If you can only invest in one area: structured pruning plus distillation (with a serious evaluation protocol). If you serve large MoEs: expert compression. If you care about small-batch decode latency: training-free activation sparsity. Treat the rest as options contingent on hardware and workload, and treat unstructured pruning as science rather than as a deployment plan.

Chapter 20

Open Problems, Research Directions, and Conclusions

20.1 Open problems

The following problems are the author's selection of questions that, if answered, would change practice. Each is stated with why it matters and what a first step might be. The "impact" and "tractability" labels are subjective.

20.1.1 P1. Reasoning-preserving compression

Pruned reasoning models may think longer and answer worse [121]; depth pruning and 2:4 post-training damage reasoning disproportionately [64, 101]. Open: (a) a theory connecting per-token distribution shift to chain-level accuracy beyond the simple compounding model of Example 17.1; (b) pruning objectives that reconstruct *trajectories* rather than single-step activations at scale; (c) criteria that account for how pruning changes the *length* distribution of the chain; (d) accuracy-per-token or accuracy-per-dollar metrics that incorporate test-time compute. *Impact: very high; tractability: medium.*

20.1.2 P2. Hardware–pattern co-design

Flexible N:M patterns (such as 8:16 or 16:32) are argued to approach unstructured quality [209], and Table 7.2 shows retained energy rising with M, yet commodity hardware supports only 2:4 (with decompositions such as SlideSparse and hybrid formats as workarounds [64, 65]). Open: the optimal trade-off between mask entropy and decoder/multiplexer cost for a given silicon budget; a rigorous cost model of sparse tensor-core area versus pattern flexibility; software patterns that approximate $M \gg 4$ using 2:4 primitives with less overhead than $N/(N-1)$-bound decompositions. *Impact: high; tractability: medium (needs vendor involvement).*

20.1.3 P3. Theory and practice of calibration

Proposition 15.1 shows that coverage governs transfer, but no principled procedure picks calibration data to cover the deployment distribution's Hessian directions. Open: sample-complexity results for estimating $\mathbf{H}$ in the dominant subspace under heavy-tailed activations; active selection of calibration sequences that maximize coverage (for instance by greedy selection to maximize the log-determinant of the aggregate covariance); calibration-free methods with guarantees [89]; methods robust to unknown deployment domains [86, 88]. *Impact: high; tractability: high.*

20.1.4 P4. Predicting the accuracy cliff

Where does the knee occur as a function of model size, data, architecture and method? Scaling laws exist for sparse pre-training [171, 172, 174] but not for *post-training* pruning accuracy as a

function of sparsity. Open: an empirical law for the knee location, ideally predicted from cheap per-layer statistics (spectral exponents [54], outlier ratios [46], effective rank of activations). *Impact: high; tractability: medium.*

20.1.5 P5. Lottery tickets at LLM scale

Are there small subnetworks of an *untrained or early-trained* LLM that, trained in isolation, match the dense model, and can they be found at acceptable cost? The strong LTH is an existence result [10]; the LLM-scale status is open (Chapter 3). A tractable first step is a study at 100M–1B scale with rewinding and measurement of linear mode connectivity [6] for pretrained transformers. *Impact: scientific; tractability: medium.*

20.1.6 P6. Controlled comparison of routes to a small model

The comparison among pruning plus distillation, from-scratch training, and sparse pre-training is confounded by data, recipe and teacher (Chapter 13). Open: carefully controlled studies at equal total compute (including teacher cost) across model scales and shrink ratios, with a focus on extreme shrinkage (e.g., 70B to 1B). *Impact: high; tractability: high but expensive.*

20.1.7 P7. MoE compression

Open: (a) resolution of the pruning-versus-merging question with common calibration, models and metrics [131, 132]; (b) joint optimization of expert pruning, merging, and expert-level quantization; (c) router re-calibration after pruning; (d) effects on multilinguality, safety and tool use; (e) dynamic, batch-aware expert skipping [133]; (f) hybrid offloading policies informed by routing statistics. *Impact: high; tractability: high.*

20.1.8 P8. Activation sparsity beyond batch size one

Proposition 12.3 shows the batch-union problem. Open: token grouping and scheduling that makes active sets overlap (sparsity-aware batching); structured activation sparsity where whole channel groups are skipped for all tokens in a micro-batch; combination with speculative decoding (verification of several tokens at once is a small batch); kernels for medium batch sizes. *Impact: medium-high; tractability: medium.*

20.1.9 P9. Unified compression of weights and KV cache

At long context, KV bytes dominate (Example 14.2). Open: joint allocation of a byte budget across weights, KV and activations; head-level heterogeneity [144]; pruning criteria that treat retrieval capability explicitly [143]. *Impact: high; tractability: medium.*

20.1.10 P10. Safety, alignment and security of pruned models

Evidence on bias and trust is thin and mixed [184, 187]. Open: mechanistic accounts of which circuits (refusal, truthfulness) pruning damages; pruning methods with safety-preservation constraints (e.g., including refusal data in the reconstruction objective); security analysis of

compression-conditioned behavior (whether a compression step can be exploited to alter model behavior), where the author is not aware of definitive work for pruning. *Impact: high; tractability: medium.*

20.1.11 P11. Non-transformer and multimodal models

Hybrid SSM–attention models need SSM-aware pruning (group-aware SSM pruning in Minitron-SSM [210]), and diffusion LLMs, vision–language models and speech models bring their own structures (visual tokens, cross-attention). Open: recipes, importance criteria and evaluation for these architectures. *Impact: medium-high; tractability: high.*

20.1.12 P12. Standards for evaluation and reporting

Open: shared calibration sets and seeds, shared hardware benchmarks (a sparsity analog of inference benchmark suites), mandatory reporting of recovery budget and end-to-end speed versus a quantized dense baseline, and community-maintained tail and safety slices. *Impact: high; tractability: high (coordination problem).*

20.1.13 P13. Sparse and elastic training at scale

SMET addresses a cold-start instability of DST [173]; sparse pre-training laws exist [171]. Open: end-to-end sparse pre-training with hardware-supported patterns and FP4-class precision; elastic/nested training with guarantees on extractable sub-models and routers that pick architectures at serving time [90, 91, 115]. *Impact: high; tractability: low-medium.*

20.1.14 P14. Better theory for second-order and global pruning

Layerwise reconstruction is exact locally but ignores cross-layer interaction (Proposition 3.4 is loose). Open: tractable global second-order objectives (Kronecker-factored or sketched Hessians), tighter error-propagation bounds that exploit residual-stream structure, and understanding of when local-objective optimality implies global quality [52]. *Impact: medium; tractability: medium.*

20.2 Conclusions

Pruning and sparsity in LLMs have matured from a question of feasibility to a question of *engineering fit*. A decade after Deep Compression and three decades after Optimal Brain Surgeon, the mathematical core is well understood: layerwise reconstruction is an exactly quadratic problem with a shared Hessian, whose OBS solution scales to 100B-parameter models through a Cholesky identity; cheap activation-aware scores are its diagonal approximation; relaxations, learned masks and allocation heuristics extend it; and rate–distortion theory warns that, for homogeneous weights, sparsity is a poor use of bits relative to quantization.

The empirical core is equally clear. Roughly half of a large model's weights can be zeroed post-training with modest perplexity loss, but this almost never yields GPU speedups; hardware supports only 2:4, which is lossy for reasoning and gives end-to-end gains near 1.2–1.3× in the

Table 20.1: Research agenda summary (subjective).

#	Problem	Impact	Tractability	Key dependency
P1	Reasoning-preserving compression	very high	medium	on-policy data, metrics
P2	Hardware–pattern co-design	high	medium	vendors
P3	Calibration theory	high	high	coverage measures
P4	Predicting the cliff	high	medium	cheap statistics
P5	LTH at LLM scale	scientific	medium	rewinding studies
P6	Routes to small models, controlled	high	high (costly)	compute
P7	MoE compression	high	high	common benchmarks
P8	Activation sparsity under batching	medium-high	medium	kernels, scheduling
P9	Unified weight + KV compression	high	medium	long-context evals
P10	Safety and security	high	medium	red-teaming
P11	Non-transformer / multimodal	medium-high	high	recipes
P12	Evaluation standards	high	high	community
P13	Sparse and elastic training	high	low–medium	hardware, optimizers
P14	Global second-order theory	medium	medium	sketching

best recent reports. The practical value of pruning comes instead from *structure*: pruning plus distillation to produce families of smaller dense models, expert reduction for large MoEs, depth and width reduction where latency matters, and training-free activation sparsity at small batch. These work because their output is either executable on any stack or because they reduce the dominant byte stream (weights in memory-bound decode; experts in MoE; columns read per token).

The deepest lesson is methodological. The quantities that pruning papers optimize and report—perplexity, short zero-shot accuracy, theoretical FLOPs—are the quantities least sensitive to the damage that matters: compounding errors in long generation, rare knowledge, tail domains, long-context retrieval, safety and bias, and real end-to-end cost. Adopting a multi-axis evaluation protocol, declaring recovery budgets, diversifying calibration data and measuring latency against a quantized dense baseline would resolve many apparent contradictions in the literature and prevent most deployment failures.

Looking forward, the highest-value work lies in reasoning-preserving compression, calibration theory, hardware–pattern co-design, MoE compression, standards for evaluation, and the unification of weight and KV-cache compression. The subjective priority ranking of Chapter 19 puts structured pruning with distillation first, MoE expert compression second and training-free activation sparsity third; unstructured one-shot pruning remains the most influential *scientific* tool and the least valuable *deployment* one on today's GPUs. The field will keep moving quickly, and the numbers in this survey, many from preprints less than a year old, should be re-checked against the latest versions before being relied upon.

Appendix A

Notation

Symbol	Meaning
$d,\ d_{\text{ff}}$	hidden (residual-stream) size; FFN intermediate width
$h_q,\ h_{kv},\ d_h$	number of query heads, key–value heads; head dimension
$L,\ V$	number of transformer blocks; vocabulary size
$P,\ N,\ D$	number of parameters; (inference) parameter count; number of training tokens
$\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$	weight matrix of a linear layer; w_i is its i-th row, $\mathbf{W}_{:,j}$ its j-th column
$\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$	calibration input activations (columns are tokens); $\mathbf{X}_{j,:}$ the j-th input channel across tokens
$\mathbf{H} = 2\mathbf{X}\mathbf{X}^\top$	layerwise Hessian of the reconstruction error (Proposition 3.3); $\mathbf{H}^{-1} = \mathbf{U}^\top\mathbf{U}$ with $\mathbf{U}$ upper triangular
$\mathbf{H}_\theta$	Hessian of the loss with respect to all parameters
$\mathbf{M} \in \{0,1\}^{d_{\text{out}} \times d_{\text{in}}}$	binary mask; $\mathbf{M} \odot \mathbf{W}$ the masked weights ($\odot$: elementwise product)
$s,\ \rho$	sparsity (fraction of zeros); density $\rho = 1 - s$
$\lVert\cdot\rVert_0,\ \lVert\cdot\rVert_1,\ \lVert\cdot\rVert_2,\ \lVert\cdot\rVert_F$	number of non-zeros; ℓ_1, ℓ_2, Frobenius norms
$\text{Top}_k(\cdot)$	keeps the k largest-magnitude entries, zeroes the rest
$N{:}M$	at most N non-zeros in each aligned group of M weights
$g,\ \hat{F}$	gradient; empirical Fisher information
ρ_q (Ch. 5)	OBS saliency $\theta_q^2/(2[\mathbf{H}^{-1}]_{qq})$ (context distinguishes from density)
S_{ij}	Wanda score $\lvert W_{ij}\rvert\,\lVert\mathbf{X}_{j,:}\rVert_2$
BI_ℓ	block influence of layer ℓ (Eq. (9.2))
$z,\ \gamma, \zeta, \beta, \log\alpha$	hard-concrete gate and its parameters (Eqs. (3.5)–(3.7))
$E,\ k,\ g_j,\ E_j$	number of experts; number of active experts; gate value and output of expert j (Ch. 11)
$\text{KL}(p\Vert q)$	Kullback–Leibler divergence
$p_T,\ p_S$	teacher and student token distributions
$B,\ B_{\text{mem}},\ F_{\text{peak}}$	batch size (Ch. 12, 14); memory bandwidth; peak compute
I	arithmetic intensity (FLOP per byte)
$\bar{N}$	average parameter count during training (Eq. (13.2))

Evidence tags. †: figure quoted from a cited source as recalled by the author, not re-verified in the preparation session. ‡: checked against the abstract or page of the cited source in October 2026. "Illustrative": synthetic data or arithmetic under stated assumptions. "Author's estimate/judgment": subjective.

Appendix B

Additional Derivations

B.1 Lazy block updates equal sequential updates

Consider a block of columns $j_0, \dots, j_0+B-1$ of a row w. By Proposition 5.1, processing column j with current value w_j and target $\tilde{w}_j$ subtracts $e_j U_{jk}$ from every later column $k > j$, where $e_j = (w_j - \tilde{w}_j)/U_{jj}$. The value w_j at the time of processing equals its original value minus the accumulated contributions $\sum_{i<j,\, i\geq j_0} e_i U_{ij}$ from earlier columns in the block (contributions from columns before the block have already been applied). Hence the sequence $e_{j_0}, \dots, e_{j_0+B-1}$ depends only on the original values within the block and on $\mathbf{U}$ restricted to the block, *not* on columns after the block. For a column k beyond the block, the total update is $-\sum_{j=j_0}^{j_0+B-1} e_j U_{jk}$, a linear function of the stored errors, which is exactly $-\mathbf{E}\,\mathbf{U}_{j_0:j_0+B,\,k}$ in matrix form with $\mathbf{E} = [e_{j_0}, \dots, e_{j_0+B-1}]$. Applying it once at the end of the block therefore gives the same result as applying it after every column, but as a single matrix–matrix product, which is far more efficient on accelerators.

B.2 Closed-form expected ℓ_0 of a hard-concrete gate

The gate is $z = \min(1, \max(0, \bar{s}))$ with $\bar{s} = s(\zeta - \gamma) + \gamma$ and $s = \sigma\big((\operatorname{logit} u + \log\alpha)/\beta\big)$, $u \sim \mathcal{U}(0,1)$. The gate is exactly zero iff $\bar{s} \leq 0$, i.e., $s \leq -\gamma/(\zeta-\gamma)$. Since $\operatorname{logit}\big(-\gamma/(\zeta-\gamma)\big) = \log(-\gamma) - \log\zeta = \log(-\gamma/\zeta)$, this is equivalent to $(\operatorname{logit} u + \log\alpha)/\beta \leq \log(-\gamma/\zeta)$, i.e., $\operatorname{logit} u \leq \beta\log(-\gamma/\zeta) - \log\alpha$. For $u \sim \mathcal{U}(0,1)$, $\operatorname{logit} u$ has the standard logistic CDF σ, so $\Pr(z=0) = \sigma\big(\beta\log(-\gamma/\zeta) - \log\alpha\big)$ and $\Pr(z\neq 0) = 1 - \Pr(z=0) = \sigma\big(\log\alpha - \beta\log(-\gamma/\zeta)\big)$, which is Eq. (3.7). (Note that $\Pr(z=0) > 0$ requires $-\gamma > 0$, which is why the stretch has $\gamma < 0$.)

B.3 Fisher information equals the Hessian of the negative log-likelihood

Let $\ell(\theta) = \log p_\theta(y|x)$. Then $\nabla^2\ell = \nabla^2 p_\theta / p_\theta - \nabla\ell\nabla\ell^\top$. Taking the expectation over $y \sim p_\theta(\cdot|x)$, the first term gives $\int \nabla^2 p_\theta \, dy = \nabla^2 \int p_\theta \, dy = \nabla^2 1 = 0$. Hence $\mathbb{E}_{y\sim p_\theta}[-\nabla^2\ell] = \mathbb{E}_{y\sim p_\theta}[\nabla\ell\nabla\ell^\top]$: the Fisher equals the expected Hessian of the negative log-likelihood under the *model's own* distribution. For labels drawn from the data, the empirical Fisher is a different quantity; it coincides with the Hessian only when the model is well-specified and the data are drawn from it.

B.4 Truncated second moments used in Table 3.1

Gaussian. $\int_t^\infty x^2\varphi(x)\,dx = \big[-x\varphi(x)\big]_t^\infty + \int_t^\infty \varphi(x)dx = t\varphi(t) + Q(t)$ by integration by parts using $\varphi'(x) = -x\varphi(x)$; doubling for both tails gives $2(t\varphi(t)+Q(t))$, and $\Pr(|x| > t) = 2Q(t) = \rho$ determines t. Total energy is 1.

Laplace. With density $\frac{1}{2}e^{-|x|}$: $\Pr(|x| > t) = e^{-t} = \rho$, so $t = -\ln\rho$; $\int_t^\infty x^2 e^{-x}dx = e^{-t}(t^2+2t+2)$; both tails with density $\frac{1}{2}$ give $e^{-t}(t^2+2t+2)$; the total energy is $\mathbb{E}x^2 = 2$, so the retained fraction is $e^{-t}(t^2+2t+2)/2$.

B.5 Expected distinct experts and batch union

For a batch of B tokens each selecting k of E experts uniformly and independently, expert j is unused by one token with probability $1-k/E$ and by all B tokens with probability $(1-k/E)^B$. By linearity of expectation the expected number of used experts is $E[1-(1-k/E)^B]$. For input-coordinate sparsity (Proposition 12.3), replace $1-k/E$ by s and consider the single column.

B.6 Average parameter count under cubic pruning

For $N(t) = N_f+(N_0-N_f)(1-u)^3$ over $u \in [0,1]$, the window average of $(1-u)^3$ is $\int_0^1(1-u)^3du = 1/4$. With dense training for fraction a of steps, the window of fraction b, and the final fraction c ($a+b+c=1$): $\bar{N} = aN_0+b\big(N_f+\frac{1}{4}(N_0-N_f)\big)+cN_f$. For $a=c=0.25$, $b=0.5$, $N_f = 0.25N_0$: $\bar{N}/N_0 = 0.25+0.5\cdot 0.4375+0.0625 = 0.53125$ (Example 13.1).

B.7 Amdahl-style end-to-end speedup

If a fraction f of decode time is spent in matmuls whose time shrinks by factor s, and the remainder is unchanged, the end-to-end speedup is $1/\big((1-f)+f/s\big)$. This yields $1.30\times$ for $(f,s) = (0.7, 1.5)$, $1.65\times$ for $(0.9, 1.78)$ and $1.20\times$ for $(0.5, 1.5)$, as used in Chapter 7.

B.8 Why the $(2N-2)$:$2N$ decomposition has speedup bound $N/(N-1)$

A block of $2N$ weights with at most $2N-2$ non-zeros is covered by $N-1$ windows of 4 consecutive positions that overlap by 2 (window i covers positions $2i,\ldots,2i+3$), which sparse hardware executes as $N-1$ operations on 2:4 operands, each doing 2 multiplications: $2(N-1)$ multiplications versus $2N$ dense. The speedup bound is $2N/(2(N-1)) = N/(N-1)$, e.g., $4/3$ for 6:8 ($N=4$) [64]. The feasibility of covering every such block by 2:4-compliant windows is the content of the decomposition result in the cited paper; the arithmetic of the bound is as above.

B.9 Merging two experts

See Proposition 11.2; a corollary is that for a router whose mixing ratio $r(x)$ has variance σ_r^2 and experts with $\|a - b\| \approx \delta$ on routed tokens, the best constant-coefficient merge has root-mean-square error $\approx (g_a + g_b)\,\sigma_r\,\delta$, which grows linearly in both the router's variability and the experts' functional distance.

Appendix C

Glossary

Activation sparsity Fraction of (near-)zero entries in an activation vector, or the deliberate skipping of computation on small activations.

Arithmetic intensity FLOPs performed per byte moved from memory; compares to the ridge point of the roofline model.

Attention sink Early tokens that attract disproportionate attention; relevant to streaming KV-cache policies.

Block influence (BI) One minus the average cosine similarity between a block's input and output hidden states; a layer-importance score.

Calibration data Small dataset passed through the model to compute statistics (Hessians, activation norms, routing frequencies) for post-training compression.

Compensation Updating retained weights to offset the effect of removed ones (OBS, SparseGPT).

Contextual sparsity Existence of small, input-dependent subsets of heads or neurons that suffice for a layer's output.

Cholesky factorization Decomposition $A = \mathbf{U}^\top \mathbf{U}$ of a positive-definite matrix into triangular factors.

Damping Adding λI to a Hessian to make it invertible and stabilize updates.

Distillation (knowledge) Training a student to match a teacher's output distribution, typically by KL divergence on logits.

Dynamic sparse training (DST) Training with a fixed sparsity budget while changing the mask by drop-and-grow rules.

Early exit Stopping computation at an intermediate layer when a confidence criterion is met.

Elastic (nested) model A model containing several extractable sub-models that share weights.

Empirical Fisher Average outer product of per-example gradients; a PSD proxy for the Hessian.

ERK Erdős–Rényi-Kernel layerwise density allocation used in sparse training.

GQA / MQA Grouped-/multi-query attention: several query heads share key–value heads.

Hard-concrete gate Stretched, clipped Concrete random variable giving differentiable ℓ_0 regularization.

Hessian Matrix of second derivatives; for layerwise reconstruction equals $2\mathbf{X}\mathbf{X}^\top$.

Intrinsic dimension Minimal dimension of a random subspace in which a target performance can be reached.

IMP Iterative magnitude pruning, the procedure used to find lottery tickets.

KV cache Stored attention keys and values of past tokens, size $2Lh_{kv}d_h$ per token.

Lottery ticket hypothesis Dense networks contain sparse subnetworks that train to similar accuracy from the original initialization.

Magnitude pruning Removing the smallest-$|w|$ weights.

MaskLLM Learnable 2:4 masks via Gumbel-softmax over the six candidate patterns.

Minitron NVIDIA recipe for pruning plus distillation of LLMs into smaller models.

Mixture-of-Experts (MoE) Layer with many experts and a router that activates a few per token.

N:M **sparsity** At most N non-zeros in each aligned group of M weights.

OBD / OBS Optimal Brain Damage / Surgeon: second-order pruning criteria; OBS includes compensation.

OWL Outlier-weighted layerwise sparsity allocation.

Perplexity Exponential of average per-token cross-entropy.

Predictor (Deja Vu) Small network predicting which heads/neurons to compute.

RIA Relative importance and activations score with channel permutation.

RigL Dynamic sparse training that regrows connections by gradient magnitude.

Roofline model Performance bound $\min(F_{\text{peak}}, B_{\text{mem}} I)$.

Router Gating network in an MoE layer.

SliceGPT Structured pruning of the hidden dimension after an orthogonal rotation (PCA).

Sparse tensor core Hardware unit multiplying a 2:4-compressed operand by a dense operand.

SparseGPT One-shot second-order layerwise pruning using sequential OBS updates via Cholesky.

Straight-through estimator (STE) Gradient approximation passing gradients through a non-differentiable operation.

Super weight Individually critical weight whose removal collapses model quality.

TEAL Training-free magnitude-based activation sparsity across all matrices.

Teacher correction Light fine-tuning of the teacher on the distillation data before distillation.

Wanda Weight×activation-norm pruning score with per-row comparison group.

Bibliography

[1] Y. LeCun, J. S. Denker, S. A. Solla. Optimal brain damage. *NeurIPS (NIPS) 2*, 1989/1990.

[2] B. Hassibi, D. G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. *NIPS 5*, 1992/1993.

[3] S. Han, J. Pool, J. Tran, W. J. Dally. Learning both weights and connections for efficient neural networks. *NeurIPS*, 2015.

[4] S. Han, H. Mao, W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *ICLR*, 2016.

[5] J. Frankle, M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *ICLR*, 2019.

[6] J. Frankle, G. K. Dziugaite, D. Roy, M. Carbin. Linear mode connectivity and the lottery ticket hypothesis. *ICML*, 2020.

[7] J. Frankle, G. K. Dziugaite, D. Roy, M. Carbin. Pruning neural networks at initialization: Why are we missing the mark? *ICLR*, 2021.

[8] H. Zhou, J. Lan, R. Liu, J. Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. *NeurIPS*, 2019.

[9] V. Ramanujan et al. What's hidden in a randomly weighted neural network? *CVPR*, 2020.

[10] E. Malach, G. Yehudai, S. Shalev-Shwartz, O. Shamir. Proving the lottery ticket hypothesis: Pruning is all you need. *ICML*, 2020.

[11] T. Chen et al. The lottery ticket hypothesis for pre-trained BERT networks. *NeurIPS*, 2020.

[12] S. Prasanna, A. Rogers, A. Rumshisky. When BERT plays the lottery, all tickets are winning. *EMNLP*, 2020.

[13] J. Su et al. Sanity-checking pruning methods: Random tickets can win the jackpot. *NeurIPS*, 2020.

[14] U. Evci, Y. A. Ioannou, C. Keskin, Y. Dauphin. Gradient flow in sparse neural networks and how lottery tickets win. *AAAI*, 2022.

[15] C. Li, H. Farkhoor, R. Liu, J. Yosinski. Measuring the intrinsic dimension of objective landscapes. *ICLR*, 2018.

[16] A. Aghajanyan, S. Gupta, L. Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *ACL*, 2021.

[17] S. Arora, R. Ge, B. Neyshabur, Y. Zhang. Stronger generalization bounds for deep nets via a compression approach. *ICML*, 2018.

[18] C. Louizos, M. Welling, D. P. Kingma. Learning sparse neural networks through L_0 regularization. *ICLR*, 2018.

[19] C. J. Maddison, A. Mnih, Y. W. Teh. The Concrete distribution: A continuous relaxation of discrete random variables. *ICLR*, 2017.

[20] E. Jang, S. Gu, B. Poole. Categorical reparameterization with Gumbel-softmax. *ICLR*, 2017.

[21] R. Tibshirani. Regression shrinkage and selection via the lasso. *J. Royal Statistical Society B*, 58(1), 1996.

[22] D. L. Donoho. Compressed sensing. *IEEE Trans. Information Theory*, 52(4), 2006.

[23] E. J. Candès, J. Romberg, T. Tao. Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information. *IEEE Trans. Information Theory*, 52(2), 2006.

[24] B. K. Natarajan. Sparse approximate solutions to linear systems. *SIAM J. Computing*, 24(2), 1995.

[25] T. Blumensath, M. E. Davies. Iterative hard thresholding for compressed sensing. *Applied and Computational Harmonic Analysis*, 27(3), 2009.

[26] M. Zhu, S. Gupta. To prune, or not to prune: Exploring the efficacy of pruning for model com-

pression. arXiv:1710.01878, 2017.

[27] T. Gale, E. Elsen, S. Hooker. The state of sparsity in deep neural networks. arXiv:1902.09574, 2019.

[28] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, A. Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *JMLR*, 22, 2021.

[29] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, J. Guttag. What is the state of neural network pruning? *MLSys*, 2020.

[30] Z. Liu, M. Sun, T. Zhou, G. Huang, T. Darrell. Rethinking the value of network pruning. *ICLR*, 2019.

[31] P. Molchanov, S. Tyree, T. Karras, T. Aila, J. Kautz. Pruning convolutional neural networks for resource efficient inference. *ICLR*, 2017.

[32] P. Molchanov et al. Importance estimation for neural network pruning. *CVPR*, 2019.

[33] L. Theis et al. Faster gaze prediction with dense networks and Fisher pruning. arXiv:1801.05787, 2018.

[34] N. Lee, T. Ajanthan, P. Torr. SNIP: Single-shot network pruning based on connection sensitivity. *ICLR*, 2019.

[35] S. P. Singh, D. Alistarh. WoodFisher: Efficient second-order approximation for neural network compression. *NeurIPS*, 2020.

[36] E. Frantar, D. Alistarh. Optimal Brain Compression: A framework for accurate post-training quantization and pruning. *NeurIPS*, 2022.

[37] E. Kurtić et al. The Optimal BERT Surgeon: Scalable and accurate second-order pruning for large language models. *EMNLP*, 2022.

[38] V. Sanh, T. Wolf, A. M. Rush. Movement pruning: Adaptive sparsity by fine-tuning. *NeurIPS*, 2020.

[39] H. Cheng, M. Zhang, J. Q. Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE TPAMI*, 2024 (arXiv:2308.06767).

[40] X. Zhu, J. Li, Y. Liu, C. Ma, W. Wang. A survey on model compression for large language models. *TACL*, 2024 (arXiv:2308.07633).

[41] W. Wang et al. Model compression and efficient inference for large language models: A survey. arXiv:2402.09748, 2024.

[42] E. Frantar, D. Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. *ICML*, 2023.

[43] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*, 2023.

[44] M. Sun, Z. Liu, A. Bair, J. Z. Kolter. A simple and effective pruning approach for large language models (Wanda). *ICLR*, 2024.

[45] Y. Zhang et al. Plug-and-play: An efficient post-training pruning method for large language models (RIA). *ICLR*, 2024.

[46] L. Yin et al. Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity. *ICML*, 2024.

[47] Y. Zhang et al. Dynamic sparse no training: Training-free fine-tuning for sparse LLMs. *ICLR*, 2024.

[48] R. J. Das, M. Ma, R. Dutta et al. Beyond size: How gradients shape pruning decisions in large language models (GBLM-Pruner). arXiv:2311.04902, 2023.

[49] P. Dong et al. Pruner-Zero: Evolving symbolic pruning metric from scratch for large language models. *ICML*, 2024.

[50] V. Boza. Fast and effective weight update for pruned large language models. *TMLR*, 2024 (arXiv:2401.02938).

[51] X. Meng, K. Behdin, H. Wang, R. Mazumder. ALPS: Improved optimization for highly sparse one-shot pruning for large language models. *NeurIPS*, 2024.

[52] G. Bai et al. SparseLLM: Towards global pruning of pre-trained language models. *NeurIPS*, 2024.

[53] P. Xu et al. BESA: Pruning large language models with blockwise parameter-efficient sparsity allocation. *ICLR*, 2024.

[54] H. Lu et al. AlphaPruning: Using heavy-tailed self regularization theory for improved layer-wise pruning of large language models. *NeurIPS*, 2024.

[55] M. Mozaffari, A. Yazdanbakhsh, M. Mehri Dehnavi. SLiM: One-shot quantized sparse plus low-rank approximation of LLMs. *ICML*, 2025 (arXiv:2410.09615).

[56] Y. Li et al. LoSparse: Structured compression of large language models based on low-rank and sparse approximation. *ICML*, 2023.

[57] E. Kurtić, E. Frantar, D. Alistarh. ZipLM: Inference-aware structured pruning of language models. *NeurIPS*, 2023.

[58] A. Mishra et al. Accelerating sparse deep neural networks. arXiv:2104.08378, 2021.

[59] J. Pool, C. Yu. Channel permutations for N:M sparsity. *NeurIPS*, 2021.

[60] A. Zhou et al. Learning N:M fine-grained structured sparse neural networks from scratch. *ICLR*, 2021.

[61] I. Hubara et al. Accelerated sparse neural training: A provable and efficient method to find N:M transposable masks. *NeurIPS*, 2021.

[62] G. Fang et al. MaskLLM: Learnable semi-structured sparsity for large language models. *NeurIPS*, 2024 (arXiv:2409.17481).

[63] D. Haziza et al. Accelerating transformer inference and training with 2:4 activation sparsity. arXiv:2503.16672, 2025.

[64] H. Shao et al. SlideSparse: Fast and flexible (2N-2):2N structured sparsity. arXiv:2603.05232, 2026 (ICML 2026).

[65] J. Lee, S.-w. Hwang, S. Rajbhandari. SpenseGPT: Practical one-shot pruning enabling sparse and dense GEMMs for LLM inference. arXiv:2606.10445, 2026.

[66] E. Frantar et al. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. arXiv:2408.11743, 2024.

[67] H. Xia et al. Flash-LLM: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *PVLDB*, 2024 (arXiv:2309.10285).

[68] T. Gale, M. Zaharia, C. Young, E. Elsen. Sparse GPU kernels for deep learning. *SC*, 2020.

[69] S. Gray, A. Radford, D. P. Kingma. GPU kernels for block-sparse weights. OpenAI technical report, 2017.

[70] NVIDIA. NVIDIA A100 Tensor Core GPU architecture whitepaper, 2020.

[71] NVIDIA. cuSPARSELt documentation (structured sparse matrix multiplication library), accessed 2026.

[72] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[73] X. Ma, G. Fang, X. Wang. LLM-Pruner: On the structural pruning of large language models. *NeurIPS*, 2023.

[74] S. Ashkboos et al. SliceGPT: Compress large language models by deleting rows and columns. *ICLR*, 2024.

[75] M. Xia, T. Gao, Z. Zeng, D. Chen. Sheared LLaMA: Accelerating language model pre-training via structured pruning. *ICLR*, 2024.

[76] M. Xia, Z. Zhong, D. Chen. Structured pruning learns compact and accurate models (CoFi). *ACL*, 2022.

[77] Z. Wang, J. Wohlwend, T. Lei. Structured pruning of large language models. *EMNLP*, 2020.

[78] F. Lagunas et al. Block pruning for faster transformers. *EMNLP*, 2021.

[79] W. Kwon et al. A fast post-training pruning framework for transformers. *NeurIPS*, 2022.

[80] Y. An et al. Fluctuation-based adaptive structured pruning for large language models (FLAP). *AAAI*, 2024.

[81] L. Dery, S. Kolawole, J.-F. Kagy, V. Smith, G. Neubig, A. Talwalkar. Everybody prune now:

Structured pruning of LLMs with only forward passes (Bonsai). arXiv:2402.05406, 2024.

[82] G. Fang et al. DepGraph: Towards any structural pruning. *CVPR*, 2023.

[83] S. Guo et al. Compresso: Structured pruning with collaborative prompting learns compact large language models. arXiv:2310.05015, 2023.

[84] M. Zhang et al. LoRAPrune: Structured pruning meets low-rank parameter-efficient fine-tuning. *Findings of ACL*, 2024.

[85] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[86] G. Wu et al. Iterative structured pruning for large language models with multi-domain calibration. arXiv:2601.02674, 2026.

[87] D. Coello de Portugal Mecke, T. Hanika, L. Schmidt-Thieme. Prune, Update and Trim: Robust structured pruning for large language models (Putri). arXiv:2605.18331, 2026.

[88] X. Liu et al. GPrune-LLM: Generalization-aware structured pruning for large language models. arXiv:2603.13418, 2026.

[89] S. Behera et al. LILA: Calibration-free structured pruning of large language models via latent spectral geometry. arXiv:2609.11163, 2026.

[90] Devvrit et al. MatFormer: Nested transformer for elastic inference. *NeurIPS*, 2024 (arXiv:2310.07707).

[91] R. Cai et al. Flextron: Many-in-one flexible large language model. *ICML*, 2024.

[92] A. Fan, E. Grave, A. Joulin. Reducing transformer depth on demand with structured dropout. *ICLR*, 2020.

[93] H. Sajjad et al. On the effect of dropping layers of pre-trained transformer models. *Computer Speech and Language*, 2023.

[94] X. Men et al. ShortGPT: Layers in large language models are more redundant than you expect. arXiv:2403.03853, 2024.

[95] A. Gromov, K. Tirumala, H. Shapourian, P. Glorioso, D. A. Roberts. The unreasonable ineffectiveness of the deeper layers. arXiv:2403.17887, 2024 (ICLR 2025).

[96] B.-K. Kim et al. Shortened LLaMA: Depth pruning for large language models with comparison of retraining methods. arXiv:2402.02834, 2024.

[97] J. Song et al. SLEB: Streamlining LLMs through redundancy verification and elimination of transformer blocks. *ICML*, 2024.

[98] Y. Yang, Z. Cao, H. Zhao. LaCo: Large language model pruning via layer collapse. *Findings of EMNLP*, 2024.

[99] X. Chen et al. Streamlining redundant layers to compress large language models (LLM-Streamline). *ICLR*, 2025.

[100] S. A. Siddiqui et al. A deeper look at depth pruning of LLMs. arXiv:2407.16286, 2024.

[101] S. Shrestha, A. Shrestha, A. Nepal, M. Kim, K. Ross. On the limits of layer pruning for generative reasoning in large language models. arXiv:2602.01997, 2026.

[102] T. Schuster et al. Confident adaptive language modeling. *NeurIPS*, 2022.

[103] M. Elhoushi et al. LayerSkip: Enabling early exit inference and self-speculative decoding. *ACL*, 2024.

[104] D. Raposo et al. Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. arXiv:2404.02258, 2024.

[105] J. Xin et al. DeeBERT: Dynamic early exiting for accelerating BERT inference. *ACL*, 2020.

[106] J. Zhang et al. Draft & Verify: Lossless large language model acceleration via self-speculative decoding. *ACL*, 2024.

[107] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. *ICML*, 2023.

[108] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[109] V. Sanh et al. DistilBERT, a distilled version of BERT. arXiv:1910.01108, 2019.

[110] R. Agarwal et al. On-policy distillation of language models: Learning from self-generated mistakes. *ICLR*, 2024.

[111] S. Muralidharan et al. Compact language models via pruning and knowledge distillation. *NeurIPS*, 2024 (arXiv:2407.14679).

[112] S. Turuvekere Sreenivas et al. LLM pruning and distillation in practice: The Minitron approach. arXiv:2408.11796, 2024.

[113] NVIDIA. NVIDIA Nemotron Nano 2: An accurate and efficient hybrid Mamba-Transformer reasoning model. arXiv:2508.14444, 2025.

[114] NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. arXiv:2504.03624, 2025.

[115] A. Taghibakhshi et al. Nemotron Elastic: Towards efficient many-in-one reasoning LLMs. arXiv:2511.16664, 2025.

[116] V. Thangarasa et al. Self-data distillation for recovering quality in pruned large language models. arXiv:2410.09982, 2024.

[117] Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models (blog post, 25 Sep 2024).

[118] A. Grattafiori et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[119] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

[120] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[121] R. Lucas, K. Behdin, Z. Wang, Q. Song, S. Tang, R. Mazumder. Reasoning models can be accurately pruned via chain-of-thought reconstruction. *NeurIPS*, 2025 (arXiv:2509.12464).

[122] R. Liu et al. Quantization hurts reasoning? An empirical study on quantized reasoning models. arXiv:2504.04823, 2025.

[123] N. Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017.

[124] W. Fedus, B. Zoph, N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23, 2022.

[125] A. Q. Jiang et al. Mixtral of experts. arXiv:2401.04088, 2024.

[126] D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *ACL*, 2024.

[127] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.

[128] Kimi Team. Kimi K2: Open agentic intelligence. arXiv:2507.20534, 2025.

[129] GLM-4.5 Team. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. arXiv:2508.06471, 2025.

[130] OpenAI. gpt-oss-120b and gpt-oss-20b model card. arXiv:2508.10925, 2025.

[131] M. Lasby et al. REAP the experts: Why pruning prevails for one-shot MoE compression. arXiv:2510.13999, 2025 (ICLR 2026).

[132] S. Jha et al. REAM: Merging improves pruning of experts in LLMs. arXiv:2604.04356, 2026.

[133] X. Lu et al. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. *ACL*, 2024.

[134] P. Li et al. Merge, then compress: Demystify efficient SMoE with hints from its routing policy (MC-SMoE). *ICLR*, 2024.

[135] A. Muzio, A. Sun, C. He. SEER-MoE: Sparse expert efficiency through regularization for mixture-of-experts. arXiv:2404.05089, 2024.

[136] T. Chen et al. Task-specific expert pruning for sparse mixture-of-experts. arXiv:2206.00277, 2022.

[137] Y. Xie et al. MoE-Pruner: Pruning mixture-of-experts large language model using the hints from its router. arXiv:2410.12013, 2024.

[138] Z. Zhang et al. MoEfication: Transformer feed-forward layers are mixtures of experts. *Findings of ACL*, 2022.

[139] P. Michel, O. Levy, G. Neubig. Are sixteen heads really better than one? *NeurIPS*, 2019.

[140] E. Voita et al. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *ACL*, 2019.

[141] J. Ainslie et al. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023.

[142] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150, 2019.

[143] W. Wu et al. Retrieval head mechanistically explains long-context factuality. arXiv:2404.15574, 2024.

[144] G. Xiao et al. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. arXiv:2410.10819, 2024.

[145] G. Xiao et al. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[146] Z. Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

[147] M. Sun, X. Chen, J. Z. Kolter, Z. Liu. Massive activations in large language models. *COLM*, 2024.

[148] M. Yu et al. The super weight in large language models. arXiv:2411.07191, 2024.

[149] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *NeurIPS*, 2022.

[150] D. Dai et al. Knowledge neurons in pretrained transformers. *ACL*, 2022.

[151] M. Geva, R. Schuster, J. Berant, O. Levy. Transformer feed-forward layers are key-value memories. *EMNLP*, 2021.

[152] R. Child et al. Generating long sequences with sparse transformers. arXiv:1904.10509, 2019.

[153] Z. Liu et al. Deja Vu: Contextual sparsity for efficient LLMs at inference time. *ICML*, 2023.

[154] I. Mirzadeh et al. ReLU strikes back: Exploiting activation sparsity in large language models. *ICLR*, 2024.

[155] Z. Zhang et al. ReLU^2 wins: Discovering efficient activation functions for sparse LLMs. arXiv:2402.03804, 2024.

[156] C. Song et al. ProSparse: Introducing and enhancing intrinsic activation sparsity within large language models. arXiv:2402.13516, 2024.

[157] K. Alizadeh et al. LLM in a flash: Efficient large language model inference with limited memory. *ACL*, 2024.

[158] Y. Song et al. PowerInfer: Fast large language model serving with a consumer-grade GPU. *SOSP*, 2024.

[159] D. Lee et al. CATS: Contextually-aware thresholding for sparsity in large language models. *COLM*, 2024 (arXiv:2404.08763).

[160] J. Liu et al. Training-free activation sparsity in large language models (TEAL). *ICLR*, 2025 (arXiv:2408.14690).

[161] K. Liu et al. La RoSA: Enhancing LLM efficiency via layerwise rotated sparse activation. *ICML*, 2025 (arXiv:2507.01299).

[162] H. Wang et al. Q-Sparse: All large language models can be fully sparsely-activated. arXiv:2407.10969, 2024.

[163] Y. Luo et al. Sparsing law: Towards large language models with greater activation sparsity. arXiv:2411.02335, 2024.

[164] M. Kurtz et al. Inducing and exploiting activation sparsity for fast inference on deep neural networks. *ICML*, 2020.

[165] D. C. Mocanu et al. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science (SET). *Nature Communications*, 9, 2018.

[166] G. Bellec, D. Kappel, W. Maass, R. Legenstein. Deep rewiring: Training very sparse deep networks. *ICLR*, 2018.

[167] H. Mostafa, X. Wang. Parameter efficient training of deep convolutional neural networks by dynamic sparse reparameterization. *ICML*, 2019.

[168] T. Dettmers, L. Zettlemoyer. Sparse networks from scratch: Faster training without losing performance. arXiv:1907.04840, 2019.

[169] U. Evci, T. Gale, J. Menick, P. S. Castro, E. Elsen. Rigging the lottery: Making all tickets winners (RigL). *ICML*, 2020.

[170] S. Liu et al. Do we actually need dense over-parameterization? In-time over-parameterization in sparse training. *ICML*, 2021.

[171] T. Jin et al. The journey matters: Average parameter count over pre-training unifies sparse and dense scaling laws. *ICLR*, 2025 (arXiv:2501.12486).

[172] E. Frantar et al. Scaling laws for sparsely-connected foundation models. *ICLR*, 2024.

[173] Q. Xiao et al. Memory-efficient LLM training with dynamic sparsity: From stability to practical scaling. arXiv:2606.00888, 2026 (ICML 2026).

[174] B. Wu et al. When data is scarce: Scaling sparse language models with repeated training. arXiv:2606.01155, 2026.

[175] J. Kaplan et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.

[176] J. Hoffmann et al. Training compute-optimal large language models. arXiv:2203.15556, 2022.

[177] J. Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*, 2024.

[178] G. Xiao et al. SmoothQuant: Accurate and efficient post-training quantization for large language models. *ICML*, 2023.

[179] T. Dettmers et al. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *ICLR*, 2024.

[180] A. Jaiswal et al. Compressing LLMs: The truth is rarely pure and never simple (LLM-KICK). *ICLR*, 2024.

[181] S. Hooker, A. Courville, G. Clark, Y. Dauphin, A. Frome. What do compressed deep neural networks forget? arXiv:1911.05248, 2019.

[182] S. Hooker, N. Moorosi, G. Clark, S. Bengio, E. Denton. Characterising bias in compressed models. arXiv:2010.03058, 2020.

[183] C. Tran et al. Pruning has a disparate impact on model accuracy. *NeurIPS*, 2022.

[184] J. Hong et al. Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression. *ICML*, 2024.

[185] M. Williams, N. Aletras. How does calibration data affect the post-training pruning and quantization of large language models? arXiv:2311.09755, 2023.

[186] A. Bandari et al. Is C4 dataset optimal for pruning? An investigation of calibration data for LLM pruning. *EMNLP*, 2024.

[187] P. K. Rath, R. Maliakkal. Weight pruning amplifies bias: A multi-method study of compressed LLMs for edge AI. arXiv:2605.08137, 2026.

[188] L. Gao et al. A framework for few-shot language model evaluation (lm-evaluation-harness). Zenodo, 2021.

[189] D. Hendrycks et al. Measuring massive multitask language understanding. *ICLR*, 2021.

[190] K. Cobbe et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[191] M. Chen et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

[192] S. Merity et al. Pointer sentinel mixture models. *ICLR*, 2017.

[193] C. Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21, 2020.

[194] H. Touvron et al. LLaMA: Open and efficient foundation language models. arXiv:2302.13971, 2023.

[195] H. Touvron et al. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288, 2023.

[196] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[197] C. Zhang et al. Understanding deep learning requires rethinking generalization. *ICLR*, 2017.

[198] A. Bercovich et al. Puzzle: Distillation-based NAS for inference-optimized LLMs.

arXiv:2411.19146, 2024.

[199] A. Bercovich et al. Llama-Nemotron: Efficient reasoning models. arXiv:2505.00949, 2025.

[200] Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118, 2024.

[201] T. M. Cover, J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[202] E. Kurtić, D. Marques, S. Pandit, M. Kurtz, D. Alistarh. Sparse fine-tuning for inference acceleration of large language models. arXiv:2310.06927, 2023.

[203] Neural Magic / Red Hat AI. Sparse-Llama-3.1-8B-2of4 (model card, Hugging Face), accessed October 2026.

[204] S. Han et al. EIE: Efficient inference engine on compressed deep neural network. *ISCA*, 2016.

[205] Z. Allen-Zhu, Y. Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. arXiv:2404.05405, 2024.

[206] L. Chen, Y. Meng, X. Zhan, Z. Wang, W. Zhu. WiSparse: Boosting LLM inference efficiency with weight-aware mixed activation sparsity. arXiv:2602.14452, 2026.

[207] J. Liu et al. Prox: Training-free FFN activation sparsity via approximate intermediate-channel salience in LLMs. arXiv:2607.27591, 2026.

[208] H. Liu et al. SparseForge: Efficient semi-structured LLM sparsification via annealing of Hessian-guided soft-mask. arXiv:2605.06402, 2026.

[209] S. Alanova et al. Motivating next-gen accelerators with flexible (N:M) activation sparsity via benchmarking lightweight post-training sparsification approaches. arXiv:2509.22166, 2025.

[210] A. Taghibakhshi et al. Minitron-SSM: Efficient hybrid language model compression through group-aware SSM pruning. *NeurIPS*, 2025 (arXiv:2504.11409).